

Modeling and Control of Dynamical Systems with Reservoir Computing

DISSERTATION

Presented in Partial Fullfillment of the Requirements for the Degree Doctor of
Philosophy in the Graduate School of The Ohio State University

BY

DANIEL CANADAY, MS
GRADUATE PROGRAM IN PHYSICS

THE OHIO STATE UNIVERSITY

2019

COMMITTEE MEMBERS:

DANIEL J. GAUTHIER, ADVISER

GREGORY LAFYATIS

RICHARD FURNSTAHL

MIKHAIL BELKIN

Copyright by Daniel Canaday

2019

Abstract

There is currently great interest in applying artificial neural networks to a host of commercial and industrial tasks. Such networks with a layered, feedforward structure are currently deployed in technologies ranging from facial recognition software to self-driving cars. They are favored by a large portion of machine learning experts for a number of reasons. Namely: they possess a documented ability to generalize to unseen data and handle large data sets; there exists a number of well-understood training algorithms and integrated software packages for implementing them; and they have rigorously proven expressive power making them capable of approximating any bounded, static map arbitrarily well.

Within the last couple of decades, reservoir computing has emerged as a method for training a different type of artificial neural network known as a recurrent neural network. Unlike layered, feedforward neural networks, recurrent neural networks are non-trivial dynamical systems that exhibit time-dependence and dynamical memory. In addition to being more biologically plausible, they more naturally handle time-dependent tasks such as predicting the load on an electrical grid or efficiently controlling a complicated industrial process. Fully-trained recurrent neural networks have high expressive power and are capable of emulating broad classes of dynamical systems. However, despite many recent insights, reservoir computing remains relatively young as a field. It remains unclear what fundamental properties yield a well-performing reservoir computer. In practice, this results in their design being left to domain experts, despite the actual training process being remarkably simple to implement.

In this thesis, I describe a number of numerical and experimental results that expand the understanding and application of reservoir computing techniques. I develop an algorithm for controlling unknown dynamical systems with layers of reservoir computers. I demonstrate this algorithm by stabilizing a range of complex behavior in simulated Lorenz and Mackey-Glass systems. I additionally control an experimental, chaotic circuit with fast fluctuations. Using my technique, I demonstrate control within the measured noise level for some trajectories.

This control algorithm is executed on a lightweight, readily-available platform with a 1 MHz closed-loop controller.

I also develop a reservoir computing scheme with autonomous, Boolean networks capable of processing complex, real-valued data. I show that this system is capable of emulating, in real time, a benchmark chaotic time-series with high precision and a record-breaking speed of 160 million predictions per second.

Finally, I present a technique for obtaining efficient, low dimensional reservoir computers. I demonstrate with numerical examples that the efficient reservoir computers can predict a benchmark time-series more accurately than standard reservoir computers 25 times larger. Through a linear analysis, I find that these efficient reservoirs prefer specific topologies over the random, unstructured reservoir computers that are currently standard.

Dedication

This thesis is dedicated to my parents, my sister, and my wife.

Acknowledgements

Although the results presented in this thesis are my own, none of it would be possible without the professional collaboration and personal support of many people.

I would first like to acknowledge the support and guidance of my advisor, Prof. Daniel J. Gauthier. I have greatly benefited from his wide expertise, his ability to communicate clearly, and his willingness to engage with students such as myself. He has taught me through example the importance of having excellent presentation and networking skills, some of which I hope have rubbed off on me these past several years.

I would like to also acknowledge the many useful scientific discussions with our many collaborators, including Prof. Edward Ott, Prof. Brian Hunt, Prof. Michelle Girvan, and Dr. Andrew Pomerance. These interactions helped clarify many important and difficult concepts for me, as well as seed the ideas that became the projects discussed in this thesis.

I would like to thank the support of my committee members Prof. Greg Lafyatis, Prof. Richard Furnstahl, and Prof. Mikhail Belkin. They have all been helpful in navigating the candidacy and defense processes. I appreciate their thoughtful questions during our meetings and their willingness to take the time to read my thesis.

I would like to also thank the support of Prof. Nandini Trivedi, Prof. Yuan-Ming Lu, and Prof. Lou DiMauro, who have all advised me at some point in my academic career at The Ohio State University. I would also like to acknowledge Kris Dunlap, who was always willing to answer my many questions throughout graduate school.

I want to thank my previous and current office-mates—particularly Kathryn Nicolich and Taimur Islam—who helped break up my workday with interesting conversations, as well as provided emotional support through our shared graduate school experience.

I also want to thank my house-mates Michael Darcy, Brendan McCullian, and Noah Charles for all of their support. I am very lucky to have made such good friends in graduate school.

Most importantly, I want to thank my family for their unwavering love and support. My parents Cheryl Canaday and Marcus Canaday have always been my most vocal supporters, and for that I am forever grateful. Visits from my sister Emily Canaday are always wonderful. My wife Alexandra Cisek has provided constant emotional support that has been critical to making it through to graduation.

Finally, I gratefully acknowledge the financial support of U.S. Army Research Office Grant No. W911NF-12-1-0099, the Army STTR Program Office Contract No. W31P4Q-19-C-0014, Potomac Research, LLC, and The Ohio State University.

Vita

Bachelor of Science, Mathematics and Physics2010-2014
The Ohio State University

Master of Science, Physics 2014-2017
The Ohio State University

Data Science Internship 2019
Potomac Research, LLC

Publications

D. Canaday, A. Griffith, and D.J. Gauthier, 'Rapid Time Series Prediction with a Hardware-Based Reservoir Computer,' *Chaos* **28**, 123119 (2018).

Field of Study

Major Field: Physics

Contents

Abstract	iii
Dedication	v
Acknowledgements	vi
Vita	viii
List of Figures	xiii
List of Tables	xxiv
1 Introduction	1
1.1 Novel Contribution and Outline	4
2 Foundations of Reservoir Computing	8
2.1 Dynamical Systems	8
2.1.1 Types of Dynamical Systems	10
2.1.2 Delay Embedding	11
2.2 Machine Learning	12
2.2.1 Performance Measures	13
2.2.2 Hyperparameters	14
2.3 Artificial Neural Networks	14
2.3.1 Feedforward ANNs	15

2.3.2	Training	18
2.3.3	The Problem of RNNs	18
2.4	The Reservoir Computing "Trick"	19
2.4.1	The Echo State Network	20
2.4.2	Matrix Generation	21
2.4.3	Hyperparameter Selection	22
2.4.4	Training an ESN	24
2.5	Necessary Properties of RC	25
2.5.1	Generalized Synchronization	25
2.5.2	Separability	27
2.5.3	Approximation	28
2.6	Conclusions	28
3	Control of Unknown Systems with Deep Reservoir Computing	30
3.1	Problem Formulation	32
3.2	Single Layer Reservoir Controller	33
3.2.1	Choosing $\mathbf{v}_{train}$	36
3.2.2	Hyperparameter Considerations–Mackey-Glass System	36
3.3	Adding Controller Layers	42
3.3.1	Deep Hyperparameters	42
3.4	Numerical Results–Lorenz System	44
3.4.1	Unstable Steady States	45
3.4.2	Additional Layers	47
3.4.3	Lorenz Origin	48
3.4.4	Known Fixed Points	49
3.4.5	Ellipses Near Attractor	49
3.4.6	Synchronization	52

3.5	Experimental Circuit	54
3.5.1	FPGA-Accelerated Controller	56
3.5.2	Control Results	57
3.6	Conclusions	63
4	Reservoir Computing with Autonomous, Boolean Networks	66
4.1	Challenges of Real-Time Prediction	67
4.1.1	Physical RC	68
4.1.2	Real-Time Prediction with Optical RC	69
4.2	Field-Programmable Gate Arrays	70
4.2.1	Synchronous versus Autonomous Logic	70
4.2.2	FPGA-Accelerated RC	71
4.3	Autonomous Boolean Reservoirs	71
4.3.1	Matching Time Scales with Delays	73
4.3.2	Fading Memory	74
4.4	Synchronous Components	76
4.4.1	Input Layer	76
4.4.2	Binary Representations of Real Data	78
4.5	Output Layer	79
4.6	Results Analysis	80
4.6.1	Generation of the Mackey-Glass System	81
4.6.2	Spectral Radius	83
4.6.3	Connectivity	84
4.6.4	Mean Delay	85
4.6.5	Input Density	86
4.6.6	Attractor Reconstruction	86
4.7	Conclusion and Future Directions	87

5	Dimensionality Reduction in Reservoir Computers	94
5.1	Previous Pre-Training Algorithms	95
5.2	Collinearity in Echo State Networks	96
5.2.1	Dynamical Equivalence	99
5.2.2	Autonomous Reduced Network	99
5.3	SVD Compression Algorithm	101
5.3.1	SVD	103
5.3.2	Compressed Echo State Networks	105
5.3.3	Performance Analysis	107
5.4	Re-using Reduced Reservoirs	109
5.5	Deriving High-Performance ESNs	112
5.5.1	Linear Analysis	113
5.5.2	Linear-Equivalent ESNs	114
5.6	Conclusion and Future Directions	115
6	Conclusions and Future Research	118
6.1	Discussion	118
6.1.1	Future Directions	120
	Bibliography	123
A	Hardware Descriptions for ABN-RC	131
A.1	LUT Nodes	131
A.2	Autonomous Reservoir	133
A.3	Synchronous Components	135
B	Hardware Description for dESN Controller	138
B.1	Tanh LUT	138
B.2	Synchronous Delay Line	140

B.3	Weights	141
B.4	Regulator	142

List of Figures

2.1	Types of ANNs. a) A very general ANN. The presence of the connection in red creates a closed loop, making this an RNN. b) Removing the recurrent connection yields a feedforward ANN. The new connection in red prevents the separation of the network into layers. c) Removing this connection yields a restricted, feedforward ANN. There are now distinct layers to the network, which I indicate with blue, green, and red colors. Efficient training algorithms exist for these types of ANNs. d) By adding recurrent connections <i>only</i> within the middle layer, I have a reservoir computer. The reservoir is surrounded by the green dashed line and contains all of the recurrent connections.	16
2.2	An artificial neuron. Generally, an artificial neuron can perform any function on its inputs to produce a real-valued output signal. Most commonly, artificial neurons act on a weighted sum of its input signals. For example, parameterized by the weights $w_{1,2}$, $w_{1,3}$, and $w_{1,4}$, this artificial neuron executes the associated weighted sum on nodes x_2 , x_3 , and x_4 and applies a nonlinear activation function to produce x_1	17

2.3	An illustration of the generalized synchronization of an ESN to the Lorenz system. a) With $\rho = 0.9$, the reservoir exhibits generalized synchronization. Given two identical ESNs in different initial conditions subject to a common Lorenz input, the two network states quickly converge to each other. b) With a much larger $\rho = 5.0$, the reservoir no longer synchronizes to the Lorenz system. The two ESNs in separate initial conditions never converge. In other words, the reservoir never "forgets" what its initial conditions are.	27
3.1	A schematic representation of the plant and reservoir controller. a) The plant and reservoir controller in training configuration. The plant is driven with an exploratory training signal $\mathbf{v}_{train}$. Measurements of the plant state $\mathbf{y}(t)$ and a delayed plant state $\mathbf{y}(t - \delta)$ are fed into the reservoir. Measurements of the reservoir state $\mathbf{u}(t)$ are made and used to train the reservoir. b) The plant and reservoir controller in control configuration. The signals $\mathbf{y}(t)$ and $\mathbf{y}(t - \delta)$ have been replaced with $\mathbf{r}(t + \delta)$ and $\mathbf{y}(t)$ respectively, where $\mathbf{r}(t + \delta)$ is a reference signal that defines the desired plant behavior. The reservoir output $\mathbf{v}(t)$ drives the plant towards the reference signal.	35

3.2	A study varying the temporal parameters in the RC control scheme applied to the Mackey-Glass system. a) I argue that $\lambda > \delta$ for good learning of the inverse system. From the figure, it appears this constraint is unnecessarily strong, and good inversion is learned as long as I do not have $\delta \gg \lambda$. b) Similarly, I argue that $\lambda \approx c$ for good inversion. This is born out by the study, where worse inversion is only found when either λ or c is significantly larger than the other. c) Even though the plant inversion error space is smooth with respect to δ and λ , the control error space is more complicated. A range of parameters yields good control, mostly with small λ and larger δ . d) Similarly, the control error space is more complicated in the $\lambda - c$ plane. There is a region of good performance consistent with $\lambda \approx c$, but only when these values are around 0.8.	40
3.3	Two performance measures of a single reservoir controlling the Mackey-Glass system. The plant inversion error (red) decreases as N is increased. This is expected, as $\mathbf{W}_{out}$ is identified to minimize this measure. On the other hand, the control error (blue) does not decrease monotonically. Rather, it is high for small values of N and reaches a sharp minimum around $N = 30$, even though the plant inversion error continues to decrease past this point.	41
3.4	The configuration of the deep reservoir controller. All layers of the controller take as input $\mathbf{y}$ and $\mathbf{r}_\delta$, which couple to the i_{th} reservoir through $\mathbf{W}_{in,i}^y$ and $\mathbf{W}_{in,i}^r$, respectfully. The trained weights $\mathbf{W}_{out,i}$ depend only on the measured dynamics of the $(i - 1)_{th}$ controller, so the deep controller can be trained sequentially. The final controller effort $\mathbf{v}$ is the sum of all the individual reservoir outputs.	43

3.5	Control of the Lorenz system to the positive USS. The parameters used in the control algorithm are listed in Table 3.2. a) The first component v_1 of the reservoir output compared to the first component $v_{train,1}$ of the training input to Lorenz. To ensure that the reservoir is generalizing $\mathbf{v}_{train}$ and not overfitting, I train $\mathbf{W}_{out}$ using only data before $t = T_{train} = 200$ and examine the signals past the training period. b) The Lorenz outputs before and after the controller is switched on. c) The control signal, as generated by the trained reservoir. d) The Lorenz system in phase space. After the controller is turned on, the system is quickly stabilized towards the desired USS.	47
3.6	A typical trajectory of a controlled Lorenz system. Dashed lines separate successive training and control phases, with the error from the requested USS displayed in the bottom panel. The control error improves by two orders of magnitude between application of the first and fourth layers.	48
3.7	The control of the Lorenz system to the origin, which appears to require multiple layers to stabilize. a) The uncontrolled Lorenz attractor (blue). b) After applying one reservoir, the Lorenz system stabilizes, but far from the requested point (orange). c) The second layer brings the system into a periodic orbit that passes through the origin (green). c) Finally, the third layer brings the system close to the origin and is stable (red). Additional layers serve to improve the control error.	50
3.8	The control error of an 3-layer controller. When appropriately selecting the bias vectors as in Eq. 3.12, the control error decays exponentially to 0.	51
3.9	The phase space portrait of the Lorenz system (blue) and the requested ellipse (orange).	51

3.10	The control of the Lorenz system to an ellipse near the attractor. From top to bottom, the number of layers in the controller is increased from $n = 1$ to $n = 4$. From the right panels, the control signal often needs a large initial perturbation to move Lorenz to the requested ellipse.	52
3.11	The synchronization (control) error for two Lorenz systems. Additional layers of the controller are switched on at every vertical dashed line. After one reservoir, the systems are synchronized with error ranging between 1 and 0.1. However, because the attractor is unchanged, additional layers do not improve performance, even up to 10 layers.	53
3.12	The control error as functions of training magnitude for different reservoir sizes. For a fixed g , control error is unchanged by N above a certain minimum N . However, this minimum depends on g , so better performance can be obtained by simultaneously increasing N and decreasing g	54
3.13	The chaotic circuit to be controlled. a) A schematic description of the circuit. Parameter values are given in Table 3.3. b) The attractor of the unperturbed, simulated circuit.	55
3.14	Control of the experimental circuit to the origin. a) In real space, the circuit is stabilized to the origin quickly after the first reservoir is switched on, but with a small DC shift. When the second reservoir is switched on, the circuit moves closer to the origin. b) In phase space, the target lies at the center of the attractor. Noise leads to a spread in the asymptotic behavior of the plant controlled with the first and second controlled system.	59
3.15	Control of the experimental circuit between USss. a) In real space, the first controller leads to substantial ringing after the circuit is moved. The second reservoir substantially reduces this. b) In phase space, it appears that dragging straight across the attractor is an unnatural trajectory for the circuit.	60

3.16	The control of the experimental circuit to an ellipse. a) A periodic input current stabilizes an ellipse trajectory in the circuit. b) The circuit tends to “slip” away from the ellipse, as can be seen from phase space. The second controller partially remedies this, bringing the circuit closer to the desired ellipse.	61
3.17	The RMSE of the settled circuit, versus the number of reservoirs, for the origin (blue), dragging (red), and ellipse (orange) control tasks described in the text. Experimental results from 30 different trials are in solid lines and are limited to two reservoirs. Numerical simulation results from 15 different trials are in dashed lines and go up to four reservoirs. The horizontal dashed line represents the RMS noise level in the circuit.	64
4.1	Experimental observation of the fading memory property and decay time for varying mean delay. The network has 100 nodes and hyperparameters $k = 2, \rho = 1.5$, and $\sigma = 0.75$. Statistics are generated by testing five reservoirs for each set of hyperparameters. Vertical error bars represent the standard error of the mean. The relationship is approximately linear with a slope of 3.99 ± 0.45 . . .	76
4.2	A schematic representation of the reservoir computer, divided into synchronous and asynchronous components. A global clock c drives the input and output layers. The values of y and v only change on the rising edge of the c , indicated on all synchronous components with red dots. On the other hand, the reservoir nodes u operate autonomously, evolving in between the rising edges of c	77
4.3	A visualization of the discretization of the input signal necessary for hardware computation. (a) In general, the true input signal may be real-valued and defined over a continuous interval. (b) Due to finite precision and sampling time, the actual $u(t)$ seen by the reservoir is held constant over intervals of duration t_{sample} and have finite vertical precision. For the prediction task, $v_d(t) = u(t)$, so the output must be discretized similarly.	78

4.4	An example of the output of a trained reservoir computer. Autonomous generation starts at $t = 0$. The target signal is the state of the Mackey-Glass system described by Eq. 4.12. The particular hyperparameters are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11 \text{ ns}, 0.5)$	82
4.5	Prediction performance and fading memory of reservoirs with varying spectral radius. (a) Somewhat consistent with observations in echo-state networks, ρ near 1.0 appears to be a good choice. However, a much wider range of ρ suffice as well. (b) As ρ becomes small and the reservoir becomes more strongly coupled to the input, the reservoir more quickly forgets previous inputs. The decay time levels out above $\rho = 1.0$. Note that λ is everywhere the same order of magnitude as $\bar{\tau}$	89
4.6	Prediction performance and fading memory of reservoirs with varying connectivity. (a) I see effectively no difference over this range, contrary to intuitions from studies of Boolean networks in discrete time. (b) For $k = 1$, λ is approximately equal to $\bar{\tau}$. However, as I increase k to 4, both the mean and variance of λ approaches almost an order of magnitude larger than $\bar{\tau}$	90
4.7	Prediction performance of reservoirs with varying mean delay. The NRMSE decreases until approximately $\bar{\tau} = 9.5$, after which point it remains approximately constant.	91
4.8	Prediction performance and fading memory of reservoirs with varying input density. (a) Choosing $\sigma = 0.5$ improves prediction performance by a factor of 3 over the usual choice of $\sigma = 1.0$ (b) With larger σ , the reservoir is more strongly coupled to the input signal. Consequently, λ decreases, signifying that the reservoir is more quickly forgetting previous inputs.	92

4.9	Phase-space representations and power spectra of the attractors of the Mackey-Glass system and trained reservoirs. (a) The true attractor and (b) normalized power spectrum of the Mackey-Glass system, as presented to the reservoir. (c) The attractor and (d) normalized power spectrum for a reservoir whose long-term behavior is similar to the true Mackey-Glass system. Although “fuzzy,” the attractor remains near the true attractor. The power spectrum shows a peak 0.10 MHz away from the true peak. The hyperparameters for this reservoir are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11 \text{ ns}, 0.75)$. (e) The attractor and (f) normalized power spectrum of a reservoir whose long-term behavior is different than the true Mackey-Glass system. The dominate frequency of the true system is highly suppressed, while a lower-frequency mode is amplified. The hyperparameters for this reservoir are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 4, 11 \text{ ns}, 0.75)$. The dashed, red line in the power spectrum plots indicates the peak of the spectrum in the true Mackey-Glass system. .	93
5.1	The attractor of the Mackey-Glass system in the chaotic regime. It is a benchmark system for prediction of chaotic time series.	97
5.2	The redundancy of a node in a typical ESN driven by the Mackey-Glass system. a) Based on observations of x_0 and $\mathbf{x}_{-0}$ from $t = 0$ to $t = 1400$, a linear transformation $\mathbf{v}$ is chosen based on the pseudoinverse of the collected data. The curves of x_0 and $\mathbf{v}^T \mathbf{x}_{-0}$ are identical to the eye, even after $t = 1400$. b) The difference between the two curves in Fig. 5.2a differ by only approximately 10^{-7} , even at times not used to identify $\mathbf{v}$	98
5.3	The difference in the dynamics of the reduced network when a node is replaced by a linear approximation from the other nodes. The median difference is around 10^{-7} , and the difference does not exceed 2.5×10^{-7} . Note that this is the total vector difference $\mathbf{x} - \tilde{\mathbf{x}}$, so the difference of a typical node is on the order of 10^{-9}	100

5.4	Comparing the autonomous evolution of a 100 node trained reservoir, a 99 node reservoir with one linear replacement, and the true Mackey-Glass system. a) Traces of the autonomous systems. Calculating the error after 1 Lyapunov time, the errors for the full and reduced system agree within 0.1%. b) Difference between the full reservoir and the reduced reservoir vs the full reservoir and the true system. The full reservoir eventually diverges from the true system, as must happen in the presence of chaos. Similarly, the full reservoir diverges with the reduced reservoir, but only after both systems have already lost track of the true system.	101
5.5	The SVD of a trace of observations of a 100 node network driven by the Mackey-Glass system. The node magnitudes indicate how much they contribute to a linear reconstruction of the full network. Despite the apparently rich dynamics in the 100 node network, only the first hand-full of reduced nodes are visible. . .	102
5.6	A comparison of the attractors of the full and reduced reservoirs.	102
5.7	A schematic comparison of an ESN to an CESN. a) The connections and nonlinear operations required to compute $\mathbf{x}(t + 1)$ for a 5-dimensional ESN. The majority of the operations come from a 5×5 matrix multiplication and 5 applications of the tanh function. b) The connections and nonlinear operations required to computer $\tilde{\mathbf{x}}(t + 1)$ for a 2-dimensional CESN that was derived from a 5-dimensional ESN.	105

5.8	Comparing the full 200 node reservoir and various reduced networks. During the listening phase, the mean difference between the full network trace and the reconstruction from the reduced trace are calculated and plotted in red, showing a smooth increase as the size is decreased. I also compare the predictions of the Mackey-Glass system of the autonomous systems, plotted in blue, as measured by the NRMSE after one Lyapunov time. Remarkably, performance is flat down to approximately $d = 100$, even though there are measurable differences in the reservoir traces.	108
5.9	Comparing the full 1,000 node reservoir and various reduced networks, where the reduction is performed based on the Mackey-Glass system. The CESNs are then tested by predicting the scaled Lorenz system. The performance of the 1,000 node ESN is represented by the horizontal dashed line. Similar to testing with the Mackey-Glass system, the performance is relatively flat until some minimum d . When testing with Lorenz, however, the dependence on d is much noisier. . . .	110
5.10	A visualization of typical adjacency matrices. a) The random adjacency matrix in a typical ESN. Note that the typical weight is very small, and weights are randomly distributed. b) The effective adjacency matrix derived from the Mackey-Glass system. c) The effective matrix derived from the Lorenz system. d) The effective matrix derived from a random input. Note that all effective matrices are approximately upper-triangular, with strong self-coupling, and a preference to couple to particular nodes.	117
A.1	The LUT for the AND function. It can be specified by the Boolean string that makes up the right-most column.	131
A.2	Verilog code for a generic node that can implement any 3-input Boolean function, specified by a Boolean string of length 8.	132
A.3	Verilog code for a delay line.	133

A.4	Verilog code describing a simple reservoir. The connections and LUTs are determined from Eq. 5.2 and Eq. A.1-A.3. Lines 9-11 declare 3 nodes. Lines 13-18 declare delay lines that connect them.	135
A.5	Verilog code describing the reservoir computer. It contains the reservoir module discussed in App. A and various synchronous components.	136
B.1	Verilog code for the TanhLUT module. It only outputs a single wire, which defines the LUT for the tanh function. The assignments in the initial block are the rows of the LUT as determined by the procedure outlined in this appendix. .	139
B.2	Verilog code for the Tanh module. It takes in a 10-bit input and the tanh_lut wire outputted by an instance of the TanhLUT module. The always block defines combinational logic that is effectively a 10-to-10 multiplexer.	139
B.3	Verilog code for the SyncDelayLine module. It has a single parameter determining the maximum number of delaying registers. It operates by generating a series of registers, passing along the in wire on the rising edge of clk . The selector wire delay determines which of these registers is connected to the output.	140
B.4	Verilog code for the multiplying by hard-coded weights. Note that this is just a snippet of code that might go inside a reservoir module or a top-level module, depending on design. The parameter N specifies the number of nodes. The weights are hard-coded as 4-bit signed, decimal numbers (4'sdxx). The multiplied matrices $\mathbf{W}_{in}\mathbf{u}$ and $\mathbf{W}\mathbf{x}$ correspond to the sign register arrays W_in_u and W_x in hardware, respectively.	141
B.5	Verilog code for the multiplying by hard-coded weights. Note that this is just a snippet of code that might go inside a top-level module, depending on design. It takes various signals and directs them appropriately, depending on the operating mode.	143

List of Tables

1.1	The novel contributions and their impact for the various ideas presented in this thesis.	7
3.1	The hyperparameters used to control the Mackey-Glass system, unless otherwise noted.	39
3.2	The hyperparameters used to control the Lorenz system to the positive USS, unless otherwise specified.	46
3.3	The values of the parameters describing the circuit in Eq. 3.13. All values are measured within 1%.	55
3.4	The hyperparameters used to control the experimental circuit for the various control tasks. Note that the hyperparameters describing the physical reservoir ($N, \rho, k, \sigma, b_{mean}, b_{max}$, and c) are identical for all three tasks. That is, one only needs to change the control hyperparameters to target a new trajectory.	58
5.1	The hyperparameters used for the compression experiments, unless otherwise noted. See Ch. 2 for an explanation of these parameters and the reservoir computing algorithm.	98
5.2	Prediction errors for the ESN versus CESN at the Mackey-Glass prediction task. Optimal results are obtained by the CESN.	109

5.3 Prediction error for the ESN versus several CESNs at the Lorenz prediction task. Each CESN has been derived from an ESN based on its response to a different input signal. All CESNs outperform the standard ESN, and all perform within a standard error of each other. 112

Chapter 1

Introduction

Over the last several decades, there has been an increasing interest in artificial neural networks (ANNs) from a wide range of scientific disciplines, such as computer science, biology, sociology, mathematics, and physics. While notable work has been done with ANNs as models for biologically plausible neural networks (Mazzoni, Andersen, and Jordan, 1991), ANNs are primarily of practical interest as tools for machine learning (ML) applications. Interest has been spurred, in large part, both by recent advances in theoretical understanding as well as advances in computational power, making ANNs attractive tools for the processing of large amounts of data. Commercial and industrial applications of ANNs range from voice-recognition algorithms used in cell phones (Melin *et al.*, 2006) to end-to-end learning for self-driving vehicles (Bojarski *et al.*, 2016). They have also more recently found significant application as scientific tools, facilitating the classification of astronomical data (Kim and Brunner, 2016) and phase transitions in topological materials (Van Nieuwenburg, Liu, and Huber, 2017).

Most of the significant applications—including all of those cited above—rely on a particular type of ANN that is *feedforward*, meaning that there are no closed cycles in the network graph, and *layered*, meaning that nodes are arranged in discrete layers, where one layer feeds forward into the next layer only. The popularity of these networks rose substantially after it was discovered (Hinton, 2007) that a backpropagation algorithm can be employed to train such a network in a way that is (relatively) computationally efficient and, more importantly, generalized well

on a host of practical tasks. From a conceptual standpoint, these types of networks also present attractive analogies to well-studied physics tools (Mehta and Schwab, 2014). So many expansions and variations of this idea have been studied and applied to practical problems that it is often collectively referred to as simply *deep learning*, particularly in computer science disciplines.

Despite the success of deep learning, this family of algorithms suffers from some common drawbacks, both practical and conceptual. First, due to the large number of tunable parameters, deep learning tends to be data-hungry (Ng *et al.*, 2015). To avoid overfitting, this means requiring sometimes millions of examples in a training data set in order to perform. Second, the training process takes a long time, especially when working with large data sets. This is due to the large amount of data required, the computational complexity of the error gradient, and the unsupervised pre-training phase required for good performance. For example, ANNs such as those that beat chess grandmasters require days of time on thousands of parallel, specialized processing units to train (Silver *et al.*, 2017). Third, the feedforward restriction results in static, time-independent systems that are both biologically implausible and awkward to apply to intrinsically temporal tasks, such as time-series prediction or control engineering.

The last of the aforementioned drawbacks can be relieved by relaxing the feedforward restriction, resulting in recurrent neural networks (RNNs) that have time-dependence. While the computational power of RNNs is well-known (Funahashi and Nakamura, 1993), training them is a notoriously difficult task (Pascanu, Mikolov, and Bengio, 2013). The backpropagation algorithm can be generalized to apply to RNNs, but often fails due to an exploding or vanishing gradient. Despite significant research devoted to the subject, efficient training of full RNNs remains elusive.

In 2001-2002, a fundamentally new approach to training RNNs was introduced independently by Jaeger, 2001 in the form of echo state networks (ESNs) and by Maass, Natschläger, and Markram, 2002 in the form of liquid state machines (LSMs). Although the mathematical form of the networks described in these works are quite different, the approaches to training

were quickly realized to be similar and identified as two realizations of what is now known as reservoir computing (RC). In each of these early works, the RNN was partitioned into three distinct layers—an input layer, a recurrent layer known as the *reservoir*, and an output layer—with the critical restriction that only feedforward connections existed between the reservoir and output layer. The RC "trick" was to *randomly* instantiate input-reservoir and reservoir-reservoir connections, and leave these fixed during a "listening" period. During the listening period, an input signal drives the reservoir, and the time-dependent reservoir state is recorded. After this phase, the reservoir-output connections are determined by simple linear regression to minimize the difference between the output and a desired output. Because of the separation of recurrent and feedforward layers, this connection selection can be done in a one-shot fashion, avoiding the problems that plague previous RNN training algorithms.

The ESN and LSM are time-dependent objects that can naturally handle time-dependent tasks. The ESN in particular quickly became a popular tool for time-series prediction tasks, becoming state-of-the-art at a number of benchmarks (Jaeger, 2002). They also present a more biologically plausible model for learning systems, exhibiting short-term, dynamical memory. Indeed, the LSM was developed not to explicitly be a computational tool, but to explore biologically plausible models for how the brain operates (Maass, Natschläger, and Markram, 2002). These new ANNs have additional advantages over deep learning techniques. Likely due to the reduced free parameters, they require much less data to obtain good performance. No unsupervised pre-training is required, and the training reduces to simply inverting a matrix, resulting in training time many orders of magnitude less time than for a commensurate deep ANN.

Since their introduction, RC algorithms have been applied to a number of time-dependent problems with state-of-the-art performance. In addition to time-series prediction, ESNs have been applied to hidden-variable observation (Lu *et al.*, 2017), control engineering (Waegeman and Schrauwen, 2012), and signal classification (Carroll, 2018). They have also been applied as light-weight solutions for tasks at which deep learning has excelled, such as handwritten-digit classification (Schaetti, Salomon, and Couturier, 2016) and spoken-word recognition (Hinton,

2007).

In addition to novel problem applications, much research has been devoted to novel forms of the reservoir, part of which is seeking to understand what makes RC such an effective approach in the first place. Because the properties of the reservoir do not change during the training process, it is actually not necessary for the reservoir dynamics to be simulated on a computer or even known at all. This invites the possibility of novel and complex hardware to be used as the reservoir, leading to potentially superior results. In recent years, researchers have used optical elements (Larger *et al.*, 2012), mechanical oscillators (Dion, Mejaouri, and Sylvestre, 2018), and even a bucket of water (Fernando and Sojakka, 2003) as the reservoir.

Presently, one of the largest problems in RC research is a lack of understanding of why and when reservoirs perform well. As a result of this knowledge gap, the parameters involved in reservoir design are often determined by heuristics rather than rigorous processes (Lukoševičius, 2012). A notion of short-term memory is understood to be important, but identifying appropriate memory time-scales is difficult, and the importance of the subtle distinctions in the definition of memory is unclear. The degree to which nonlinearity (Carroll, 2018) and even recurrence (Griffith, Pomerance, and Gauthier, 2019) are required is less apparent than when RC was originally conceived.

In this thesis, I aim to reduce this knowledge gap with discussions of a number of original projects. My contributions to the field are primarily involved in the application of RC to dynamical systems. In addition to exploring fundamental properties of RC, these projects demonstrate practical and novel algorithms relying on reservoir computers, as demonstrated with numerical and experimental data.

1.1 Novel Contribution and Outline

In this section, I outline the remainder of this thesis, with emphasis on illustrating my novel contributions to the field of RC. These contributions are summarized in Table 1.1.

In Ch. 2, I provide an introduction to the foundational concepts in RC. I first provide a definition of a dynamical system and then illustrative examples of the key concepts used in this thesis. I then introduce machine ML and explain the concepts of training and hyperparameters. I give precise definitions of neural network terms such as neuron and ANN and contextualize RC within the greater field. I then explain the RC framework, including training algorithms and selection of appropriate hyperparameters. I also define common performance metrics that are used throughout this thesis.

In Ch. 3, I describe an algorithm for control of an unknown system to a desired trajectory with RC. I use an initial ESN to learn to invert the dynamics of an unknown system of interest, in a sense which I explain further in this chapter. This learned inverse model can be thought of as an extension of learning to predict a time-series, presenting RC as a natural approach to the problem. I present a thorough analysis of the resulting dynamical system and the effects of hyperparameter selection with concrete examples. I then develop an algorithm for obtaining more precise control laws by iterating the simple control algorithm on the controlled system. The resulting controller structure is that of a layered ESN, which I refer to as a deep ESN (dESN). I demonstrate that this control algorithm is capable of controlling a wide range of systems. Previous control algorithms are either capable of only controlling a small subset of possible behavior, require knowledge of the underlying system equations, or require a complicated two-step process in which the system was first identified with an ANN or other ML model. My approach is fast and precise, even when applied to complex systems.

Also in Ch. 3, I apply the control algorithm to an experimental circuit with fast oscillations. Due to the simplicity of the ESN equations, a controller is simulated efficiently on a field-programmable gate array (FPGA) and used to control the experimental circuit. I explain the design of the FPGA-based controller and study the performance, controlling the chaotic circuit to a variety of target behavior. I also perform simulations to support the experimental conclusions. This work demonstrates that my algorithm is capable of light-weight control of a real-world system with the associated noise and non-ideal behaviors.

In Ch. 4, I develop a novel hardware implementation of RC using the autonomous dynamics of FPGAs. This work expands upon studies of RC with autonomous, Boolean networks (ABNs), creating a framework capable of processing real-valued input signals with real-time output feedback. I describe the design of the ABN reservoir computer and study the resulting dynamical properties. I demonstrate explicitly the memory capabilities and measure performance at a benchmark task that requires output feedback. I demonstrate that the ABN-RC is capable of performance comparable to state-of-the-art simulated algorithms of similar network size, but with vastly improved prediction rate. To my knowledge, this scheme produces the fastest real-time prediction algorithm, significantly outperforming optical RC.

In Ch. 5, I discuss a dimensionality reduction algorithm for ESNs. The algorithm is based on a simple concept from statistical learning known as singular-value decomposition (SVD). I use the SVD representation of a measured reservoir response to find equivalent, low-dimensional reservoirs that perform as well as reservoirs 20 times their size. I study the dynamics and structure of the resulting low-dimensional reservoirs and consider the extent to which they are universally applicable. I find emergent structure in the topology of these low-dimensional reservoirs, suggesting stochastic but data-driven procedures for developing efficient ESNs beyond the completely random paradigm.

Finally, in Ch. 6, I conclude by summarizing and contextualizing my findings. I end by proposing several avenues of future research that expand upon the projects outlined in the preceding chapters.

Novel Contribution	Impact / New Physics	Previous Work	Publications
Model free control algorithm (Ch. 3)	Direct learning to control a completely unknown dynamical system; control of chaos to arbitrary trajectories; experimental demonstration on a physical circuit	Previously done for shallow networks on less complicated systems	Canaday, Pomerance, and Gauthier, <i>in preparation</i>
Reservoir computing with autonomous, Boolean networks (Ch. 4)	A technique for reservoir computing with real-valued signals on compact, autonomous Boolean network; enables record-fast time-series generation on a readily-available platform	Previously done for classification of Boolean inputs	Canaday, Griffith, and Gauthier, 2018
Dimensionality reduction algorithm for RC (Ch. 5)	More efficient representations for ESNs, reducing their required memory and footprint without sacrificing performance	Largely unsuccessful pruning algorithms	Canaday, Gauthier, <i>in preparation</i>

TABLE 1.1: The novel contributions and their impact for the various ideas presented in this thesis.

Chapter 2

Foundations of Reservoir Computing

In this thesis, I apply the tool of reservoir computing (RC) to the study of dynamical systems. This, of course, requires some background understanding of what I mean by dynamical system as well as how RC can be practically applied to data derived from dynamical systems. In this chapter, I discuss and motivate these concepts with simple examples.

The rest of this chapter is organized as follows: I first introduce the concept of a dynamical system with concrete examples and motivate their study. I proceed to define machine learning (ML) and then artificial neural networks (ANNs) as a tool for modern ML techniques. Next, I describe the reservoir computing "trick" to training recurrent neural networks (RNNs). I then describe, in detail, how a common type of reservoir computer is generated and trained. I move on to describe the necessary properties for a reservoir within the RC framework. Finally, I conclude by summarizing the key points and how they will be used in the following chapters.

2.1 Dynamical Systems

Broadly speaking, a dynamical system is something that evolves in time according to some rule. Examples are innumerable and include mundane phenomena such as a pendulum moving under the influence of gravity, as well as complex systems such as the Earth's climate and its evolution. Specifying a dynamical system requires determining the "something" and the "some rule."

To move towards a precise definition, I refer to the "something" as the *state space variables*, which I label $\mathbf{x}$ throughout this thesis, unless otherwise noted. The "some rule" is the *state space evolution function*, which I label $\mathbf{f}$. In the mundane example, the state space variables are the angular position and angular velocity of the pendulum bob, and the state space evolution function is a simple differential equation that depends on the local acceleration of gravity, the mass of the pendulum, and the length of the pendulum. Note that this joint description $(\mathbf{x}, \mathbf{f})$ is not unique. Instead of describing the state space variables this way, I could use the x and y coordinates and their derivatives, or I could use the angular position at time t and the angular position at time $t - \tau$ (in either of these alternative cases, the accompanying $\mathbf{f}$ will be more complicated).

When examining a dynamical system, it is often not the case that the entire state space variable is available for measurement. In the climate example, tomorrow's weather likely depends on the temperature and pressure of the air all over the world. Instead of measuring the temperature and pressure everywhere, meteorologists are only able to measure at discrete locations. This is a common situation and necessitates defining an *observable* and an *observation function*, which I denote with $\mathbf{y}$ and $\mathbf{g}$, respectively. The observation function acts on the state space variables and yields the observable, *i.e.*,

$$\mathbf{y} = \mathbf{g}(\mathbf{x}). \tag{2.1}$$

In the climate example, the observation function is simply a projection of the state space variables onto a smaller subspace of $\mathbf{x}$. The observation function may have a more complicated effect and mix the space space variables in a potentially nonlinear way. It is often of critical importance whether or not it is possible to infer $\mathbf{x}$ from $\mathbf{y}$, a situation in which I say the dynamical system is *observable*.

2.1.1 Types of Dynamical Systems

The state space evolution function can specify a "rule" for evolving $\mathbf{x}$ in a number of different ways, several of which yield a natural classification for different types of dynamical systems. In perhaps the most common case for physical systems, $\mathbf{f}$ describes the derivative of $\mathbf{x}$, *i.e.*,

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}). \quad (2.2)$$

This means that the state space variables simply evolve according to a differential equation.

Alternatively, the state space might evolve according to a difference equation in which $\mathbf{f}$ determines not $\dot{\mathbf{x}}$, but $\mathbf{x}(t + h)$ in the form

$$\mathbf{x}(t + h) = \mathbf{f}(\mathbf{x}(t)). \quad (2.3)$$

These types of dynamical systems are useful descriptions when there is a natural discretization of time, such as the closing value of a stock market index.

In either the difference equation or differential equation case, the state space evolution function may include a delay operator D_τ , defined by its action on $\mathbf{x}$ as

$$D_\tau(\mathbf{x}(t)) = \mathbf{x}(t - \tau). \quad (2.4)$$

When present in any part of $\mathbf{f}$, these terms define a delay differential equation or a delay difference equation. These types of systems are important in my study of control engineering with RC in Ch. 3.

One final distinguishing property that is of interest in this thesis is autonomous versus nonautonomous dynamical systems. An autonomous system is what I have explicitly defined so far, where the state space evolution function *only* depends on $\mathbf{x}$. Alternatively, the evolution of the dynamical system might depend on an outside signal called the *input*, which I label $\mathbf{u}$.

The complete description of a nonautonomous dynamical system with a differential equation is then

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{u}), \\ \mathbf{y} &= \mathbf{g}(\mathbf{x}).\end{aligned}\tag{2.5}$$

In some sense, the distinction in Eq. 2.5 is a semantic rather than physical difference, as it is often the case that $\mathbf{u}$ is itself the observation of a separate dynamical system. This means that a single dynamical system can be defined that includes $\mathbf{u}$, its state variables, and its observation and state space evolution functions. However, thorough analysis of nonautonomous systems when $\mathbf{u}$ is assumed to be an arbitrary input signal is quite complex—see, *e.g.*, “[Theory of Input Driven Dynamical Systems](#)” for a discussion of these issues.

2.1.2 Delay Embedding

As noted above, it is often a critical question whether a dynamical system is observable. Note that this is possible even when the observation function $\mathbf{g}$ is not invertible, such as when $\mathbf{y}$ is of smaller dimension than $\mathbf{x}$. This can be done by constructing a *delay embedding* of $\mathbf{y}$, which is a vector defined by parameters ΔT and n and given by

$$\mathbf{y}_{DE} = (\mathbf{y}(\Delta t), \mathbf{y}(t - \Delta T), \dots, \mathbf{y}(t - n\Delta T)).\tag{2.6}$$

The connection between $\mathbf{y}_{DE}$ and $\mathbf{x}$ is made precise in the work of Takens, 1981 with what is called Takens’ embedding theorem. This theorem states that, under some very mild assumptions on the dynamical system, there exists a diffeomorphism between the attractor of the dynamical system and the delay embedding. This allows us to infer a number of properties by examining $\mathbf{y}_{DE}$.

Important to the discussion of RC in this thesis is the fact that, if the assumptions of Takens' theorem are met (a situation in which I say the dynamical system is *Takens observable*), then there exists some time-independent function $\mathbf{G}$ such that $\mathbf{y}_{DE} = \mathbf{G}(x)$. This means that, if a dynamical system (such as a reservoir) driven by $\mathbf{y}$ retains memory of past values of $\mathbf{y}$, then the reservoir state contains all of the important dynamical information about the driving system.

2.2 Machine Learning

Given a dynamical system, one often wants to measure or infer some of its physical properties. In the case of a pendulum, one might measure the observables for some time and wish to infer the mass of the pendulum. Given complete knowledge of the dynamical system, *i.e.*, given $\mathbf{f}$ and $\mathbf{g}$, this inference is straightforward.

For the case of the climate system, one might want to know tomorrow's temperature as well as the uncertainty in this prediction. This inference is much more complicated, despite the massive amount of data that is available, in large part simply because $\mathbf{f}$ is not fully understood. A possible strategy for making this inference is to take past values of climate measurements and next-day temperatures and use this data to make a model that maps these data sets.

This general approach, where one uses data to construct useful models in an automated fashion, is the essence of *machine learning*. A machine learning algorithm depends both on the type of model and the process for determining model parameters, which I refer to as *training*. In perhaps the simplest example, I can model the next-day temperatures as a linear function of present-day temperatures, measured in various parts of the world. I can determine the linear transformation by minimizing a measure of the difference between the model's output and the actual next-day temperature. The linear function is then the model, and the procedure for minimizing this difference measure is the training.

To be even more precise, assume that $\mathbf{C}$ is a $n \times m$ matrix of climate measurements, where each of the m columns is the set of n climate observables, measured at the same time every day.

Let $\mathbf{T}$ be a $1 \times m$ matrix of temperatures, each one taken the following day. Then the goal is to train a linear model such that

$$\mathbf{T} \approx \mathbf{AC}, \quad (2.7)$$

for some matrix $\mathbf{A}$.

2.2.1 Performance Measures

Given a trained model such as the linear model in Eq. 2.7, one needs to quantify the performance. In the temperature prediction case, a well-working model makes accurate predictions on *unseen* data. A natural way to quantify this is to define additional data $\mathbf{C}_{test}$ and $\mathbf{T}_{test}$, which are set of climate and temperature observations that were *not* used to train the model. I can then calculate the root-mean-square error (RMSE) of the model on the test set as

$$RMSE = \sqrt{\sum_{i=1}^{m_{test}} \frac{(T_{test}^i - \mathbf{AC}_{test}^i)^2}{m_{test}}}, \quad (2.8)$$

where m_{test} is the number of observations in the test set.

A small RMSE indicates that my predictions are accurate, while a large RMSE indicates that my model is not very useful. In cases such as this where the desired model output has non-zero variance, it is often helpful to report the normalized RMSE (NRMSE) by defining

$$NRMSE = \frac{RMSE}{\sqrt{\text{var}(\mathbf{T})}}, \quad (2.9)$$

where $\text{var}(\mathbf{T})$ is the variance of the temperatures. This definition is useful because, by definition, the "trivial" prediction that does not depend at all on $\mathbf{C}$ but rather simply guesses that future temperature is always the average temperature will have $NRMSE = 1$. Thus, an $NRMSE < 1$ indicates the model performs better than the trivial prediction.

2.2.2 Hyperparameters

Many machine learning algorithms involve specifying some parameters before the learning process begins. These parameters are referred to as *hyperparameters*. This is often a way of inserting *a priori* knowledge into the learning process. Other times it offers a small-dimensional set of parameters that affect the learning process and can be tuned if initial results are poor.

For example, an algorithm for predicting tomorrow's temperature might depend on measurements from the past ΔT days, where ΔT is a hyperparameter selected before a full model is constructed. The time ΔT is likely related to the memory time-scale of the climate, and a range of reasonable values can be selected by meteorologists. If a value of ΔT yields poor predictions, the value can be adjusted accordingly. Note that this is just constructing a delay-embedded vector for the observed climate variables. Thus, if the weather system is Takens' observable and if n is sufficiently large, the model I am attempting to construct does exist, even if it's surely very complicated.

2.3 Artificial Neural Networks

The linear model defined by Eq. 2.7 is likely far too simple for the climate prediction task. The true relationship between climate measurements and tomorrow's temperature is far more complicated, involving nonlinearities that are not captured by any choice of $\mathbf{A}$. To tackle these complicated, nonlinear problems, a more flexible model is needed. Artificial neural networks are exactly such models.

Within the context of ML, ANNs are families of functions for modeling data that are biologically motivated by how the brain processes data. As such, they take data as input and produce outputs, where this mapping is determined by a large number of tunable parameters. They are in fact dynamical systems (although they are sometimes memoryless, as in the case of feedforward ANNs), and can be analyzed within that framework.

Artificial neural networks are constructed from artificial neurons and a weighted graph that describes their connections. A weighted graph is simply an adjacency matrix with associated real numbers that determine the strength of interaction between neurons. An example of a small but general ANN is in Fig. 2.1a.

An artificial neuron is the fundamental processing unit of the ANN. Most broadly, it performs some parameterized function on its input neurons and produces an output, where the parameters are individually tuned by the training process. More commonly, this parameterized function involves a weighted sum of the real-valued neuron inputs. This weighted sum is referred to as the neuron's *activation*. The artificial neuron then applies a nonlinear function, often a sigmoidal function such as *tanh*, on the activation. A schematic representation of this typical neuron is in Fig. 2.2.

Many types of ANN are powerful computational tools because they have high representational power, meaning any reasonable function can be represented approximately by an ANN with some choice of the parameters. This notion can be made precise with various *universal approximator theorems* (Hornik, Stinchcombe, and White, 1989), which depend on the particular form of the ANN.

2.3.1 Feedforward ANNs

The type of ANN displayed in Fig. 2.1a is very general. In particular, it allows the possibility of closed loops in the weighted graph. This necessarily makes the ANN with recurrent connections a time-dependent object, where either updates are made at discrete time intervals (a difference equation dynamical system) or node outputs are determined by some differential equation (a differential equation dynamical system). Biological neural networks have these closed loops, which allow for short-term memory in the network. However, these objects are notoriously difficult to train (see sec. 2.3.3).

A feasible training algorithm can be derived for a very general ANN that is restricted to be *feedforward*, indicating an absence of recurrent connections, as in Fig. 2.1b. In this case, there

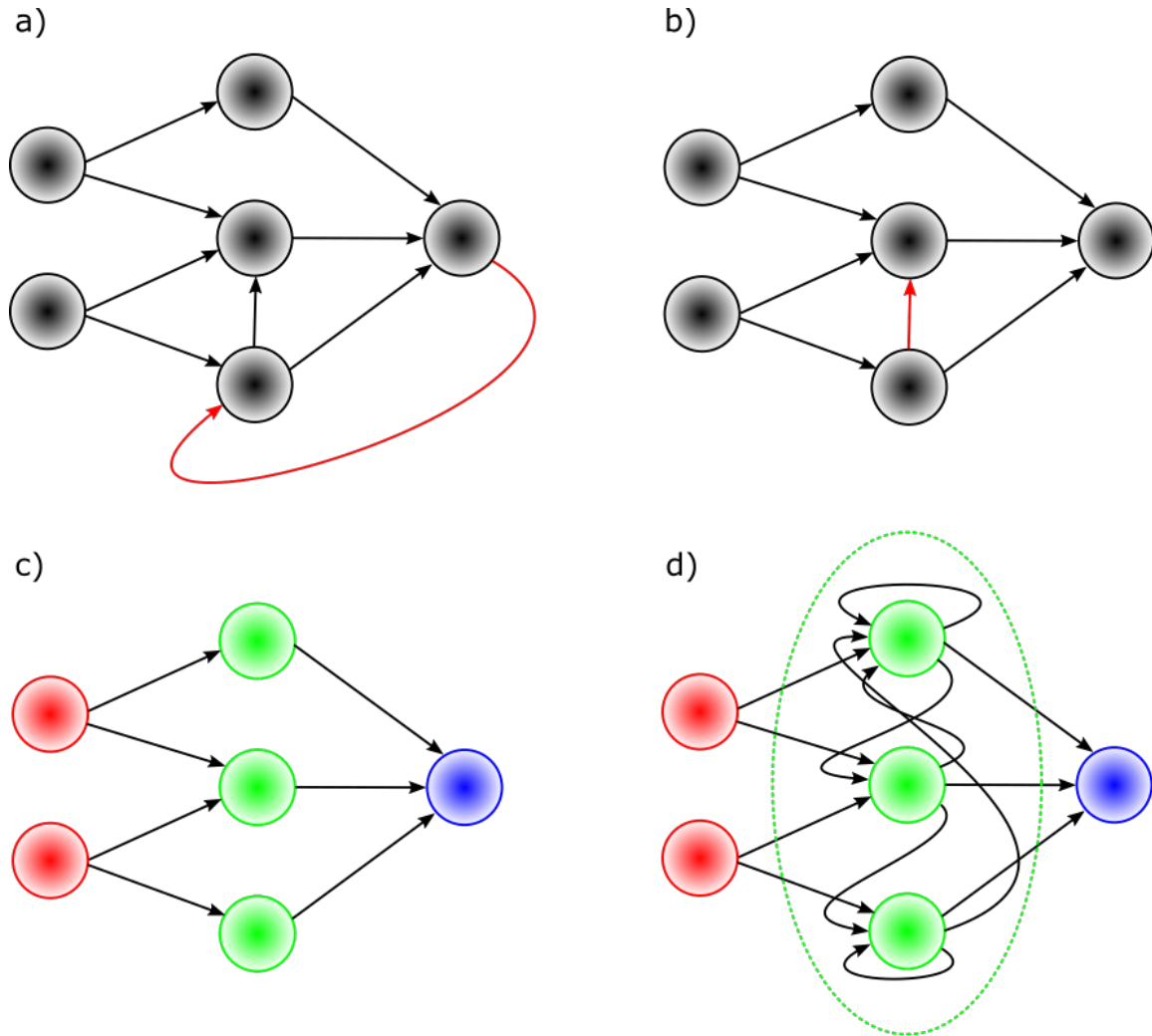


FIGURE 2.1: Types of ANNs. a) A very general ANN. The presence of the connection in red creates a closed loop, making this an RNN. b) Removing the recurrent connection yields a feedforward ANN. The new connection in red prevents the separation of the network into layers. c) Removing this connection yields a restricted, feedforward ANN. There are now distinct layers to the network, which I indicate with blue, green, and red colors. Efficient training algorithms exist for these types of ANNs. d) By adding recurrent connections *only* within the middle layer, I have a reservoir computer. The reservoir is surrounded by the green dashed line and contains all of the recurrent connections.

are no closed loops and no dynamics—the network is simply a static map. However, under very mild assumptions, these ANNs are still universal approximators of static maps (Hornik, Stinchcombe, and White, 1989) and can be extremely useful for time-independent problems.

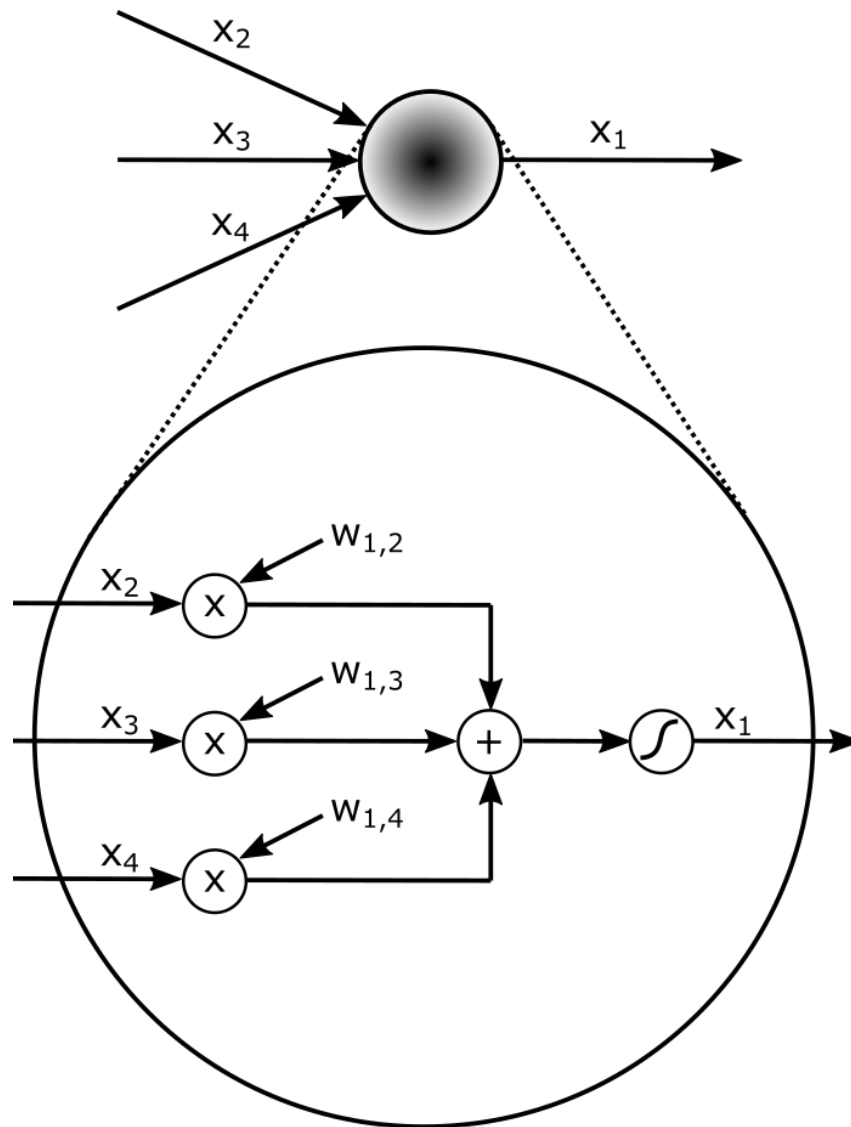


FIGURE 2.2: An artificial neuron. Generally, an artificial neuron can perform any function on its inputs to produce a real-valued output signal. Most commonly, artificial neurons act on a weighted sum of its input signals. For example, parameterized by the weights $w_{1,2}$, $w_{1,3}$, and $w_{1,4}$, this artificial neuron executes the associated weighted sum on nodes x_2 , x_3 , and x_4 and applies a nonlinear activation function to produce x_1 .

More progress still can be made by considering a *restricted* network, meaning that the nodes are arranged in multiple layers as in Fig. 2.2c with no intra-layer connections. These networks have a layered structure and facilitate efficient learning algorithms, as discussed in the next

section.

2.3.2 Training

To *train* an ANN means to identify the optimal network parameters. Training can generally be divided into *unsupervised* and *supervised*, where unsupervised training is without respect to a specified output. For example, a deep ANN might have an initial, unsupervised training phase where the parameters are chosen to maximize the final layer's mutual information with the input. Supervised training, on the other hand, is with respect to a specified output, typically by minimizing some performance measure that compares the model outputs with target outputs. Unlike the linear model in Eq. 2.7, training even a small ANN is not an obvious procedure. Ultimately, like any other training algorithm, the goal is to minimize some performance measure, but in the case of ANNs, this function is highly complicated and nonlinear.

As mentioned in the previous section, the ANN training process becomes much easier when constraining attention to restricted, feedforward ANNs, often referred to as *deep neural networks* (DNN). In 2007, an effective training strategy was discovered (Hinton, 2007). The key discovery was the use of *backpropagation*, which simply expresses the output error in terms of the error in individual layers using the chain rule. This facilitates the identification of local minimum in the parameter space by decomposing the error calculation. Further, backpropagation is easy to implement on emerging graphical processing unit (GPU) technology, leading to its quick rise to popularity in certain applications.

2.3.3 The Problem of RNNs

With the development of efficient training algorithms such as those discussed in the previous section, it was quickly realized that a similar approach could be applied to RNNs, referred to as backpropagation through time. For discrete-time RNNs, the idea is to "unravel" the network

by viewing the network state at time t as one layer of restricted, feedforward ANN, the state at time $t - h$ as another layer, *etc.*

However, this approach quickly runs into problems. If the recurrent weights are such that the network is contracting—that is, shrinking on every iteration—then the effect of error will shrink exponentially as it is backpropagated through time. Conversely, if the network is expanding, the error will grow exponentially. Either case results in inability to understand how small changes in weights effect the long-term behavior of the network. This is known as the *exploding / vanishing gradient problem* and presents a major obstacle to full training of RNNs.

2.4 The Reservoir Computing "Trick"

The barrier to training RNNs described in the previous section prevailed for many years. In 2001 and 2002, a fundamentally new approach to training RNNs was introduced independently by Jaeger, 2001 in the form of echo state networks (ESNs) and by Maass, Natschläger, and Markram, 2002 in the form of liquid state machines (LSMs). The ESN and LSM differ by the form of the network dynamics, but the underlying concepts are similar and later understood to be part of a broader class of ANN algorithms known as RC.

The RC "trick" is to partition the RNN into three layers—an input layer, a recurrent layer known as the *reservoir*, and an output layer—such that the only recurrent connections exist within the reservoir. An illustration of this partition is in Fig. 2.2d, where the reservoir is enclosed by the dashed green ellipse.

The advantage of this partition is to realize that there exist many accessible feedforward connections from the reservoir to the output layer. The RC approach prescribes *randomly* instantiating the other weights and leaving them fixed throughout the training process. The model inputs then drive the reservoir, and the reservoir response is observed, during what is called the listening phase. Because most of the parameters are fixed, any training algorithm

requires less data than DNNs or fully trained RNNs where every parameter must be tuned (although, some similar ideas have been implemented for DNNs (Rahimi and Recht, 2008)). Further, because of the chosen partition, output weights are identified by a simple linear regression algorithm, resulting in much faster training times than algorithms which require calculating an error gradient.

2.4.1 The Echo State Network

One of the original papers on RC discussed a particular form of the reservoir in what is known as an echo state network (ESN). While many forms of RC exist, ESNs remain one of the most commonly used and thoroughly investigated. In Ch. 3 and 5, I develop algorithms that explicitly use the ESN. While the reservoir I use in Ch. 4 is not an ESN, it is motivated in part by its construction. It is therefore prudent to discuss this particular type of RC in greater detail.

The ESN was originally introduced as a difference equation. The neurons execute the $\tanh$ function on a weighted sum of their inputs. The ESN evolution function also includes a "leaking" term that slows down the response of the neurons and provides each node with intrinsic memory. Explicitly, the ESN equations in their difference equation form are given by

$$\begin{aligned}\mathbf{x}(t+h) &= (1-a)\mathbf{x}(t) + a \tanh(\mathbf{W}\mathbf{x}(t) + \mathbf{W}_{in}\mathbf{u}(t) + \mathbf{b}), \\ \mathbf{y}(t) &= \mathbf{W}_{out}\mathbf{x}(t),\end{aligned}\tag{2.10}$$

where $\mathbf{W}_{in}$, $\mathbf{W}$, and $\mathbf{b}$ are randomly instantiated matrices referred to as the adjacency matrix, the input matrix, and the bias vector, respectively. The constant a is referred to as the leak rate. It is typically taken to be identical for all neurons and determines the degree of memory in each node.

Although Eq. 2.10 defines a discrete map and is often used as such, it can also be interpreted as an Euler discretization of a differential equation with time-step h . Specifically, if I take $a =$

h/c , the differential equation is

$$\begin{aligned} c\dot{\mathbf{x}} &= -\mathbf{x} + \tanh(\mathbf{W}\mathbf{x} + \mathbf{W}_{in}\mathbf{u} + \mathbf{b}) \\ \mathbf{y} &= \mathbf{W}_{out}\mathbf{x}, \end{aligned} \tag{2.11}$$

where c is now a constant with units of time. This constant may, in general, vary from node to node, but is typically taken to be equal throughout the network. Note that Eq. 2.11 is only a faithful approximation of Eq. 2.10 if $h \ll c$ or, equivalently, $a \ll 1$.

2.4.2 Matrix Generation

As noted in the previous section, the ESN training process begins with a randomly instantiated network whose dynamics depend on the random matrices $\mathbf{W}$, $\mathbf{W}_{in}$, and $\mathbf{b}$. As with any random object, a distribution must be specified. Throughout this thesis, this distribution is specified with a number of hyperparameters.

The distribution of the recurrent matrix $\mathbf{W}$ is specified with three hyperparameters k , ρ , and N , which are the connectivity, spectral radius, and network size, respectively. The matrix is constructed with the following procedure: a matrix of size $N \times N$ is initially created, which I label $\mathbf{W}_0$. For each row of $\mathbf{W}_0$, k elements are randomly selected to be nonzero. The nonzero elements are then taken from a uniform distribution between -1 and 1, and the other $N - k$ elements are left at 0. The largest absolute value ρ_0 of the eigenvalues is calculated. Finally, $\mathbf{W} = \rho\mathbf{W}_0/\rho_0$ ensures that the largest absolute value of the eigenvalues (the spectral radius) of $\mathbf{W}$ is ρ and each node has only nonzero connections from k other nodes. Note that I allow the possibility that one of these k connections is a self-connection. Some authors determine $\mathbf{W}$ similarly but take the nonzero elements from a uniform distribution, but I find little difference between these two schemes in practice.

The distribution of the input matrix $\mathbf{W}_{in}$ is specified with the hyperparameters σ and N , where σ is sometimes referred to as the input scaling factor. The matrix is determined by selecting each element from a uniform distribution between $-\sigma$ and σ , where $\mathbf{W}_{in}$ is of size $N \times m$ and m is the dimension of the input vector. Note that $\mathbf{W}_{in}$ is dense, containing no non-zero elements (although see Sec. 4.6.5).

Finally, the bias vector $\mathbf{b}$ is specified with the hyperparameters b_{mean} , b_{max} , and N . The bias vector is of size N and each element is selected from a uniform distribution with mean b_{mean} and max b_{max} . Note that this vector is also dense.

2.4.3 Hyperparameter Selection

In the previous section, I described six hyperparameters whose specification is required to generate the random matrices. There are two additional hyperparameters that fully specify the standard ESN and training process. The first is the time-constant c , which defines the global time-scale of the reservoir. The last one is the ridge regression parameter λ_{reg} , which is not required until the training process that I discuss in the next section.

These eight hyperparameters must be determined prior to training the ESN, and most of them can make or break the performance of the RC scheme. They are often determined through experience and heuristics, some by-hand optimization, or more advanced automated optimization techniques (Yperman and Becker, 2016). Here, I discuss some of the heuristics that motivate my hyperparameter selection for the projects in this thesis.

The dimension of the network or number of nodes N can be thought of as a proxy for the computational resources devoted to the task. Generally speaking, increasing N will increase performance (although, see Ch. 3 for a counter example). It also, however, increases the computational complexity of both simulating the ESN equations and training the reservoir. It is commonly the case that the remaining hyperparameters are optimal across a range of N . This suggests an efficient approach in identifying well-working hyperparameters with a small N , then using a large N for the final ESN.

Perhaps the most attention and research has historically been devoted to the spectral radius ρ , which was recognized early to be related to an essential RC property known as the echo state property (ESP) (see Sec. 2.5.1.). This comes from the result in Jaeger, 2001 that shows, if $\rho > 1.0$, then ESP is violated for the null input. However, this condition is known to be too strict, and ρ slightly larger than 1.0 is optimal for many problems, for $\mathbf{W}$ determined by the procedure outlined in the previous section (Jaeger, 2001; Jiang, Berry, and Schoenauer, 2008). Typically, $\rho \approx 1.0$ is a good starting point for optimization. Because ρ is related to the stability of the origin, it is loosely related to the memory capacity of the reservoir. Thus, problems that require more memory may require a larger ρ .

The connectivity k is often recommended to be much less than N to promote a diverse reservoir response (Jaeger, 2002). This is likely also motivated by the sparsity of connections in biological neural networks such as the human brain (Robinson *et al.*, 2010) and rigorous results on certain types of ANNs that show high computational power at sparse connectivity (Büsing, Schrauwen, and Legenstein, 2010). However personal experience with the problems in this thesis show that k typically has very little effect on performance. There is another reason, however, to prefer sparse networks, and that is the reduction in computational complexity, particularly with hardware implementations of RC (see Sec. 3.5.1).

The σ serves to scale the input signal before being processed by the nonlinear neurons. If σ is too small, the reservoir response will not be highly correlated with the input signal, resulting in poor performance. Conversely, if σ is too large, the tanh functions will saturate. A general starting point is to allow $\mathbf{W}_{in}$ to scale the input signal to unit variance by selecting $\sigma = 1/\text{var}(\mathbf{W}_{in})$.

Similarly, the mean bias b_{mean} can act to shift the input signal to 0 mean with the selection $b_{mean} = -\text{mean}(\mathbf{W}_{in}/\sigma)$. Note that an equivalent scheme for selecting σ and b_{mean} that is often employed is to pre-process the input signal by scaling and shifting it to unit variance and 0 mean.

The max bias b_{max} controls the diversity of how neurons act on their inputs, serving to shift

the mean of the tanh function away from 0. To my knowledge, there is not a good heuristic for its selection, except that $b_{max} = 0$ can lead to symmetry problems (Pathak *et al.*, 2018b). I commonly find a value of $b_{max} = 0.5$ yields good results.

Finally, the reservoir time constant c determines the characteristic time-scale of the reservoir. Generally speaking, this should be commensurate with the time-scale of the inputs and/or outputs. This also affects the memory time-scale of the reservoir, as is evident from it being the only constant with units of time and as explicitly verified in Verstraeten, 2009. Another way for gaining intuition for this parameter is to realize that each tanh neuron is a nonlinear filter with cut-off frequency $1/c$.

2.4.4 Traing an ESN

To train an ESN, I define an input signal $\mathbf{u}(t)$ and a desired output signal $\mathbf{v}_d(t)$, both of which are observed for some time $0 \leq t \leq T_{train}$. After a time T_{init} , the reservoir is driven with the input signal and the response of the reservoir $\mathbf{x}(t)$ is collected in a matrix $\mathbf{X}$. The initialization time T_{init} is to discard the transient response of the reservoir and is related to the time it takes the reservoir to synchronize with the input signal (see Sec. 2.5.1).

Similar to the linear climate example, the goal is to identify a matrix $\mathbf{W}_{out}$ such that

$$\mathbf{v}(t) = \mathbf{W}_{out}\mathbf{x}(t) \approx \mathbf{v}_d(t). \quad (2.12)$$

This linear fit can be done a multitude of ways. In this thesis, I identify $\mathbf{W}_{out}$ with ridge regression. That is, I minimize the sum (or integral, if a continuous signal)

$$\sum_{t=T_{init}}^{T_{train}} (\mathbf{v}_d - \mathbf{v})^2 + |\lambda_{ridge}\mathbf{W}_{out}|^2, \quad (2.13)$$

where λ_{ridge} is a small hyperparameter whose role is to penalize large coefficients in $\mathbf{W}_{out}$. This has the effect of preventing overfitting to data by reducing the complexity of the linear fit.

Given $\mathbf{v}_d$, λ_{ridge} , and the collected $\mathbf{X}$, one can identify the $\mathbf{W}_{out}$ that minimizes Eq. 2.13 in closed form as

$$\mathbf{W}_{out} = \left(\mathbf{X}^T \mathbf{X} + \lambda_{ridge}^2 \mathbf{I} \right)^{-1} \mathbf{X}^T \mathbf{v}_d, \quad (2.14)$$

where $\mathbf{I}$ is the identity matrix of size N . The value of λ_{ridge} is typically chosen to be a very small number on the order of 10^{-6} or determined by cross-validation techniques (Jaeger, 2002).

2.5 Necessary Properties of RC

Since their inception, efforts have been made to unify RC methods under a universal framework (Verstraeten *et al.*, 2007; Lukoševičius and Jaeger, 2009). It is desirable that the underlying operating principles of RC be well-understood so that they may be applied readily to emerging ML problems. This includes identifying the minimum set of necessary properties for successful application of RC and how they may be optimized with accessible hyperparameters.

Though not universally applicable (such as applications where the transient response of the reservoir is used for classification, *e.g.* Schaetti, Salomon, and Couturier, 2016), commonly cited necessary reservoir properties are *generalized synchronization*, *separation of inputs*, and *approximation* (Verstraeten *et al.*, 2007). Despite the diversity of reservoirs, output layers, and training algorithms, these properties are seen as important in a wide range of applications.

2.5.1 Generalized Synchronization

The first criterion is sometimes called the ESP or *fading memory* in specific RC contexts. However, generalized synchronization is a generic property of unidirectionally-coupled dynamical systems (Rulkov *et al.*, 1995; Kocarev and Parlitz, 1996; Abarbanel, Rulkov, and Sushchik, 1996) that is well understood. It is satisfied when the response system (the reservoir, in this case) tends towards a continuous function of the internal dynamics of the drive system (the input

system, in this case). More formally, let the reservoir state vector $\mathbf{x}$ with inputs $\mathbf{u}$ have dynamics defined by

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{u}), \\ \dot{\mathbf{y}} &= \mathbf{g}(\mathbf{y}), \\ \mathbf{u} &= \mathbf{h}(\mathbf{y}),\end{aligned}\tag{2.15}$$

where $\mathbf{y}$ describes the internal state of the drive system, and $\mathbf{h}$ is an observation function. We say that the reservoir is synchronized, in a generalized sense, if there exists a function $\mathbf{H}$, a manifold $\mathbf{M} = \{(\mathbf{x}, \mathbf{y}) : \mathbf{y} = \mathbf{H}(\mathbf{x})\}$, and a basin of attraction $\mathbf{B}$, such that all trajectories of Eq. 3.1 that begin in $\mathbf{B}$ approach $\mathbf{M}$ as $t \rightarrow \infty$. Note that if $\mathbf{x}$ and $\mathbf{y}$ have the same dimension and $\mathbf{H}$ is the identity function, then generalized synchronization reduces to identical synchronization.

As an illustration of generalized synchronization and how it can be examined, I employ the auxiliary system approach (Abarbanel, Rulkov, and Sushchik, 1996), which states that, for two identical systems with different initial conditions in $\mathbf{B}$ that are subject to a common drive, they are in generalized synchronization with that drive if and only if the systems converge to each other after some time. For example, if I take two ESNs with identical parameters and drive them with the Lorenz system (see Ch. 3), then the generalized synchronization criteria is satisfied if and only if the nodes converge to each other. This can be explicitly verified by simulating the ESN equations for different initial conditions. In Fig. 2.3a, I consider such a situation with $\rho = 0.9$. Here, it is seen that the reservoir states converge to each other, indicating synchronization. Conversely, with $\rho = 5.0$ in Fig. 2.3b, the reservoirs never converge. This reservoir is a poor ML tool for studying the Lorenz system.

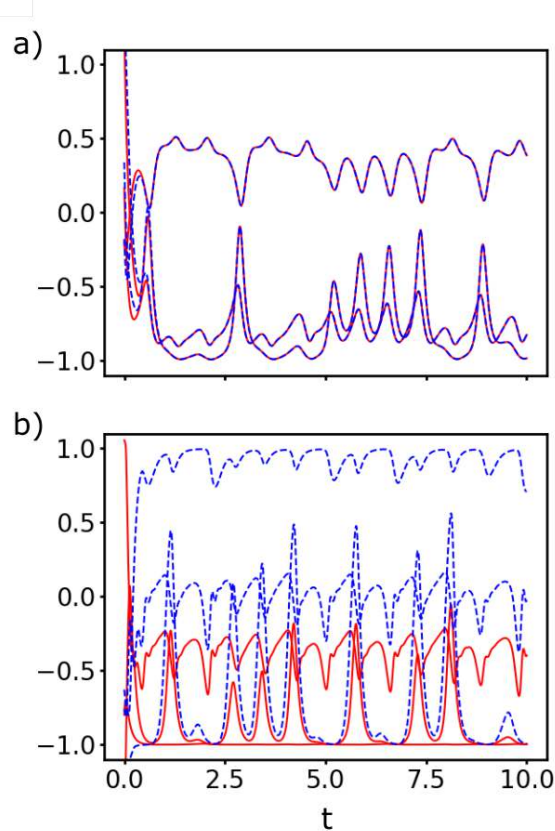


FIGURE 2.3: An illustration of the generalized synchronization of an ESN to the Lorenz system. a) With $\rho = 0.9$, the reservoir exhibits generalized synchronization. Given two identical ESNs in different initial conditions subject to a common Lorenz input, the two network states quickly converge to each other. b) With a much larger $\rho = 5.0$, the reservoir no longer synchronizes to the Lorenz system. The two ESNs in separate initial conditions never converge. In other words, the reservoir never "forgets" what its initial conditions are.

2.5.2 Separability

The second criterion states that different inputs $\mathbf{u}_1(t)$ and $\mathbf{u}_2(t)$ should yield sufficiently different reservoir responses $\mathbf{x}_1(t)$ and $\mathbf{x}_2(t)$. Intuitively, larger differences in inputs should correspond to larger differences in outputs. Conversely, similar inputs should result in only slightly different reservoir responses, such that noise in either the input or physical reservoir does not corrupt the ability of the readout layer to reconstruct some properties of the signal. Together

with the generalized synchronization property, separation implies that even very different input sequences yield similar reservoir responses as long as the differences are sufficiently far back in time—that is, the reservoir eventually "forgets" these past differences.

2.5.3 Approximation

The last criterion is perhaps the least well-understood. It states that for an input $\mathbf{u}(t)$ and desired output $\mathbf{v}_d(t)$, there exists some readout function $\mathbf{f}_{readout}$ that approximates $\mathbf{v}_d(t)$ when acting on the reservoir response. In the common case of a linear readout function, this means that the state-space matrix $\mathbf{X}$ is approximately linearly invertible. The approximation property is difficult to precisely define due to the large space of possible input and output sequences, particularly since some do not permit such an approximation property at all (if, for example, the desired output is random).

2.6 Conclusions

In this chapter, I have introduced the notion of a dynamical system. I have defined the relevant terms as they are used throughout this thesis and given several motivating examples. I then explained how a particular family of machine learning algorithms known as RC can be used to effectively study dynamical systems. I have emphasized that RNNs themselves are dynamical systems and have measurable dynamical properties that are often of interest.

I now make clear how these concepts are connected throughout the core chapters in this thesis, representing my original work. In Ch. 3, I develop an algorithm for using RC to control an arbitrary dynamical system, in a sense to be defined in the text. I improve the algorithm's performance by iteratively adding layers to the reservoir computer, yielding a layered structure known as deep RC.

In Ch. 4, I describe the construction of a novel form of RC based on an autonomous, Boolean network (ABN). The ABN is a dynamical system whose synchronization properties I

explore. I use the ABN reservoir computer as to emulate the dynamics of the Mackey-Glass system. The trained network both makes useful short-term predictions and reveals the long-term behavior of the target system. Because of how the ABN reservoir computer is constructed, predictions are made in an extremely rapid fashion, yielding a real-time prediction algorithm faster than any previous technique.

In Ch. 5, I investigate the degree of collinearity in untrained ESNs. Using a well-known technique for dimension-reduction of collinear data, I construct equivalent neural networks that have much lower dimension but contain all of the relevant dynamics needed to construct the full network response. These networks show emergent structure and allow for the development of highly efficient ESNs.

Chapter 3

Control of Unknown Systems with Deep Reservoir Computing

Control of dynamical systems is a ubiquitous problem in disciplines ranging from engineering to medicine. The fundamental problem in control engineering is the following: given a system with some accessible inputs, how does one design the inputs such that the system behaves in some desired fashion? Solutions to this problem have far-reaching applications, such as the design of autonomous vehicles (Chiou *et al.*, 2009), where the system is the car, the accessible inputs are the position of the steering wheel and gas pedal, and the desired behavior is for the car to arrive safely at its destination. Complex systems such as this are referred to as *plants* in the control engineering context. Other examples of plants requiring controllers are robotic arms (Islam, Iqbal, and Khan, 2014), airplanes (Chowdhary *et al.*, 2013), and chemical industrial processes (Nagy *et al.*, 2007).

Reservoir computing (RC) is capable of emulating complex systems given only a segment of a system observable (Jaeger, 2001; Lu *et al.*, 2017). In the control context, this means that RC can create a model for an arbitrary plant in the absence of inputs. Unsurprisingly, this notion can be extended to include plants with accessible inputs (Khodabandehlou and Fadali, 2017), a task commonly referred to as *system identification*.

Once a plant is identified, a control law can be devised. Most often, a closed loop controller

is desired, where the plant input is a function of not only the desired plant observable, but the actual plant observable. A review of the wide range of techniques for deriving such a function is beyond the scope of this thesis. They range from utilizing a piece-wise linear approximation of the plant to direct construction with a feedforward neural networks—see, *e.g.*, Paraskevopoulos, 2017 for a modern review.

System identification is a common first step towards controlling a partially or completely unknown system, particularly when applying machine-learning based techniques such as artificial neural networks. Recently, it has been shown (Antonik *et al.*, 2016) that this two-step process is not necessary with RC. In fact, reservoir computers are capable of *directly* learning an appropriate control law. This is accomplished through a re-thinking of the system identification process to identify an "inverse" system, which I explain further in the following sections.

The first contribution of this chapter is to expand the study of the RC-control method first introduced in Antonik *et al.*, 2016. I provide motivation for the algorithm, explicitly demonstrate and quantify the ability of an echo state network (ESN) to "invert" a system, and I study the effects of varying the temporal parameters new to the control problem. The second contribution is to develop an iterative technique for adding layers to the ESN controller, forming a deep ESN (dESN) and achieving more precise control. I demonstrate the efficacy of the proposed algorithm with a range of numerical and experimental results.

The rest of this chapter is organized as follows: First, I define notation and formulate the control problem I investigate. I follow by explaining the concept of direct inverse control and how it can be accomplished with RC. Next, I examine the the effects of varying hyperparameters on the control of the Mackey-Glass system. Then, I develop my multi-layered control algorithm for precise control. I then apply the algorithm to a number of numerical and experimental examples. Finally, I conclude and discuss future research directions.

3.1 Problem Formulation

I assume that the plant is an unknown, nonautonomous dynamical system. From Ch. 2, this means that the plant has a complete internal state $\mathbf{x}$, an observable output $\mathbf{y}$, and an accessible input $\mathbf{v}$. These state variables and their dynamics are defined through the state-space evolution function $\mathbf{f}$ and observation function $\mathbf{g}$ by

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{v}), \\ \mathbf{y} &= \mathbf{g}(\mathbf{x}).\end{aligned}\tag{3.1}$$

Generally, $\mathbf{f}$ and $\mathbf{g}$ are completely unknown, and the only information available is the simultaneous response of the plant to a user-defined input signal $\mathbf{v}_{train}$. In the following analysis, I assume that $\mathbf{f}$ is Lipschitz continuous with respect to $\mathbf{x}$, and that $\mathbf{g}$ is "typical" in the sense defined by Takens' theorem (Takens, 1981).

To design a controller means to design an operation that reads a reference signal $\mathbf{r}$ and outputs a control signal $\mathbf{v}$ such that $\mathbf{y} \rightarrow \mathbf{r}$. This controller is a *closed loop* controller if it also reads $\mathbf{y}$ from the plant.

If $\mathbf{v}$ is constant over an interval from t to $t + \delta$, then $\mathbf{f}(\cdot, \mathbf{v}) = \mathbf{f}_{\mathbf{v}}(\cdot)$ may be viewed a differential equation parameterized by $\mathbf{v}$. The Lipschitz condition implies that the value of $\mathbf{x}(t + \delta)$ is exactly determined by initial conditions at t , i.e.,

$$\mathbf{x}(t + \delta) = \mathbf{F}_{\mathbf{v}}[\mathbf{x}(t)].\tag{3.2}$$

If $\mathbf{v}$ is instead slowly varying from t to $t + \delta$, then I expect this equality to instead be an approximation given by

$$\mathbf{x}(t + \delta) \approx \mathbf{F}[\mathbf{x}(t), \mathbf{v}(t)]\tag{3.3}$$

for some function $\mathbf{F}$. This function will not in general be fully invertible, but may be solvable for $\mathbf{v}(t)$ on some domain of $\mathbf{x}(t), \mathbf{x}(t + \delta)$ in the sense that

$$\mathbf{v}(t) \approx \mathbf{F}^{-1}[\mathbf{x}(t), \mathbf{x}(t + \delta)], \quad (3.4)$$

where $\mathbf{F}^{-1}$ is ultimately the function of interest for devising a controller for Eq. 3.1.

3.2 Single Layer Reservoir Controller

A general strategy known as direct inverse control (Nørgård *et al.*, 2000) involves modeling the relationship in Eq. 3.4, typically with some combination of physical assumptions about the plant and observation measurements $\{\mathbf{y}(t), \mathbf{u}(t); 0 \leq t \leq T\}$. The function $\mathbf{F}^{-1}$ (or an appropriate approximation) can be used to devise a closed-loop controller by replacing $\mathbf{x}(t + \delta)$ with a desired plant state. However, the entire plant state $\mathbf{x}$ is not generally available to the controller; only the observation $\mathbf{y}$ is available for measurement. The observation function $\mathbf{g}$ is not known and may not even be invertible, so there is no clear way to infer $\mathbf{x}$ from $\mathbf{y}$.

Recall from Ch. 2 that ESNs have the ability to synchronize, in a generalized sense, with their inputs. This means that a reservoir coupled to $\mathbf{y}(t + \delta)$ and $\mathbf{y}(t)$ will tend towards a function the state variables $\mathbf{x}(t + \delta)$ and $\mathbf{x}(t)$. If I denote the reservoir state by $\mathbf{u}(t)$, then I have

$$\lim_{t \rightarrow \infty} \mathbf{u}(t) = \mathbf{G}[\mathbf{x}(t + \delta), \mathbf{x}(t)], \quad (3.5)$$

for some unspecified function $\mathbf{G}$. Equivalently, $\mathbf{u}(t)$ is approximately a function of $\mathbf{x}(t + \delta)$ and $\mathbf{x}(t)$ after some appropriate waiting time T_{init} .

Given this synchronization, and if the reservoir has a sufficient approximation property (see Sec. 2.5.3), then an output matrix $\mathbf{W}_{out}$ can be identified as

$$\mathbf{v}(t) = \mathbf{W}_{out} \mathbf{G}[\mathbf{x}(t + \delta), \mathbf{x}(t)] \approx \mathbf{F}^{-1}[\mathbf{x}(t + \delta), \mathbf{x}(t)]. \quad (3.6)$$

It is this sense in which I train the reservoir to “invert” the plant dynamics.

The training data is acquired by perturbing the plant with some random, exploratory inputs $\mathbf{v}_{train}$ from $t = 0$ to $t = T_{train} + \delta$. Perturbing with random noise ensures the plant is stimulated with many frequencies, so that a complete response can be learned. During this time, triplets $\mathbf{y}(t + \delta)$, $\mathbf{y}(t)$, and $\mathbf{v}_{train}(t)$ are collected and used to train an ESN with $\mathbf{v}_d = \mathbf{v}_{train}$. The configuration of the plant and ESN in this training phase is depicted in Fig. 3.1a.

Note that the reservoir has not directly learned the function $\mathbf{F}^{-1}$, but has implicitly learned to invert the internal plant dynamics through only the observable $\mathbf{y}$. To control the plant, $\mathbf{y}(t + \delta)$ is replaced with $\mathbf{r}(t + \delta)$, where $\mathbf{r}(t)$ denotes a reference signal that describes the desired behavior of the plant. If the ESN has learned $\mathbf{F}^{-1}$, then the resulting $\mathbf{v}(t)$ is precisely the control signal that drives $\mathbf{y}(t + \delta) \rightarrow \mathbf{r}(t + \delta)$. The complete dynamics of the controlled plant are then described by

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{v}), \\ \mathbf{y} &= \mathbf{g}(\mathbf{x}), \\ c\dot{\mathbf{u}} &= -\mathbf{u} + \tanh(\mathbf{W}\mathbf{u} + \mathbf{W}_{in}^y \mathbf{y} + \mathbf{W}_{in}^r \mathbf{r}_\delta + \mathbf{b}), \\ \mathbf{v} &= \mathbf{W}_{out} \mathbf{u}.\end{aligned}\tag{3.7}$$

For notational clarity, I write $\mathbf{r}(t + \delta) = \mathbf{r}_\delta$ and split the input weights into $\mathbf{W}_{in}^y$ and $\mathbf{W}_{in}^r$, the latter of which couples to $\mathbf{y}(t + \delta)$ in the training phase and $\mathbf{r}(t + \delta)$ in the control phase. The configuration of the plant and ESN in this control phase is in Fig. 3.1b.

In physical implementations, driving the reservoir with $\mathbf{y}(t)$ and $\mathbf{y}(t + \delta)$ can be accomplished with a delay line with delay δ as in Fig. 3.1a. This couples $\mathbf{W}_{in}^y$ to $\mathbf{y}(t - \delta)$ and $\mathbf{W}_{in}^r$ to $\mathbf{y}(t)$, which is the desired configuration under a shift $t \rightarrow t + \delta$, which can be done after listening phase is complete.

As I demonstrate in the following sections, Eq. 3.7, together with an appropriate training

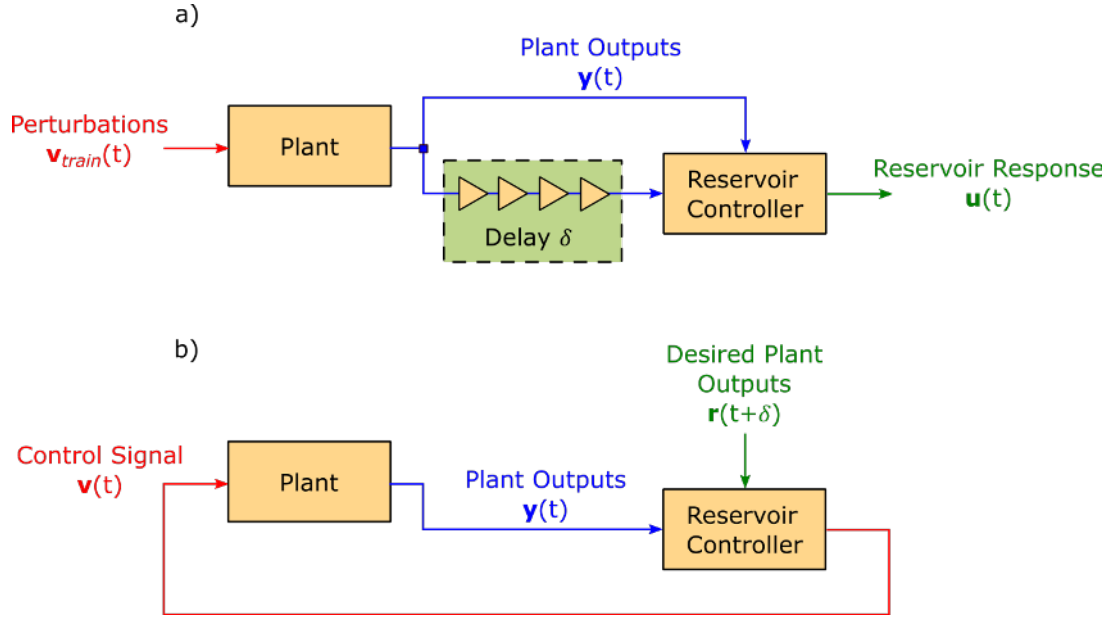


FIGURE 3.1: A schematic representation of the plant and reservoir controller. a) The plant and reservoir controller in training configuration. The plant is driven with an exploratory training signal $\mathbf{v}_{train}$. Measurements of the plant state $\mathbf{y}(t)$ and a delayed plant state $\mathbf{y}(t - \delta)$ are fed into the reservoir. Measurements of the reservoir state $\mathbf{u}(t)$ are made and used to train the reservoir. b) The plant and reservoir controller in the control configuration. The signals $\mathbf{y}(t)$ and $\mathbf{y}(t - \delta)$ have been replaced with $\mathbf{r}(t + \delta)$ and $\mathbf{y}(t)$ respectively, where $\mathbf{r}(t + \delta)$ is a reference signal that defines the desired plant behavior. The reservoir output $\mathbf{v}(t)$ drives the plant towards the reference signal after some time δ .

algorithm for $\mathbf{W}_{out}$ and selection of the training signal $\mathbf{v}_{train}$, is capable of controlling a wide range of systems. However, the error term $|\mathbf{y}(t) - \mathbf{r}(t)|$ does not converge to precisely 0, but rather some small number. This is to be expected, because the reservoir only approximately learns the inverse of the plant dynamics. One might think that the error could be reduced simply by increasing the number of nodes in the ESN, thereby increasing the computational power, as discussed in Ch. 2. However, as I demonstrate in Sec. 3.2.2, increasing N generally decreases $|\mathbf{v}(t) - \mathbf{v}_{train}|$ but *not* $|\mathbf{y}(t) - \mathbf{r}(t)|$. For situations where precise control is critical, an algorithm for improving the control error is desired. This is achieved by iteratively executing the control algorithm described above, which I describe in Sec. 3.3.

3.2.1 Choosing $\mathbf{v}_{train}$

The control algorithm described in this section requires specification of a training signal $\mathbf{v}_{train}$, with dimension m equal to the number of scalar inputs to the plant. Identification of optimal perturbation signals is an important problem in system identification, and a number of deterministic methods and heuristics have been developed (Rivera *et al.*, 2003). In keeping with the spirit of the RC framework, I randomly generate $\mathbf{v}_{train}$ according to a number of hyperparameters.

Recall from the analysis in the preceding section that the approximation in Eq. 3.3 holds if $\mathbf{v}$ varies slowly with respect to δ . This suggests that $\mathbf{v}_{train}$ be bandwidth limited with frequency cutoff $1/\lambda$, with $\lambda > \delta$. Another natural consideration is the magnitude of perturbations specified by g . Generally, the effects of large perturbations will be easier to learn, because they have a greater effect on the plant. However, this may not be the best way to learn to control the plant (see Sec. 3.2.2), and certain real-world control applications require the inputs not exceed some max threshold.

With these considerations in mind, I generate a random training signal $\mathbf{v}_{train}$ with hyperparameters λ and g with the following procedure: A white-noise, unprocessed training signal is generated by taking values from a uniform distribution between $-g$ and g . The white-noise signal is Fourier-transformed, and frequencies above $1/\lambda$ are dropped. The signal is then inverse-Fourier-transformed, yielding $\mathbf{v}_{train}$ with the required properties. A similar $\mathbf{v}_{train}$ can be obtained in a deterministic way by summing a large number of sinusoids with periods taking from a uniform distribution between 0 and λ .

3.2.2 Hyperparameter Considerations–Mackey-Glass System

As discussed in Ch. 2, the use of any ESN requires specifying certain hyperparameters that characterize the reservoir. These parameters are often selected by hand based on some heuristics (Lukoševičius, 2012), but may also be optimized by various algorithms (see Ch. 5 and

references therein). The control algorithm described so far in this chapter requires three additional hyperparameters—namely, δ , λ , and g . In this subsection, I explain how I select these hyperparameters based on the physical properties of the plant. I study the effect of these hyperparameters on the performance of a controller applied to the Mackey-Glass system. Additionally, I study the effect of N and come to the surprising conclusion that increased reservoir size does not result in increased controller performance.

The Mackey-Glass system is a nonlinear delay-differential equation (DDE) exhibiting chaotic dynamics. A driven Mackey-Glass oscillator can be created by adding a drive term to the right-hand side. The system may be fully cast as a plant defined in this section by simply observing the undelayed oscillator state, resulting in the description

$$\begin{aligned}\dot{x}(t) &= \beta \frac{x(t-\tau)}{1+x^n(t-\tau)} - \gamma x(t) + v(t) \\ y(t) &= x(t),\end{aligned}\tag{3.8}$$

where $v(t)$ is a scalar control signal. I consider the parameter set $\beta = 0.2, \alpha = 0.1, n = 10$, and $\tau = 17$, which places Eq. 3.8 in the chaotic regime without input.

I now investigate the effect of hyperparameter selection on the control of Eq. 3.8 by attempting to stabilize the unstable steady state (USS) $x(t) = 1$. I consider the effects on two measures, namely the plant inversion error and the asymptotic control error. The plant inversion error is measured by computing the NRMSE of the trained reservoir output with respect to a test segment of the training signal $\mathbf{v}_{train}$, given explicitly by

$$NRMSE = \sqrt{\sum_{i=1}^m \frac{\sum_{t=T_{train}}^{T_{train}+T_{test}} (v_i(t) - v_{train,i}(t))^2}{T_{test} \text{var}(v_i)}}.\tag{3.9}$$

This measures how well the reservoir successfully learns to approximate and generalize Eq. 3.4. The asymptotic control error is the limit of $|\mathbf{y}(t) - \mathbf{r}(t)|$ and measures how well the reservoir

controls the plant.

Unless otherwise specified, the control hyperparameters are as listed in Table 3.1. Most of the control algorithm hyperparameters have to do only with the reservoir itself and are familiar to other RC applications—see Sec. 2.4.3 for a more thorough discussion of these. As discussed in the previous subsection, the range of g is often restricted by case-specific constraints. The other control parameters δ and λ are particularly interesting in that they introduce two additional temporal parameters, where the more typical RC problem only contains c . In Sec. 3.2.1, I argue that $\lambda > \delta$ is necessary for learning Eq. 3.4, *i.e.*, for good plant inversion error. Similarly, because the signal that is ultimately produced by the reservoir has cut-off frequency $1/\lambda$ by design, it is natural to suspect that $\lambda \approx c$, because the reservoir nodes themselves are frequency filters with cut-off $1/c$. I test these intuitions by simultaneously varying the temporal parameters, the effects of which I display in Fig. 3.2.

As seen from Fig. 3.2a,b, the intuitions described above are largely accurate with respect to the inversion error. The error is a relatively smooth function in this parameter space, with minima in the λ, δ plane below the $\lambda = \delta$ line and minima in the λ, c plane along the $\lambda = c$ line. On the other hand, however, Fig. 3.2c,d reveal that the effects of these parameters on the control error are much more complicated, and that many different parameter combinations work. The observed variation in the control error is also much higher than the inversion error, leaving much more uncertainty in the error distributions as functions of δ , λ , and c .

Finally, I investigate the effect of reservoir size N . As noted in Ch. 2, the effect of increasing N is typically to decrease error metrics (as long as appropriate regularization is employed). Note from the training algorithm described so far that $\mathbf{W}_{out}$ is chosen to minimize the plant inversion error explicitly. This means that I expect the plant inversion error to decrease with increasing N . From Fig. 3.2c,d, it is clear that the relationship between this measure and the control error is not always obvious, so larger reservoirs may not be optimal for the control task. Indeed, this is what I observe in Fig. 3.3.

Surprisingly, it appears that there is an optimal N near $N = 30$ for which control error is

Hyperparameter	Value
N	100
ρ	1.15
k	10
σ	1.0
b_{mean}	0
b_{max}	1.0
c	0.6
δ	0.6
λ_{train}	0.6
g	0.1
T_{init}	100
T_{train}	1500
β	10^{-8}

TABLE 3.1: The hyperparameters used to control the Mackey-Glass system, unless otherwise noted.

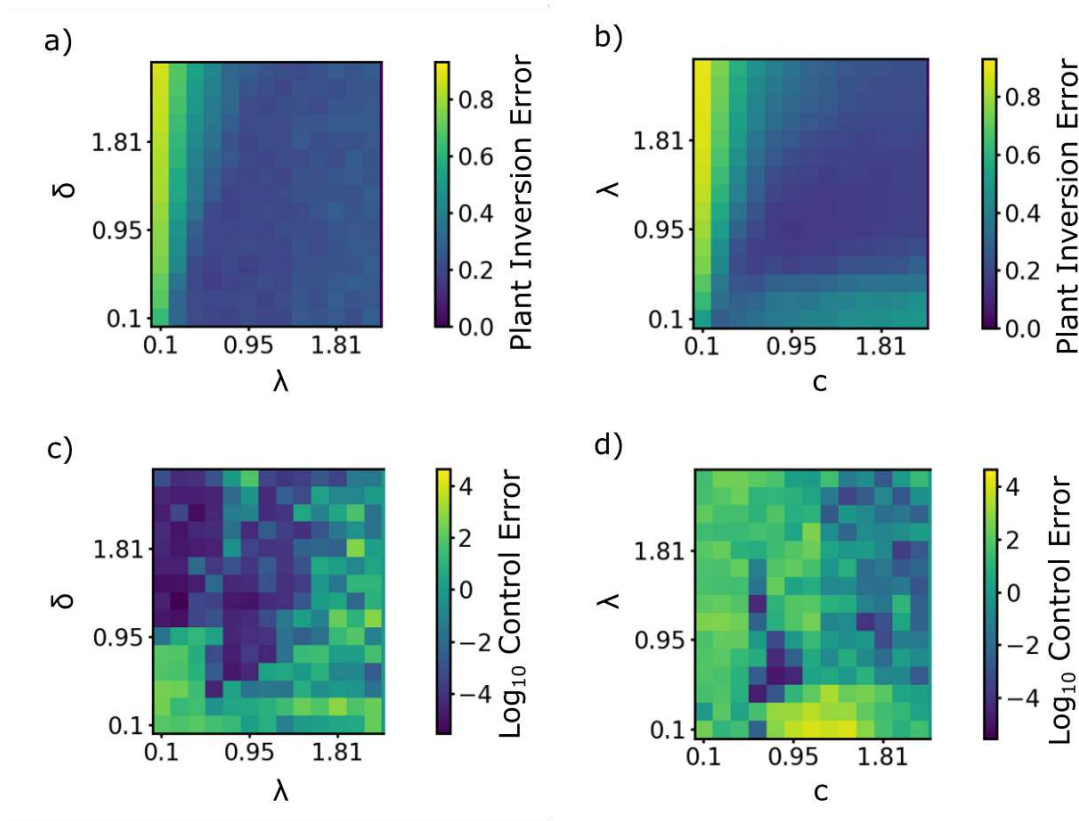


FIGURE 3.2: A study varying the temporal parameters in the RC control scheme applied to the Mackey-Glass system. a) I argue that $\lambda > \delta$ for good learning of the inverse system. From the figure, it appears this constraint is unnecessarily strong, and good inversion is learned as long as I do not have $\delta \gg \lambda$. b) Similarly, I argue that $\lambda \approx c$ for good inversion. This is born out by the study, where worse inversion is only found when either λ or c is significantly larger than the other. c) Even though the plant inversion error space is smooth with respect to δ and λ , the control error space is more complicated. A range of parameters yields good control, mostly with small λ and larger δ . d) Similarly, the control error space is more complicated in the $\lambda - c$ plane. There is a region of good performance consistent with $\lambda \approx c$, but only when these values are around 0.8.

obtained. The form of the curve in Fig. 3.3 is in fact typical of control problems studied in this thesis. It reveals that there is a certain minimum N for which any non-trivial control of the plant is obtained, but increasing N beyond this minimum actually slightly *increases* the control error.

Qualitatively, for too small values of N , the controller does not appear to alter the Mackey-Glass attractor at all, producing what is effectively noise as a control signal. For N too large, the system is stabilized near the requested $x(t) = 1$, but with a larger DC off-set than with an optimal N .

As mentioned in Sec. 3.2, the resilience of the control error to increasing N presents a problem. When greater performance is required, one typically can guarantee this by increasing the reservoir size, but this is not the case here. Optimizing the other hyperparameters is an option, but as revealed by plots such as in Fig. 3.2, this can only gain so much. Further, gradient-descent based algorithms struggle to handle complex performance spaces such as those in Fig. 3.2b,d. In the next section, I introduce an alternative method for obtaining precise control of the plant by iteratively performing the control algorithm described in this chapter on the partially controlled plant.

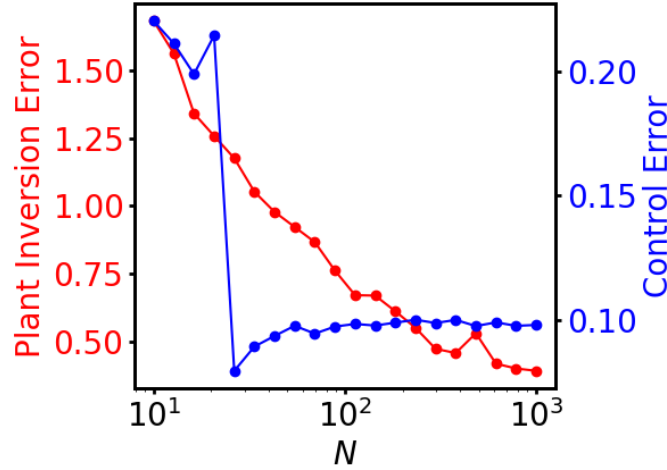


FIGURE 3.3: Two performance measures of a single reservoir controlling the Mackey-Glass system. The plant inversion error (red) decreases as N is increased. This is expected, as $\mathbf{W}_{out}$ is identified to minimize this measure. On the other hand, the control error (blue) does not decrease monotonically. Rather, it is high for small values of N and reaches a sharp minimum around $N = 30$, even though the plant inversion error continues to decrease past this point. The jump corresponds to a transition from an unstable to a stable point near the requested USS.

3.3 Adding Controller Layers

I propose a strategy for obtaining more precise control of the plant, *i.e.* smaller control error $|\mathbf{y}(t) - \mathbf{r}(t)|$, by considering the following. The controlled plant described by Eq. 3.7 can be thought of as another (partially) unknown dynamical system with internal state given by $\{\mathbf{x}, \mathbf{u}\}$ and output $\mathbf{y}$. An accessible control input $\mathbf{v}'$ can be created in a number of ways, such as with the replacement $\mathbf{v} \rightarrow \mathbf{v} + \mathbf{v}'$. Because this new plant is partially controlled, the trajectory of $\mathbf{y}$ is now much closer to $\mathbf{r}$ than in Eq. 3.1. This means that Eq. 3.7 is generally easier to control with precisely the same control strategy described above.

The process for controlling Eq. 3.1 can be repeated on Eq. 3.7 with new training inputs $\mathbf{v}_{train}$ and a new reservoir. Iterating this process results in a dESN controller, where the final control signal is the sum of each of the reservoir outputs. The complete dynamics of the plant and the n_{th} controller are depicted in Fig. 3.4 and described by

$$\begin{aligned}
 \dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{v}), \\
 \mathbf{y} &= \mathbf{g}(\mathbf{x}), \\
 c_i \dot{\mathbf{u}}_i &= -\mathbf{u}_i + \tanh\left(\mathbf{W}_i \mathbf{u}_i + \mathbf{W}_{in,i}^y \mathbf{y} + \mathbf{W}_{in,i}^r \mathbf{r}_\delta + \mathbf{b}_i\right), \\
 &\text{for } 1 \leq i \leq n, \\
 \mathbf{v} &= \sum_{i=1}^n \mathbf{W}_{out,i} \mathbf{u}_i,
 \end{aligned} \tag{3.10}$$

where the $\mathbf{W}_{out,i}$ is trained by controlling the $(i-1)_{th}$ controlled plant as described in this section.

3.3.1 Deep Hyperparameters

As already discussed in this chapter, the proposed control algorithm involves selection of hyperparameters in addition to conventional RC applications. Adding additional layers, of

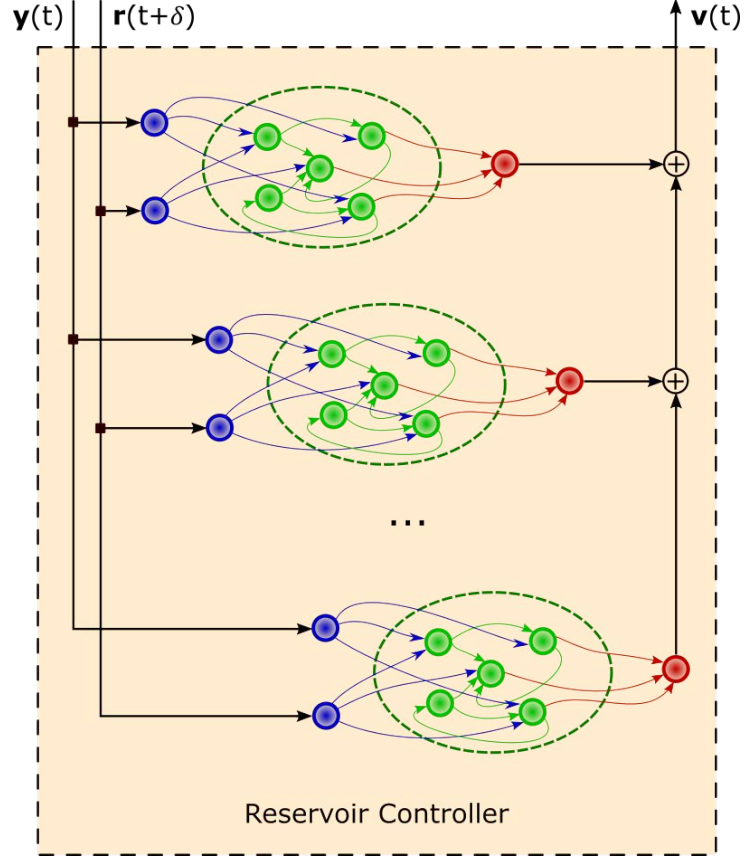


FIGURE 3.4: The configuration of the n_{th} reservoir controller. All layers of the controller take as input $\mathbf{y}$ and $\mathbf{r}_\delta$, which couple to the i_{th} reservoir through $\mathbf{W}_{in,i}^y$ and $\mathbf{W}_{in,i}^r$, respectively. The trained weights $\mathbf{W}_{out,i}$ depend only on the measured dynamics of the $(i - 1)_{th}$ controller, so the deep controller can be trained sequentially. The final controller effort $\mathbf{v}$ is the sum of all the individual reservoir outputs.

course, adds additional hyperparameters to consider. In theory, one may find that, say, the optimal spectral radius of the first reservoir is different than the 2nd or 3rd, and that these radii should be optimized individually. However, this optimization problem quickly becomes cumbersome. I find that, for the problems studied in this thesis, restricting the hyperparameters of all reservoirs to be equal yields sufficient results while greatly simplifying the design process. As such, the results in Secs. 3.4 and 3.5 are from controllers with this restriction.

3.4 Numerical Results–Lorenz System

The algorithm I have proposed in this chapter may be applied to a wide range of dynamical systems, from broken quadcopters to chaotic oscillators. In control theory, among the most difficult systems to control are those that exhibit chaos, defined by an exponential divergence of nearby trajectories in the plant state-space. This divergence results in a random-like behavior that makes long-term prediction of the plant difficult. The effects of perturbations introduced by control signals are similarly difficult to predict, making the design of controllers a challenging task. Further, chaos is an inherently nonlinear phenomenon, placing its control well outside the realm of classical control engineering. Chaos, however, is abundant in nature (Letellier, 2013), and effective chaos control techniques are desired.

The first techniques for controlling chaotic systems, such as the method of Ott, Grebogi and Yorke (Ott, Grebogi, and Yorke, 1990) and delayed-feedback methods (Chang *et al.*, 1998; Pyragas, 1992) rely on the dense-embedding of unstable periodic orbits within a chaotic attractor. While simple to employ, they are only capable of stabilizing certain types of orbits and fixed points, rather than more general control. Several advanced methods from nonlinear control theory can be applied to chaotic systems, but these methods rely on full or partial knowledge of the underlying plant dynamics and are therefore not applicable in all situations.

In this section, I apply the proposed algorithm to the control of the multi-input multi-output Lorenz system, described by

$$\begin{aligned} \dot{x}_1 &= \sigma(x_1 - x_2) + u_1, \\ \dot{x}_2 &= x_1(\rho - x_3) - x_2 + u_2, \\ \dot{x}_3 &= x_2x_3 - \beta x_3 + u_3, \\ \mathbf{y} &= \mathbf{x} \end{aligned} \tag{3.11}$$

I consider the typical parameters $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$, for which Eq. 3.11 displays

chaotic behavior. I focus on this system for concreteness and for its paradigmatic use in chaos control, but I note that similar results apply for other chaotic systems, such as Chua’s circuit and the Duffing oscillator, as well as ordered systems such as high-dimensional linear systems.

Unstable steady states exist at $(x_1, x_2, x_3) = (0, 0, 0)$ and $(x_1, x_2, x_3) = (\rho, \pm\sqrt{\rho\beta}, \pm\sqrt{\rho\beta})$, the latter of which exist at the center of the symmetric leaves of the attractor. The origin is particularly difficult to control due to the odd number of positive, real eigenvalues in the Jacobian (Chang *et al.*, 1998).

I find in the proceeding subsections that a dESN trained according to the procedure outlined in Sec 3.2-3.3 is capable of inducing a wide variety of behavior in the Lorenz system. In particular, I stabilize unstable steady states and ellipses near the attractor. I also demonstrate forced synchronization to an autonomous Lorenz system.

3.4.1 Unstable Steady States

I now illustrate the principles discussed thus far with the simple example of controlling the Lorenz system to the positive unstable fixed point. I prepare the first layer of the reservoir with the parameters listed in Table 3.2. The differential equations are simulated with a 4th order Runge-Kutta method with fixed $h = 0.001$. The resulting plant and reservoir dynamics are illustrated in Fig. 3.5.

As I can see from Fig. 3.5a, the $N = 200$ node reservoir is able to learn an approximate inverse of the plant dynamics, reconstructing the input signal from $\mathbf{y}(t)$ and $\mathbf{y}(t + \delta)$. In control configuration, *i.e.*, when the controller is switched on, the Lorenz system quickly stabilizes to a fixed point near the requested USS. The controlled plant signals are depicted in Fig. 3.5b in real space and 3.5d in phase space. Note that the control signals don’t quite tend to 0, because the error does not either.

Having identified well-working parameters for this control problem, a typical suggestion for improving performance (Lukoševičius, 2012) is to increase the network size N . As with the

Hyperparameter	Value
N	200
ρ	0.9
k	20
σ	0.05
b_{mean}	0
b_{max}	1.0
c	0.01
δ	0.05
λ_{train}	0.05
σ_{train}	25
T_{init}	25
T_{train}	250
β	10^{-8}

TABLE 3.2: The hyperparameters used to control the Lorenz system to the positive USS, unless otherwise specified.

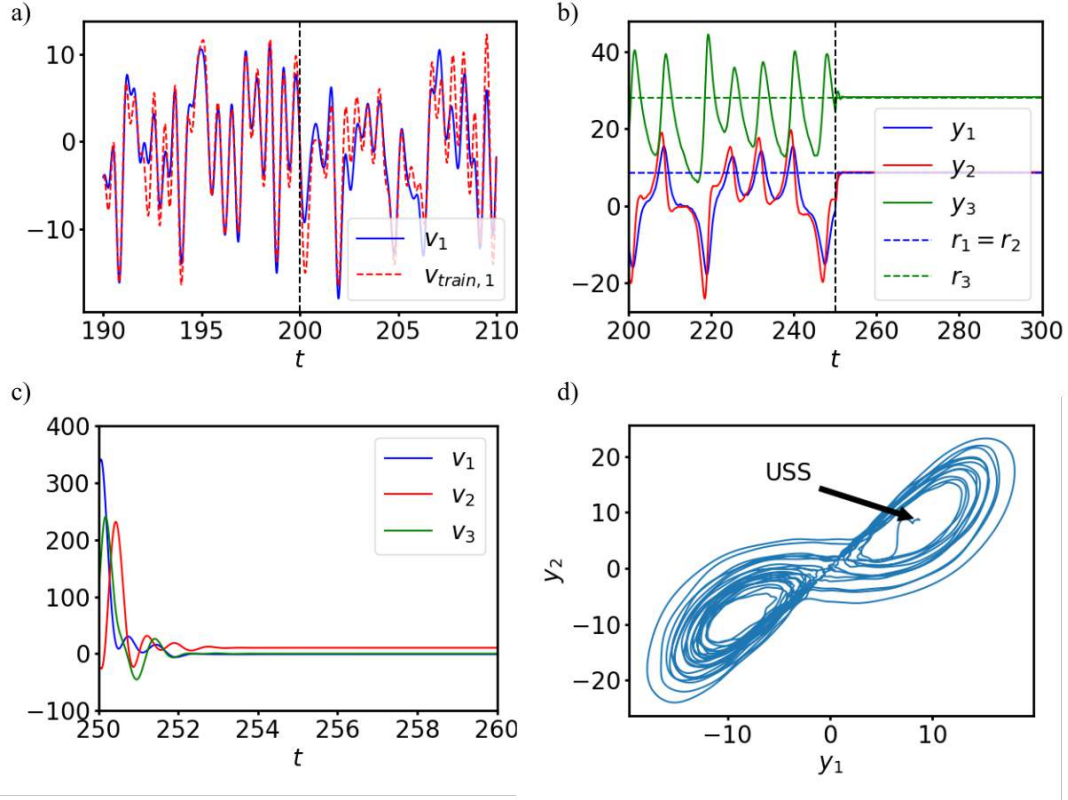


FIGURE 3.5: Control of the Lorenz system to the positive USS. The parameters used in the control algorithm are listed in Table 3.2. a) The first component v_1 of the reservoir output compared to the first component $v_{train,1}$ of the training input to Lorenz. To ensure that the reservoir is generalizing $\mathbf{v}_{train}$ and not overfitting, I train $\mathbf{W}_{out}$ using only data before $t = T_{train} = 200$ and examine the signals past the training period. b) The Lorenz outputs before and after the controller is switched on. c) The control signal, as generated by the trained reservoir. d) The Lorenz system in phase space. After the controller is turned on, the system is quickly stabilized towards the desired USS.

example in Sec. 3.2.2, this does not generally increase control performance, as measured by the asymptotic error, even though it *does* improve the plant inversion error, as measured by Eq. 3.9.

3.4.2 Additional Layers

Given the resistance of the control error to increased network size, I now turn to adding nodes in a more intelligent way, by forming a dESN controller as described in Sec. 3.3. I form such a

controller with $n = 4$ layers, each with $N = 25$ nodes that have otherwise identical hyperparameters to the previous example, listed in Table 3.2.

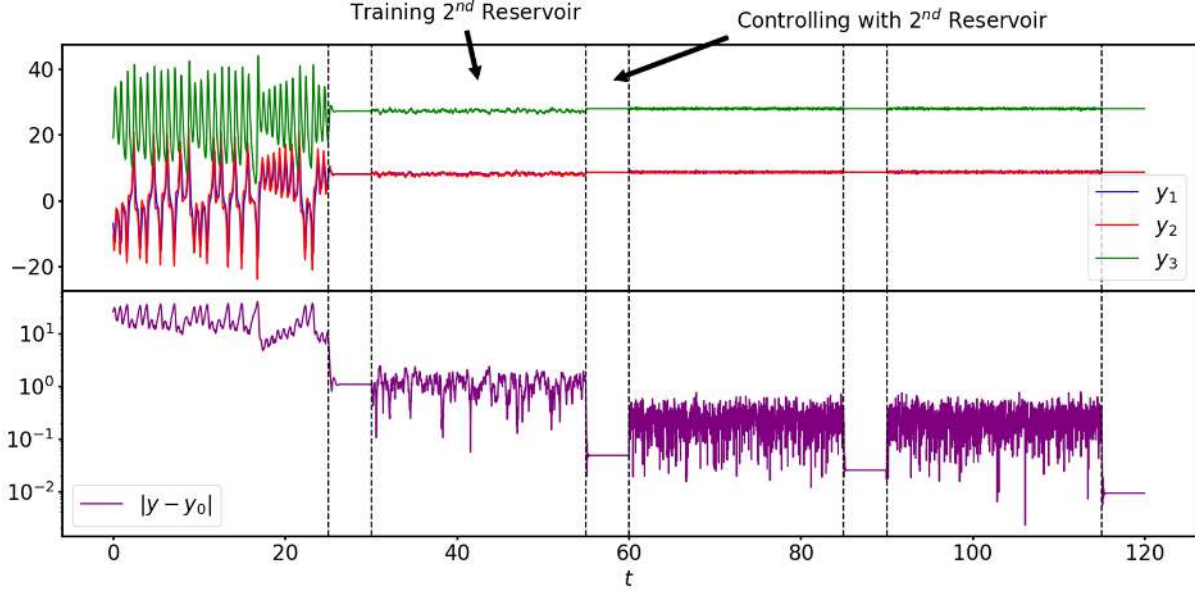


FIGURE 3.6: A typical trajectory of a controlled Lorenz system. Dashed lines separate successive training and control phases, with the error from the requested USS displayed in the bottom panel. The control error improves by two orders of magnitude between application of the first and fourth layers.

As I can see from the bottom panel in Fig. 3.6, each additional reservoir provides more precise control over the Lorenz system. After four layers, the final control error is two orders of magnitude improved from the first layer, and two orders of magnitude improved from the $N = 200$ single-layer controller, despite having half of the total nodes. I also display the training phases in Fig. 3.6 to emphasize that the controlled system is highly stable to the training perturbations used to train higher layers.

3.4.3 Lorenz Origin

As mentioned previously, the origin of the Lorenz system is particularly difficult to control, requiring a controller with nonlinear dynamics (Chang *et al.*, 1998). A curious phenomenon occurs when trying to stabilize this USS with the algorithm proposed in this chapter.

Broadly speaking, a single layer controller is not capable of stabilizing this point, but rather incorrectly stabilizes a (seemingly random) periodic orbit that is not a solution to the autonomous Lorenz system, but does pass close to the requested fixed point. As additional layers are added, the periodic attractor of the n_{th} controlled plant bends closer to the origin, until finally the origin is stabilized, typically after the 3rd or 4th iteration. This succession of controlled attractors is highly variable, but a typical illustration is in Fig. 3.7.

3.4.4 Known Fixed Points

In this subsection, I point out that perfect control (in the sense of the control error tending towards 0) is achievable in the special case that a point $\mathbf{x}_0$ is known *a priori* to be a USS. In this case, one can leverage the fact that, by definition, $\mathbf{f}(\mathbf{x}_0, \mathbf{0}) = \mathbf{0}$. The equivalent condition that the ESN produces 0 output when at the fixed point can be imposed by appropriately selecting each bias vector as

$$\mathbf{b}_i = -\mathbf{W}_{in,i}\mathbf{x}_0. \quad (3.12)$$

With this choice, it is immediate from Eq. 3.10 that $\mathbf{v} = \mathbf{0}$ for any output layer $\mathbf{W}_{out,i}$.

As an illustration, suppose the origin is known to be a fixed point of the Lorenz system. Then Eq. 3.12 prescribes a choice of $\mathbf{b} = \mathbf{0}$. Setting each bias vector to $\mathbf{0}$ in this way and controlling the origin as in the previous subsection results in a similar evolution of attractors as in Fig. 3.7, but with the final error after the 4th controller approaching 0 asymptotically, as seen in Fig. 3.8.

3.4.5 Ellipses Near Attractor

The control examples discussed so far in this section correspond to USSs whose control is possible with classical techniques. The algorithm I have proposed, however, is capable of much more general behavior. As an example, I consider an ellipse that is near the positive lobe of the

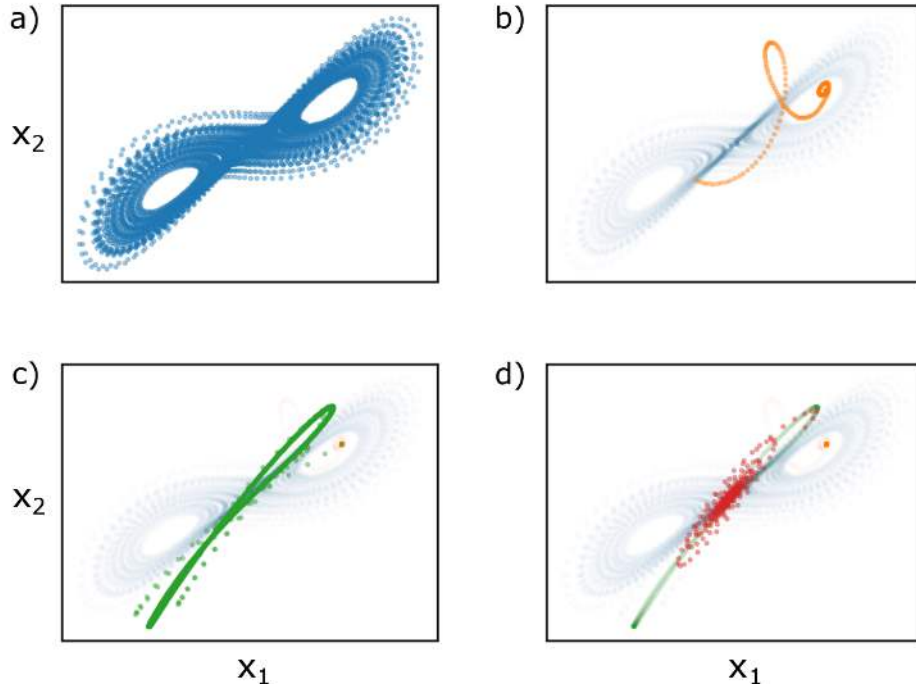


FIGURE 3.7: The control of the Lorenz system to the origin, which appears to require multiple layers to stabilize. a) The uncontrolled Lorenz attractor (blue). b) After applying one reservoir, the Lorenz system stabilizes, but far from the requested point (orange). c) The second layer brings the system into a periodic orbit that passes through the origin (green). c) Finally, the third layer brings the system close to the origin and is stable (red). Additional layers serve to improve the control error.

attractor and centered around the positive USS, as illustrated in Fig. 3.9. The ellipse coefficients are chosen by observing a segment of the Lorenz trajectory that spends several periods looping around the positive USS and fitting to an ellipse in the least-squares sense. One can verify by direct substitution into Eq. 3.11 that no ellipse is a solution to the autonomous Lorenz system, meaning this trajectory requires non-vanishing controller effort to maintain.

I proceed with the control algorithm as described in the previous section with this reference trajectory. As one can see, each added reservoir results in a more accurate controller.

One observes from Fig. 3.10 that successively deep controllers are more capable of inducing the elliptical behavior in the Lorenz system. A plethora of similar examples are possible,

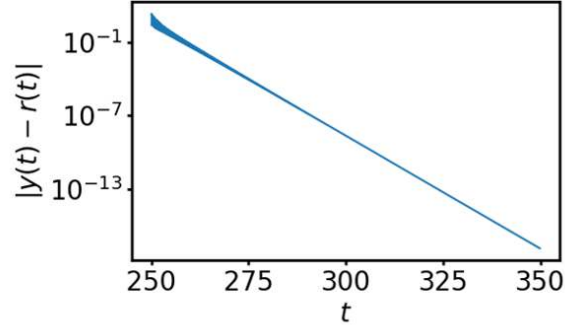


FIGURE 3.8: The control error of an $N = 3$ layer controller. When appropriately selecting the bias vectors as in Eq. 3.12, the control error decays exponentially to 0.

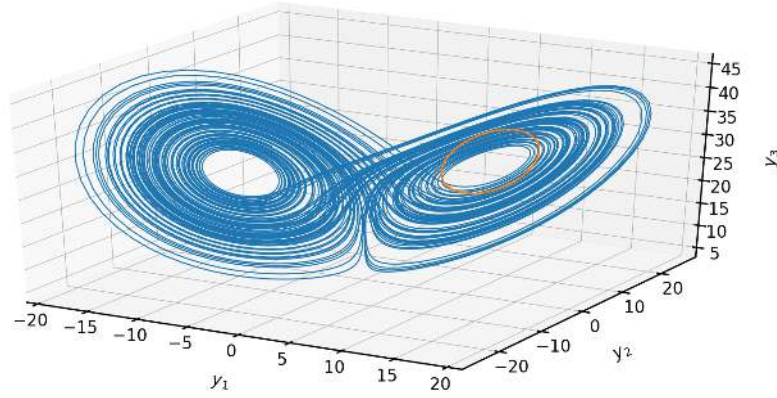


FIGURE 3.9: The phase space portrait of the Lorenz system (blue) and the requested ellipse (orange).

including "figure-eights" that traverse the attractor, or ellipses that are misaligned with respect to one of the leaves.

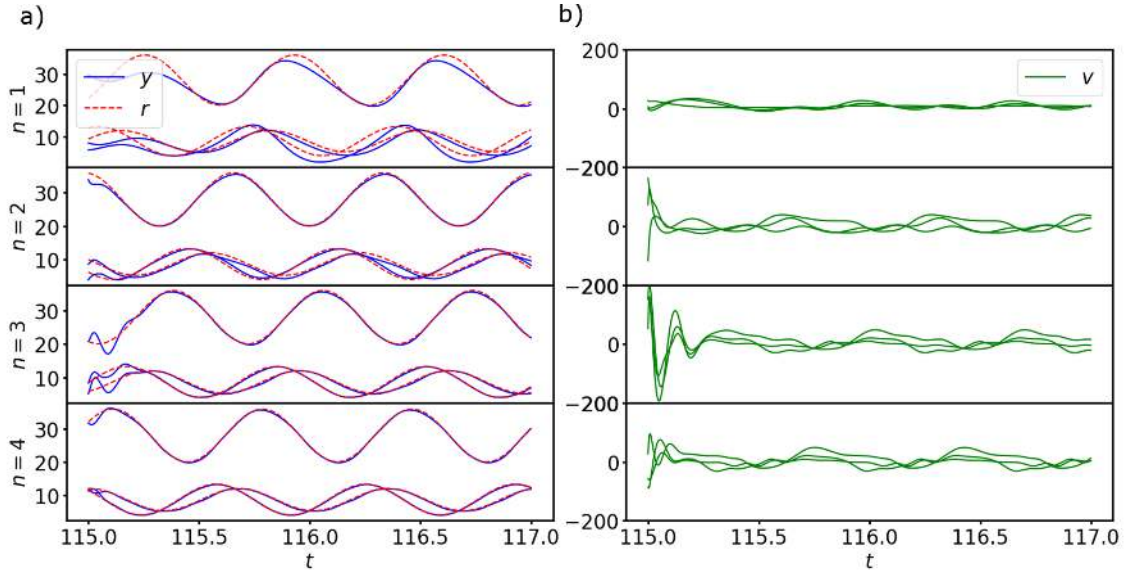


FIGURE 3.10: The control of the Lorenz system to an ellipse near the attractor. From top to bottom, the number of layers in the controller is increased from $n = 1$ to $n = 4$. From the right panels, the control signal often needs a large initial perturbation to move Lorenz to the requested ellipse.

3.4.6 Synchronization

I present one final example with the Lorenz system, both to illustrate the complete range of control laws that are possible and to provide support for the intuition described in Sec. 3.3 that additional layers to the controller improve the error, in part, because the attractors of the controlled plant are successively closer to the desired attractor.

An important application in chaos control is the synchronization of similar or identical systems. In the absence of a control signal, two distinct systems will eventually diverge from each other in the presence of any noise. The goal of synchronization is to keep these systems close to each other with a small control signal.

In the control framework introduced in this chapter, synchronizing two Lorenz systems means one system is the plant and the other is the reference system, *i.e.*, unidirectional synchronization. Note that this induces delayed synchronization rather than identical synchronization, but the latter can be obtained with an additional reservoir used to predict the reference system

a time δ ahead. Importantly, the reference attractor is the same as the original attractor. According to the motivation for the deep algorithm outlined in Sec. 3.3, this suggests adding layers won't improve control performance. Indeed, this is what I see in Fig. 3.11. Even after adding 10 layers, the synchronization error is not improved from the first layer.

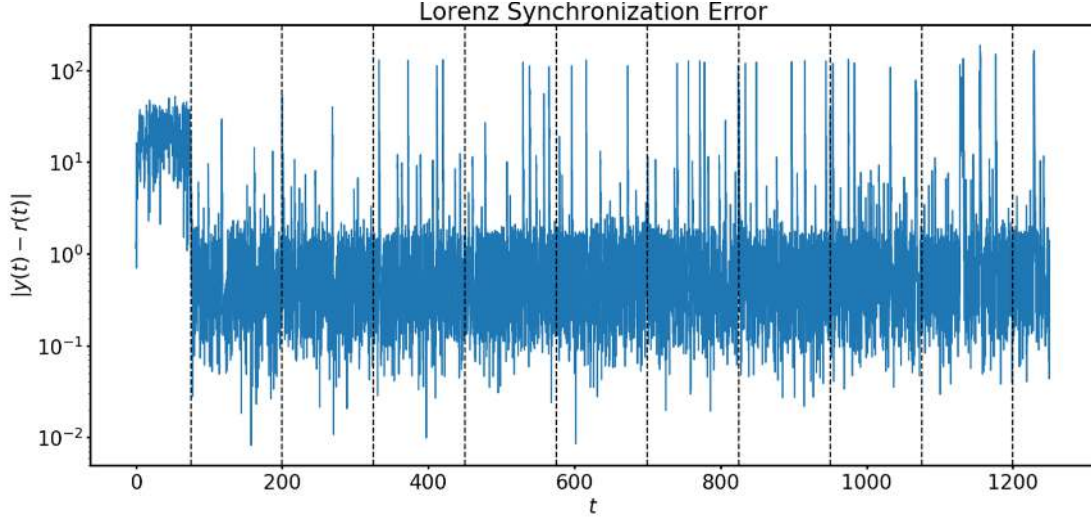


FIGURE 3.11: The synchronization (control) error for two Lorenz systems. Additional layers of the controller are switched on at every vertical dashed line. After one reservoir, the systems are synchronized with error ranging between 1 and 0.1. However, because the attractor is unchanged, additional layers do not improve performance, even up to 10 layers.

To improve synchronization error in the Lorenz systems, one alternative strategy is to use a smaller training signal magnitude g with a larger reservoir. This is based on the knowledge that only small perturbations are required to synchronize identical chaotic systems. While it remains true that, for N sufficiently large and fixed hyperparameters, increasing N does not improve control performance, I see from Fig. 3.12 that a smaller g and larger N yields improved synchronization. Increasing N is necessary because the minimum working N depends on g .

One interpretation of the results in Fig. 3.12 is that it is best for the reservoir to learn small perturbations to the plant for the synchronization task, but a large reservoir is necessary to learn the small effects of these perturbations. For a reservoir too small, the systems do not synchronize and the effect of the controller is simply noise, as I saw in Sec. 3.2.2 with the

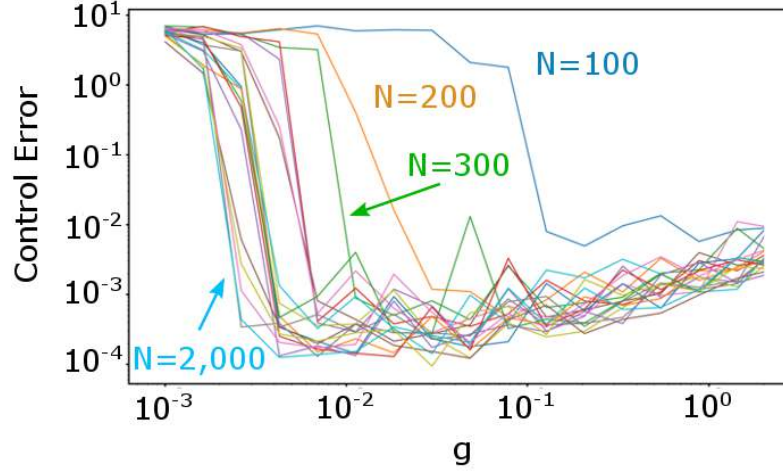


FIGURE 3.12: The control error as functions of g for different reservoir sizes N . For a fixed g , control error is unchanged by N above a certain minimum N . However, this minimum depends on g , so better performance can be obtained by simultaneously increasing N and decreasing g .

Mackey-Glass system.

3.5 Experimental Circuit

In this section, I discuss the control of a high-speed, chaotic electronic circuit. The circuit consists of passive linear components, nonlinear signal diodes, and an active negative resistor (Chang *et al.*, 1998), and is shown schematically in Fig. 3.13a. The circuit has dynamics described by

$$\begin{aligned}
 C_1 \dot{V}_1 &= \frac{V_1}{R_n} - g(V_1 - V_2) + q_1 \\
 C_2 \dot{V}_2 &= g(V_1 - V_2) - I + q_2 \\
 L \dot{I} &= V_2 - R_m I + q_3 \\
 g(V) &= \frac{V}{R_d} + 2I_r \sinh\left(\alpha \frac{V}{V_d}\right)
 \end{aligned} \tag{3.13}$$

Parameter	C_1	C_2	L	R_n	R_m	R_d	I_r	α	V_d
Value	10 nF	10 nF	55 mH	3.00 k Ω	455 Ω	7.86 k Ω	5.63 nA	11.6	0.58 V

TABLE 3.3: The values of the parameters describing the circuit in Eq. 3.13. All values are measured within 1%.

where $V_1(V_2)$ is the voltage drop across capacitor $C_1(C_2)$, I is the current through the inductor, $q_1(q_2)$ is an accessible bias current into the $V_1(V_2)$ -node, and q_3 is an accessible bias voltage across the inductor. The circuit's parameters are measured experimentally and listed in Table 3.3. The attractor of the unperturbed circuit ($\mathbf{q} = 0$) is in Fig. 3.13b. Similar to Lorenz, the circuit has a USS at the origin and two symmetric points at $(\pm V_1^{ss}, \pm V_2^{ss}, I^{ss})$. The error level is determined by adjusting R_n so that the circuit becomes stable at a fixed point and measuring the RMSE of the signal. This noise level is used in the simulations discussed in this section, as well as to contextualize the achieved control errors.

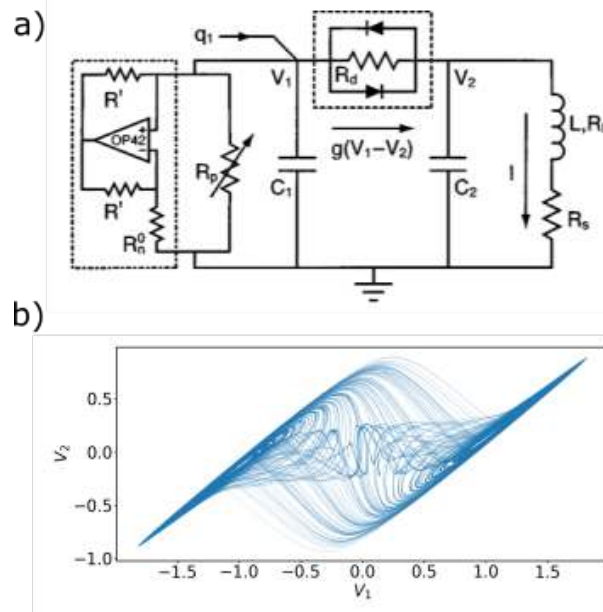


FIGURE 3.13: The chaotic circuit to be controlled. a) A schematic description of the circuit. Parameter values are given in Table 3.3. b) The attractor of the unperturbed, simulated circuit.

The system described by Eq. 3.13 and Table 3.3 exhibits chaotic oscillations up to 10 kHz.

I seek to control the circuit with a 2-layer reservoir controller whose dynamics I simulate on a field-programmable gate array (FPGA). In particular, I use a Max 10 10M50DAF484C6G Device on a Terasic Max 10 Plus development board. The device includes integrated dual 12-bit ADCs that operate up to 1 MHz, and the board includes a 16-bit DAC that operates at 1 MHz.

I use the ADCs to make simultaneous measurements of V_1 and V_2 , which I use to evolve Eq. 3.10 directly on the FPGA. I use the DAC to produce a voltage ($\mathbf{v}_{train}$ during the training period or $\mathbf{v}$ during the control period) that I send through a voltage-to-current converter with variable gain. The current is then injected into the V_1 node. To concretely describe the circuit and controller in terms of the notation used in the previous sections, note that this means I have $\mathbf{x} = (V_1, V_2, I)$, $\mathbf{y} = (V_1, V_2)$, and $u = q_1$.

3.5.1 FPGA-Accelerated Controller

As noted in the previous section, the circuit of interest possesses fast chaotic dynamics on the order of $100 \mu s$. For a controller to be sufficiently sensitive to these dynamics, it must sample, process, and produce an output many times during this short period. An efficient way to accomplish this is by simulating the ESNs with dedicated logic, using an FPGA to speed-up the calculations.

Much research is devoted to accelerating the calculation of neural network equations with FPGAs. Attention is given to the power required, the area of logic elements required, and the time per update cycle. While many neural networks are trained based on backpropagation of an error term and therefore require high-precision, floating-point calculations, ESNs work well with low-precision, fixed-point calculations down to as few as 8 bits (Büsing, Schrauwen, and Legenstein, 2010). This makes FPGA-implementations of ESNs highly efficient.

To construct the dESN controller, I employ 32-bit, fixed-point calculations and an Euler integration method for the controller in Eq. 3.7. To greatly reduce hardware space, the matrices $\mathbf{W}$, $\mathbf{W}_{in}$, and $\mathbf{b}$ and the time constant c are hard-coded at the time of design compilation. Conversely, the values of the output matrix $\mathbf{W}_{out}$ are stored in on-board RAM and updated

mid-operation by a host computer. The output is then calculated by evaluating $\mathbf{W}_{out}\mathbf{x}$ with dedicated multipliers and adders. The tanh function is implemented with a 10-bit lookup table. The ADC, DAC, and ESNs are synchronized to a 1 MHz global clock.

3.5.2 Control Results

To study the efficacy of the dESN controller on the experimental circuit, I consider three different control tasks, each characterized by a different reference trajectory $\mathbf{r}(t)$. The first trajectory $\mathbf{r}(t) = \mathbf{0}$ describes stabilizing the origin. The second describes a smooth but fast transition between the symmetric USSs described in Sec. 3.5.1. The last trajectory is an ellipse with parameters determined similarly to the ellipse in the Lorenz system in Sec. 3.4.3. The reference trajectories are loaded into on-board RAM similarly to how output weights are stored.

The hyperparameters are selected according to the reasoning outlined in Sec. 3.2.1 and are listed in Table 3.4. Additionally, simulations of the circuit and controller for these trajectories and control parameters are done for $n = 1 - 4$ layers, both to confirm experimental results and to examine the likely effects of deeper controllers. The simulations are with a 4th order Runge-Kutta method and fix integration step size $h = 0.1\mu s$.

Typical trajectories of the controlled circuit are displayed in Fig. 3.14-3.16. The real-space and phase-space plots of the circuit and the reference trajectory are given, as well as the control signal in real space. They are constructed from data collected by the ADCs and stored in on-board RAM, as described in this section.

As seen from Fig. 3.14b, the controller initially exerts a large control effort when the first reservoir is switched on. This is because the state of the circuit at $t = 80 \mu s$ is far from the origin, requiring the controller to exert a large perturbation to move the circuit to the requested USS. As seen from the middle panel of Fig. 3.14a or the insert in Fig. 3.14b, the circuit under the influence of the one-layer controller has a DC offset, not quite settling down to a mean value of 0. The variation in the circuit is however comparable to the measured RMS noise level in the circuit of 13 mV.

Hyperparameter	Task 1 Value	Task 2 Value	Task 3 Value
N	30	30	30
ρ	0.9	0.9	0.9
k	3	3	3
σ	0.95	0.95	0.95
b_{mean}	0	0	0
b_{max}	0.5	0.5	0.5
c	$24 \mu s$	$24 \mu s$	$24 \mu s$
δ	$24 \mu s$	$8 \mu s$	$32 \mu s$
λ_{train}	$64 \mu s$	$24 \mu s$	$48 \mu s$
g	$22.5 \mu A$	$22.5 \mu A$	$22.5 \mu A$
T_{init}	$512 \mu s$	$512 \mu s$	$512 \mu s$
T_{train}	$8192 \mu s$	$8192 \mu s$	$8192 \mu s$
β	10^{-8}	10^{-8}	10^{-8}

TABLE 3.4: The hyperparameters used to control the experimental circuit for the various control tasks. Note that the hyperparameters describing the physical reservoir ($N, \rho, k, \sigma, b_{mean}, b_{max}$, and c) are identical for all three tasks. That is, one only needs to change the control hyperparameters to target a new trajectory.

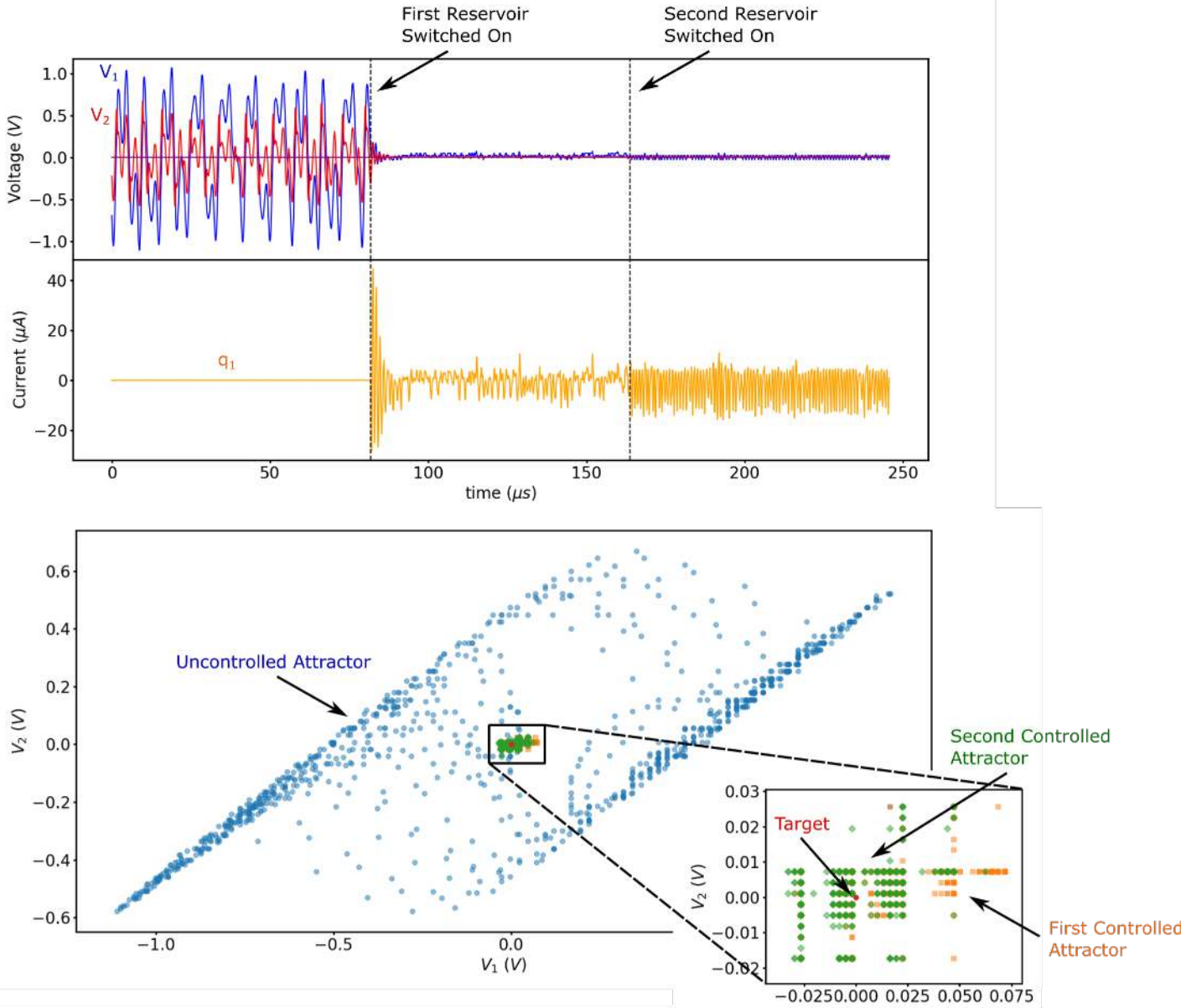


FIGURE 3.14: Control of the experimental circuit to the origin. a) In real space, the circuit is stabilized to the origin quickly after the first reservoir is switched on, but with a small DC shift. When the second reservoir is switched on, the circuit moves closer to the origin. b) In phase space, the target lies at the center of the attractor. Noise leads to a spread in the asymptotic behavior of the plant controlled with the first and second controlled system.

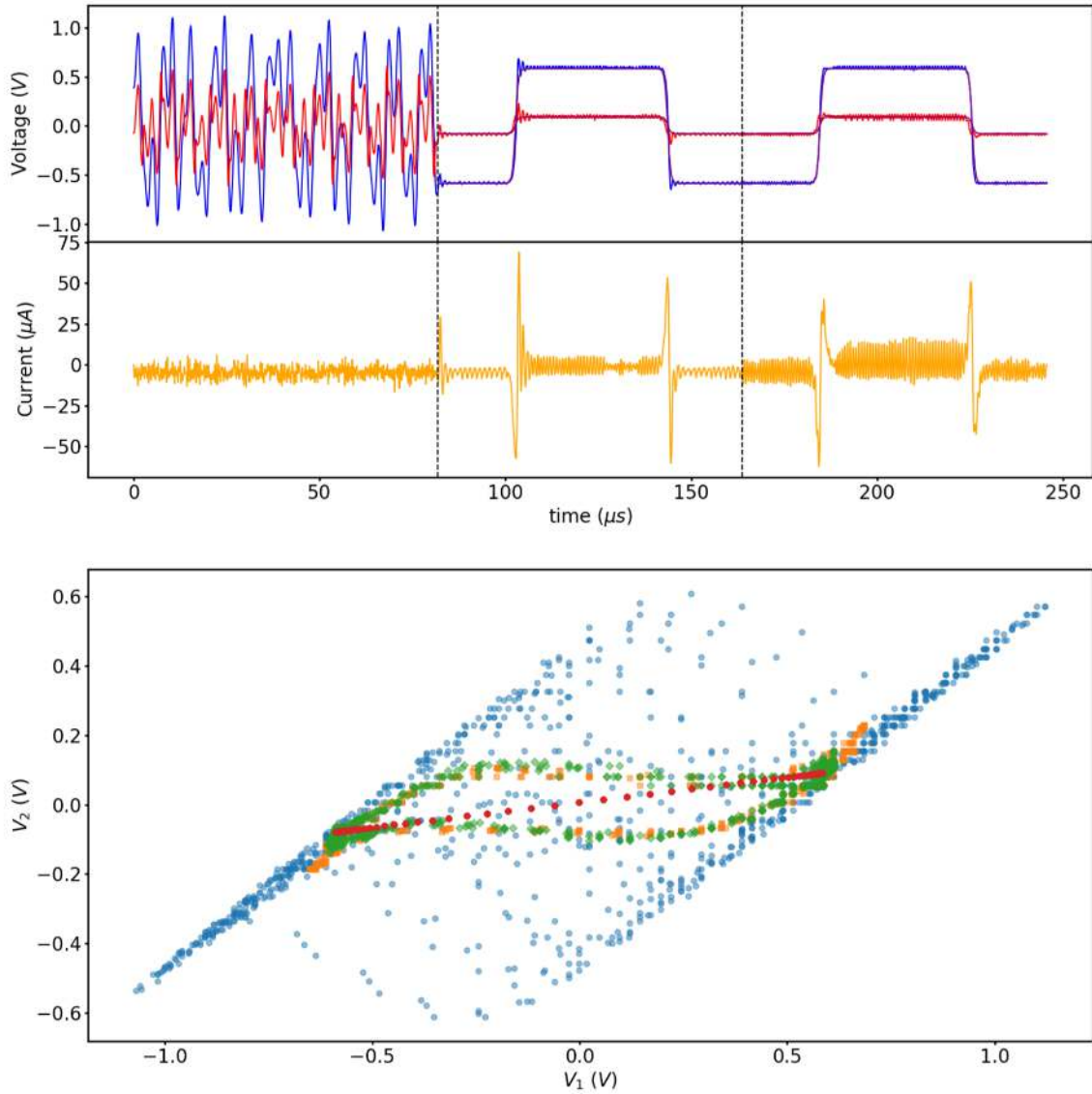


FIGURE 3.15: Control of the experimental circuit between USs. a) In real space, the first controller leads to substantial ringing after the circuit is moved. The second reservoir substantially reduces this. b) In phase space, it appears that dragging straight across the attractor is an unnatural trajectory for the circuit.

When the second controller is turned on, an initial large perturbation is no longer required, because the circuit is already settled near the origin. As seen from the right panel of Fig. 3.14a,

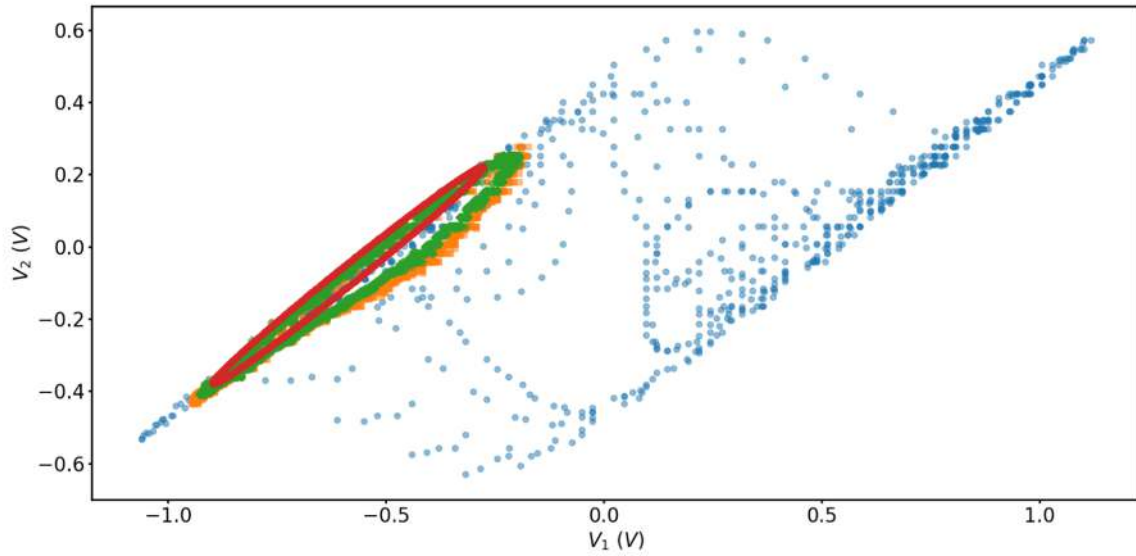
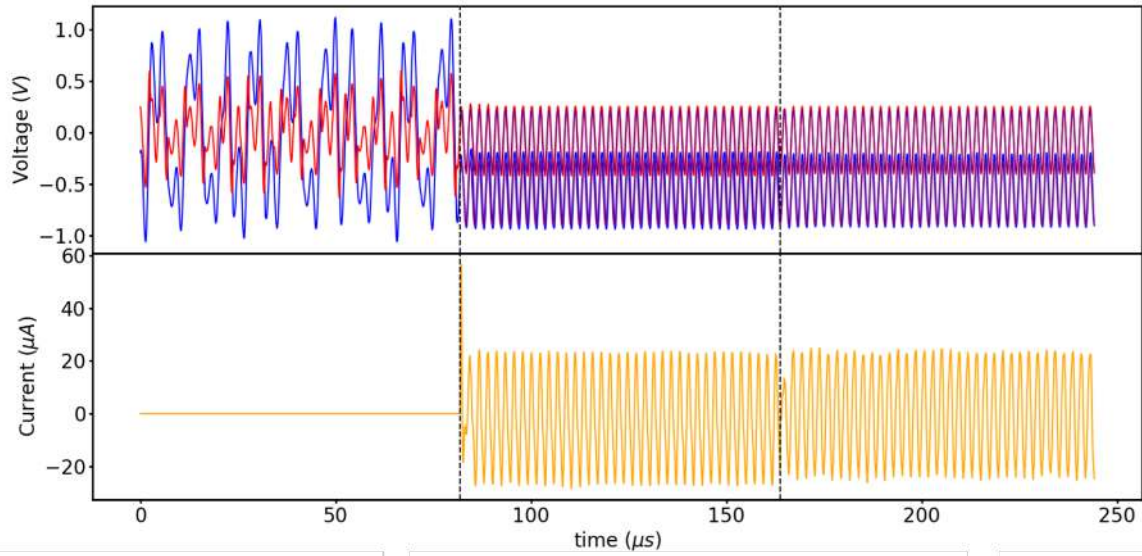


FIGURE 3.16: The control of the experimental circuit to an ellipse. a) A periodic input current stabilizes an ellipse trajectory in the circuit. b) The circuit tends to “slip” away from the ellipse, as can be seen from phase space. The second controller partially remedies this, bringing the circuit closer to the desired ellipse.

the reservoir controller produces a higher-frequency signal, indicating that it is responding more quickly to correct the impact of noise fluctuations. As is clear from the second controlled attractor in the insert in Fig. 3.14b, the mean of the circuit is now much closer to the origin for

both V_1 and V_2 values—that is, the second reservoir learned to correct the DC offset that was present in the plant controlled by the single-layer controller.

Notable in this example is the fact, as seen from Fig. 3.14b, that the uncontrolled circuit very rarely visits near the origin. It rather spends much of its time around the two scrolls. This is suggestive of why the two-layer approach is particularly effective here: the first layer brings the circuit near the requested USS so that the second controller can learn to control the plant dynamics in the actual neighborhood of interest.

In Fig. 3.15, it is apparent that the switching control task is more difficult than the origin control task. This is indicated by the larger deviations from $\mathbf{r}(t)$ in Fig. 3.15a. It appears from Fig. 3.15b that this error is due to two separate difficulties. First, there are DC offset errors near the opposite USSs, similar to the errors in the origin control example. There is additionally a ringing effect after the transition, as is particularly clear in real space in the middle panel of Fig. 3.15a. Second, the requested path straight across the attractor as indicated by red dots in Fig. 3.15b appears to be an unnatural path in phase space for the circuit. The circuit prefers to take a sigmoidal path as indicated by the orange and green dots. Note from Fig. 3.15b that the circuit requires strong and opposing kicks to move from one USS to the other.

Curiously, it appears that the first of these errors sources is much easier to fix by the second reservoir. The ringing effect is significantly reduced, but Fig. 3.15b indicates that the circuit still takes the same, curved trajectory between USSs. However, simulation results (see next section) suggests that this type of error is also possible to fix with even deeper reservoir controllers.

To quantify these results, the control task is repeated a total of 30 times per task with 5 different realizations of ESNs. The mean performance as characterized by the RMSE of the control error over one period. Similarly, 15 different reservoirs are simulated and applied to these control tasks. The mean performance for the experimental and numerical controllers are presented in Fig. 3.17.

Finally, a typical ellipse control result is presented in Fig. 3.16. As evident from the consistently large control signal in Fig. 3.16b, this orbit is neither a USS or UPO and can therefore not

be controlled by classical chaos control methods. It appears, perhaps not too surprisingly, that an oscillating control signal is required to maintain the oscillating circuit outputs.

It is less clear from the real-space curves in Fig. 3.16a what improvement is made with the additional reservoir. The improvement, as well as the original difficulty, is more clear in phase-space in Fig. 3.16b. The circuit trajectory appears difficult to maintain on the part fold of the attractor, where the circuit tends to slip towards the origin briefly. As evident by the green and orange curves, the second controller learns to more tightly control the circuit and prevent the slipping. It is also observed from the bottom-left portion of the ellipse that the circuit subject to the single-layer controller is more prone to oscillating with too large of an amplitude at this portion of the attractor, which is also partially mitigated by the second reservoir.

It is clear from Fig. 3.16 that this is the more difficult of the control tasks. From simulation results in 3.17, the other control tasks approach the noise level after 2 or 3 layers. Although the ellipse task continues to improve up to 4 layers, it does not quite reach the noise level, although many more layers might accomplish this.

From Fig. 3.15, for $n = 1, 2$ there is qualitative agreement with experimental and numerical results. Consistent with results for the Lorenz system and with the traces in Fig. 3.14-3.16, control error significantly improves as layers are added. The order of the tasks by their measured error is as described above. However, the experimental error is consistently worse than the simulated error. This is potentially due to measurement delays by the ADC and DAC that make the experimental task more difficult. For the origin and dragging tests, control error approach the noise level in the circuit after $n = 4$ layers.

3.6 Conclusions

In this chapter, I have introduced a method for control of arbitrary dynamical systems to arbitrary trajectories. It requires no knowledge of the plant, and is therefore completely model-free. Unlike other model-free techniques, the control law is learned directly, rather than through

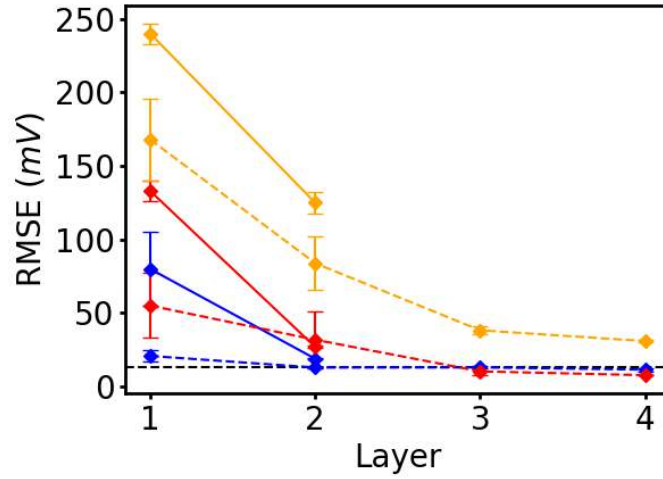


FIGURE 3.17: The RMSE of the settled circuit, versus the number of reservoirs, for the origin (blue), dragging (red), and ellipse (orange) control tasks described in the text. Experimental results from 30 different trials are in solid lines and are limited to two reservoirs. Numerical simulation results from 15 different trials are in dashed lines and go up to four reservoirs. The horizontal dashed line represents the RMS noise level in the circuit.

an initial system identification step. The algorithm is capable of controlling complex chaotic systems and is robust to the noise and non-ideal properties of physical systems. It can be implemented with a compact FPGA and used to control fast experimental systems. This work paves the way for research into control engineering with reservoir computing and provides a sufficient grounding to apply to real-world problems, as I have demonstrated.

This research suggests several future directions in control engineering and RC more generally. First, a rigorous stability analysis is required. While this is notoriously difficult when recurrent neural networks are involved, many safety standards require such a proof before deploying a control system when humans are involved. Second, the application of optimization methods is not well understood in this domain of RC. The issue is particularly salient here, given the increased number of hyperparameters. Particularly interesting is whether optimizations can be made by relaxing the constraint that all ESNs have the same set of hyperparameters. It may instead be the case that, say, deeper ESNs require different time constants, because

the local Lyapunov spectrum is different from the controlled and uncontrolled plants.

Chapter 4

Reservoir Computing with Autonomous, Boolean Networks

One of the principle appeals of the RC framework is the ability to simultaneously use a single dynamical system for multiple and often disparate computational tasks, from recognition of handwritten digits to emulation of a chaotic time series. This is contrary to machine learning frameworks such as deep learning, where the entire network is adapted to a specific task. Another appeal is the fact that one never needs to know the dynamics of the reservoir; it is only necessary to measure the response. This grants the freedom to use exotic media in place of a traditional neural network, even when simulating the medium's dynamics may be computationally intractable.

These advantages of RC have led to a wide-ranging search for novel, dedicated hardware to function as the reservoir (see Tanaka *et al.*, 2019 for a modern review). Once identified or fabricated, such a reservoir can be used as a neuromorphic computing device for a variety of tasks. Further, because the dynamics need not be simulated on a von Neumann machine, there exists the possibility of beyond-Turing computing (Larger *et al.*, 2012) with dedicated hardware RC.

In this chapter, I develop a technique for RC with autonomous, Boolean networks (ABNs)

constructed on field-programmable gate arrays (FPGAs). In addition to the advantages described above, the ABN reservoir computer has a minimally-complex reservoir state, which allows for rapid calculation of the output layer, thereby minimizing decision latency. Combined with the GHz processing potential of the ABN itself, the ABN reservoir computer particularly excels at time-series prediction tasks, which require the reservoir output to be fed in to the reservoir as successive inputs. Here, I demonstrate that the ABN reservoir computer is capable of autonomously generating a machine-learned signal at up to 160 MHz, faster than any previously known technique.

The rest of this chapter is organized as follows: First, I further describe the particular challenges of time-series prediction with physical reservoir computers, including approaches with other dedicated-hardware RC techniques. Next, I describe FPGAs—the electronic platform in which the ABN reservoir computer is constructed. I then detail the construction of the ABN reservoir computer, including the actual ABN as well as the synchronous components that form the input and output layers. Finally, I use the ABN reservoir computer to forecast a chaotic time-series and analyze the resulting data.

The major results in this chapter have previously appeared in Canaday, Griffith, and Gauthier, 2018 and are the subject of the patent Canaday, Griffith, and Gauthier, "Rapid Time-Series Prediction with an FPGA-Based Reservoir Computer." PCT/US2019/024296, filed March 27th, 2019. My principal conceptual contributions are the design of the synchronous components and the binary representation scheme. I collected and analyzed all of the data presented in this chapter.

4.1 Challenges of Real-Time Prediction

A task at which RC consistently yields state-of-the-art results is time-series prediction (Li, Han, and Wang, 2012; Wyffels and Schrauwen, 2010), where the goal is to predict the future value

of the series given a segment of its history. This is commonly achieved with a technique introduced early in the RC literature (Jaeger, 2001) in which the desired reservoir output is equal to the reservoir input. After training is complete, the input is replaced by the trained reservoir output ("closing the loop"), and the reservoir is allowed to evolve autonomously. If successful, the trained reservoir emulates the system that generated the observed time-series and thereby makes predictions for any prediction horizon.

To be more explicit, consider a time-series $\mathbf{u}(t)$ that is observed for $0 \leq t \leq T$. Then the dynamics of a trained ESN are given by

$$c\dot{\mathbf{x}} = \begin{cases} -\mathbf{x} + \tanh(\mathbf{W}\mathbf{x} + \mathbf{W}_{in}\mathbf{u} + \mathbf{b}) & 0 \leq t \leq T \\ -\mathbf{x} + \tanh(\mathbf{W}\mathbf{x} + \mathbf{W}_{in}\mathbf{W}_{out}\mathbf{x} + \mathbf{b}) & t > T. \end{cases} \quad (4.1)$$

The prediction for $\mathbf{u}(t_p)$ is then simply $\mathbf{W}_{out}\mathbf{x}(t_p)$.

Viewed this way, prediction with the ESN is simple—it just amounts to solving a differential equation. A difficulty arises, however, when the reservoir is a physical system. This is due to the fact that $\mathbf{W}_{out}\mathbf{x}$ cannot be computed instantaneously, but rather requires some finite time. This time can be thought of as a propagation delay through the output layer that must be considered. I emphasize these problems in the next subsection with a discussion of some existing physical reservoir computers.

4.1.1 Physical RC

As I emphasize in the introduction to this chapter, RC with novel, physical media is possible because the RC scheme only requires that the reservoir be stimulated with an input and a response observed. As an extreme proof of this principle, an early example of physical RC was with a bucket of water (Fernando and Sojakka, 2003). This experiment used different media for the input, reservoir, and output layers, which were implemented with a vibrating motor, a bucket of water, and laser sensors followed by a computer program, respectively.

A wide range of physical implementations of RC have been explored since, including memristor networks (Du *et al.*, 2017), physical oscillators (Caluwaerts *et al.*, 2014), skyrmions (Torrejon *et al.*, 2017), and many others—see Tanaka *et al.*, 2019 for a more complete review. One of the most heavily researched techniques is based on a single optical element with delayed feedback, often referred to as photonic or optical RC (Appeltant *et al.*, 2011). The technique utilized the more general concept of RC with delay dynamics and has been extensively applied since its introduction to a wide range of benchmark tasks (Sande, Brunner, and Soriano, 2017).

4.1.2 Real-Time Prediction with Optical RC

Made with optical elements, the processing speed of optical RC has incredible potential. In a widely-cited feat, the scheme is shown to be capable of processing spoken digits at a rate of over 1 million per second (Larger *et al.*, 2017). However, examples such as this report the impressive *information throughput* but not the less impressive *decision latency*, or the time it takes to make a classification or output after the appropriate inputs have been processed by the reservoir. This is typically done offline with a host computer, after the reservoir data has been collected. This classification step itself takes much longer than the time required to stimulate the reservoir with the millions of spoken words.

Though less important for classification tasks, the real-time processing of the output layer is critical for signal generation and time-series prediction, which require the output to be fed back into the input. Since the reservoir in the optical case is a physical system that cannot be "paused" like software can, the input / output signals must be structured in such a way that a suitable output layer can compute the "next" reservoir input in a required time, such as with a sample-and-hold procedure. This was first applied to optical RC in Antonik *et al.*, 2016, where a high-speed FPGA was used to read input voltages, calculate the required matrix transformation, and produce appropriate output voltages. A long optical fiber was used to sufficiently slow down the reservoir dynamics, and pattern generation at a 30 MHz rate was achieved.

Another approach is to compute the linear transformation itself with optical elements, creating an all optical reservoir computer. Although realized in principle (Bueno *et al.*, 2017), the errors in the output computation are sufficiently large such that errors propagate quickly, resulting in poor performance complex, real-world tasks such as generation of a chaotic signal.

4.2 Field-Programmable Gate Arrays

The principle hurdles towards fast, real-time prediction with optical RC are general. I identify them as:

- the separation of reservoir and input / output architectures, requiring transfer delays, and
- the complexity of performing the real-valued matrix transformation.

These problems are both overcome with the ABN reservoir computer, which realizes both the reservoir and input / output layers on a commercial device known as an FPGA.

Field-programmable gate arrays are semiconductor devices with matrices of reconfigurable logic blocks with reconfigurable inter- and intraconnections. Although often used to emulate a finite state machine, these individual logic blocks are highly nonlinear, Boolean-like dynamical systems that can be used for RC when properly configured.

4.2.1 Synchronous versus Autonomous Logic

Field-programmable gate arrays are most often used to speed up floating- or fixed-point operations, and thus are heavily reliant on *deterministic, repeatable* operations. To ensure that this is the case, FPGAs are operated with synchronous logic, where operations are separated by elements called registers, which hold their input value each clock cycle. Synchronous FPGA designs are therefore always in a steady state at the end of a clock cycle, making them effectively finite state machines.

On the other hand, logic can be asynchronous or *autonomous*, where steady states before a register is not required. In this usage, the details of how the silicon operates is of critical importance, and dynamical recurrent loops are possible. These details include the propagation delay through routing wires, the finite response time of logic elements, thresholding variables, and complex hysteresis effects.

4.2.2 FPGA-Accelerated RC

As a point of emphasis, I note an area of related but distinct research devoted to accelerating artificial neural networks, such as ESNs, with FPGAs. Although an important area of focus, and one which I draw on myself in Ch. 3, this is distinct from physical RC techniques, which use a physical dynamical system as the reservoir. Hardware-accelerated RC, on the other hand, simply seeks effective methods for integrating differential equations such as Eq. 4.1. Although the ABN reservoir computer is fabricated on FPGAs, it is *not* simply a hardware-accelerated neural network; rather, it utilizes a complex, analogue reservoir with time-delay dynamics, as I make clear in the next section when I describe the ABN construction.

4.3 Autonomous Boolean Reservoirs

I investigate a reservoir construction based on an autonomous, time-delay, Boolean reservoir realized on an FPGA. By forming the nodes of the reservoir out of FPGA elements themselves, this approach exhibits faster computation than FPGA-accelerated neural networks (Schrauwen *et al.*, 2008a; Alomar *et al.*, 2016), which require explicit multiplication, addition, and non-linear transformation calculations at each time-step. My approach also has the advantage of realizing the reservoir and the readout layer on the same platform without delays associated with transferring data between different hardware. Finally, due to the Boolean-valued state of the reservoir, a linear readout layer $\mathbf{v}(t) = \mathbf{W}_{out}\mathbf{X}(t)$ is reduced to an addition of real numbers

rather than a full matrix multiplication. This allows for much shorter total calculation time and thus faster real-time prediction than in opto-electronic RC (Antonik *et al.*, 2016).

The choice of reservoir is further motivated by the observation that Boolean networks with time-delay can exhibit complex dynamics, including chaos (Zhang *et al.*, 2009). In fact, a single XOR node with delayed feedback can exhibit a fading memory condition and is suitable for RC on simple tasks such as binary pattern recognition (Haynes *et al.*, 2015).

The dynamics of these complex ABNs can be approximately described (Apostel, 2017) by a Glass model (Glass and Kauffman, 1973) given by

$$\gamma_i \dot{x}_i = -x_i + \Lambda_i(X_{i1}, X_{i2}, \dots), \quad (4.2)$$

$$X_i = \begin{cases} 1 & \text{if } x_i \geq q_i, \\ 0 & \text{if } x_i < q_i, \end{cases} \quad (4.3)$$

where x_i is the continuous variable describing the state of the node, γ_i describes the time-scale of the node, q_i is a thresholding variable, and Λ_i is the Boolean function assigned to the node. The thresholded Boolean variable X_{ij} is the j_{th} input to the i_{th} node.

I construct the Boolean reservoir by forming networks of nodes described by Eq. 4.2-4.3 and the Boolean function

$$\Lambda_i = \Theta \left(\sum_j W^{ij} X_j + W_{in}^{ij} u_j \right), \quad (4.4)$$

where u_j are the bits of the input vector $\mathbf{u}$, $\mathbf{W}$ is the reservoir-reservoir connection matrix, $\mathbf{W}_{in}$ is the input-reservoir connection matrix, and Θ is the Heaviside step function defined by

$$\Theta(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{if } x \leq 0. \end{cases} \quad (4.5)$$

The matrices $\mathbf{W}$ and $\mathbf{W}_{in}$ are chosen as follows. Each node receives input from exactly k

other randomly chosen nodes, thus determining k non-zero elements of each row of $\mathbf{W}$. The non-zero elements of $\mathbf{W}$ are given a random value from a uniform distribution between -1 and 1 . The maximum absolute eigenvalue (spectral radius) of the matrix $\mathbf{W}$ is calculated and used to scale $\mathbf{W}$ such that its spectral radius is ρ . A proportion σ of the nodes are chosen to receive input, thus determining the number of non-zero rows of $\mathbf{W}_{in}$. The non-zero values of $\mathbf{W}_{in}$ must be chosen carefully (see Sec. 4.4.2), but I note here that the scale of $\mathbf{W}_{in}$ does not need to be tuned, as it is apparent from Eq. 4.4 that only the relative scale of $\mathbf{W}$ and $\mathbf{W}_{in}$ determines Λ_i .

The three parameters defined above— k, ρ , and σ —are the hyperparameters that characterize the topology of the reservoir. I introduce a final parameter $\bar{\tau}$ in the next section, which I show characterizes the global time-scale of the ABN. Together, these four hyperparameters describe the reservoirs that I investigate in this work.

4.3.1 Matching Time Scales with Delays

The presence of the $-x_i$ term in Eq. 4.2 represents the sluggish response of the node, *i.e.*, its inability to change its state instantaneously. This results in an effective propagation delay of a signal through the node. I take advantage of this phenomenon by connecting chains of pairs of inverter gates between nodes. These inverter gates have dynamics described by Eq. 4.2-4.3 and

$$\Lambda_i(X) = \begin{cases} 0 & \text{if } X = 1, \\ 1 & \text{if } X = 0, \end{cases} \quad (4.6)$$

Note that the propagation delay through these nodes depends on γ_i and q_i , both of which are heterogeneous throughout the chip due to small manufacturing differences. I denote the mean propagation delay through the inverter gates by τ_{inv} , which I measure by recording the oscillation frequencies of variously sized loops of these gates. For the Arria 10 devices considered

here,¹ I find $\tau_{inv} = 0.19 \pm 0.05$ ns.

I exploit the propagation delays by inserting chains of pairs of inverter gates in between reservoir nodes, thus creating a time-delayed network. I fix the mean delay $\bar{\tau}$ and randomly choose a delay time for each network link. This is similar to how the network topology is chosen by fixing certain hyperparameters and randomly choosing $\mathbf{W}$ and $\mathbf{W}_{in}$ subject to these parameters. The random delays are chosen from a uniform distribution between $\bar{\tau}/2$ and $3\bar{\tau}/2$ so that delays on the order of τ_{node} are avoided.

The addition of these delay chains is necessary because the time-scale of individual nodes is must faster than the speed at which synchronous FPGA logic can change the value of the input signal (see Sec. 4.4). Without any delays, it is impossible to match the time-scales of the input signal with the reservoir state, and I have poor RC performance. I find that the time-scales associated with the reservoir's fading memory are controlled by $\bar{\tau}$, as described in the next section, thus demonstrating that I can tune the reservoir's time-scales with delay lines.

4.3.2 Fading Memory

For the reservoir to learn about its input sequence, it must possess the fading memory property. Intuitively, this property implies that the reservoir state $\mathbf{X}(t)$ is a function of its input history, but is more strongly correlated with more recent inputs. More precisely, the fading memory property states that every reservoir state $\mathbf{X}(t_0)$ is uniquely determined by a left-infinite input sequence $\{\mathbf{u}(t) : t < t_0\}$.

The fading memory property is equivalent (Jaeger, 2001) to the statement that, for any two reservoir states $\mathbf{X}_1(t_0)$ and $\mathbf{X}_2(t_0)$ and input signal $\{\mathbf{u}(t) : t > t_0\}$, I have

$$\lim_{t \rightarrow \infty} \|\mathbf{X}_1(t) - \mathbf{X}_2(t)\|_2 = 0. \quad (4.7)$$

¹I use an Arria 10 SX 10AS066H3F34I2SG chip for the results discussed in this chapter.

Also of interest is the characteristic time-scale over which this limit approaches zero, which may be understood as the Lyapunov exponent of the coupled reservoir-input system conditioned on the input.

I observe the fading memory property and measure the corresponding time-scale with the following procedure. I prepare two input sequences $\{\mathbf{u}_1(i\Delta t); -N \leq i \leq N\}$ and $\{\mathbf{u}_2(i\Delta t); -N \leq i \leq N\}$, where Δt is the input sample rate (see Sec. 4.4) and N is an integer such that $N\Delta t$ is sufficiently large. Each $\mathbf{u}_1(i\Delta t)$ is drawn from a random, uniform distribution between -1 and 1 . For $i \geq 0$, $\mathbf{u}_2(i\Delta t) = \mathbf{u}_1(i\Delta t)$. For $i < 0$, $\mathbf{u}_2(i\Delta t)$ is drawn from a random, uniform distribution between -1 and 1 . I drive the reservoir with the first input sequence and observe the reservoir response $\{\mathbf{X}_1(i\Delta t); -N \leq i \leq N\}$. After the reservoir is allowed to settle to its equilibrium state, I drive it with the second input sequence and observe $\{\mathbf{X}_2(i\Delta t); -N \leq i \leq N\}$. The reservoir is perturbed to effectively random reservoir states $\mathbf{X}_1(0)$ and $\mathbf{X}_2(0)$, because the input sequences are unequal for $i < 0$. For $i \geq 0$, the input sequences are equal, and the difference in Eq. 4.7 is calculated.

For a given reservoir, this procedure is repeated 100 times with different input sequences. For each pair of sequences, the state difference is fit to $\exp(-t/\lambda)$, and the λ 's are averaged over all 100 sequences. I call λ the reservoir's *decay time*. I find $\lambda > 0$ for every reservoir examined, demonstrating the usefulness of the chosen form of Λ_i in Eq. 4.4.

I explore the dependence of the decay time as a function of hyperparameter $\bar{\tau}$. As seen from Fig. 4.1, the relationship is approximately linear for fixed k, ρ , and σ . This is consistent with $\bar{\tau}$ being the dominate time-scale of the reservoir rather than τ_{node} , which is my motivation for including delay lines in my reservoir construction. The dependence of λ on the other hyperparameters defined in this section are explored in Sec. 4.6 along with corresponding results on a time-series prediction task.

FIGURE 4.1: Experimental observation of the fading memory property and decay time for varying $\bar{\tau}$. The network has 100 nodes and hyperparameters $k = 2, \rho = 1.5$, and $\sigma = 0.75$. Statistics are generated by testing five reservoirs for each set of hyperparameters. Vertical error bars represent the standard error of the mean. The relationship is approximately linear with a slope of 3.99 ± 0.45 .

4.4 Synchronous Components

Though the reservoir itself is formed of autonomous logic, the input and output layers must be formed of synchronous logic, sharing a global clock, to regulate the input and output of data, as well as the additions necessary to compute the final output. This division of the reservoir computer into synchronous and asynchronous components is illustrated in Fig. 4.2. I describe these components in this section in detail.

4.4.1 Input Layer

As discussed in Sec. 4.3, the reservoir implementation is an autonomous system without a global clock, allowing for continuously evolving dynamics. However, the input layer is a synchronous FPGA design that sets the state of the input signal $\mathbf{u}(t)$. Prior to operation, a sequence of values for $\mathbf{u}(t)$ is stored in the FPGA memory blocks. During the training period, the input layer sequentially changes the state of the input signal according to the stored values.

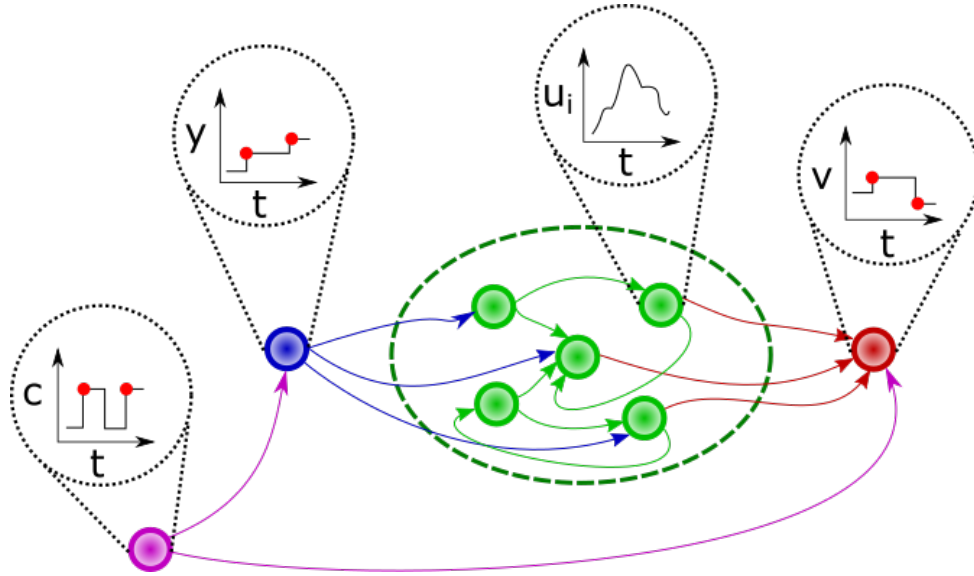


FIGURE 4.2: A schematic representation of the reservoir computer, divided into synchronous and asynchronous components. A global clock c drives the input and output layers. The values of y and v only change on the rising edge of the c , indicated on all synchronous components with red dots. On the other hand, the reservoir nodes $\mathbf{u}$ operate autonomously, evolving in between the rising edges of c .

For the prediction task, the stored values of $\mathbf{u}(t)$ are observations of some time-series from $t = -T_{train}$ to $t = 0$. This signal maybe defined on the entire real interval $[-T_{train}, 0]$, but only a finite sampling may be stored in the FPGA memory and presented as input to the reservoir. The signal may also take real values, but only a finite resolution at each sampling interval may be stored. The actual input signal $\mathbf{u}(t)$ is thus discretized in two ways:

- $\mathbf{u}(t)$ is held constant along intervals of length t_{sample} ;
- $\mathbf{u}(t)$ is approximated by an n -bit representation of real numbers.

A visualization of these discretizations is in Fig. 4.2. Note that t_{sample} is a physical unit of time, whereas Δt has whatever units (if any) in which the non-discretized time-series is defined.

As pointed out in Sec. 4.2, t_{sample} may be no smaller than the minimum time in which the clocked FPGA logic can change the state of the input signal, which is approximately 5 ns on the

Arria 10 device considered here. However, I show in Sec. 4.5 that t_{sample} must be greater than or equal to τ_{out} , which generally cannot be made as short as 5 ns.

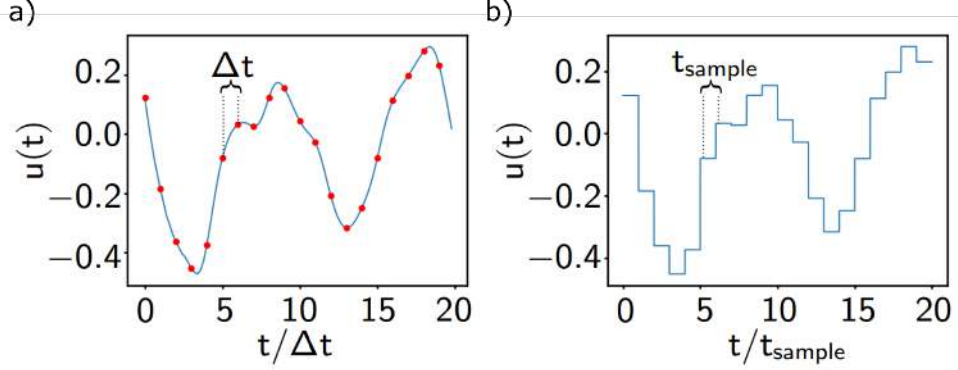


FIGURE 4.3: A visualization of the discretization of $\mathbf{u}(t)$ necessary for hardware computation. (a) In general, the true input signal may be real-valued and defined over a continuous interval. (b) Due to finite precision and sampling time, the actual $\mathbf{u}(t)$ seen by the reservoir is held constant over intervals of duration t_{sample} and have finite vertical precision. For the prediction task, $v_d(t) = u(t)$, so the output must be discretized similarly.

4.4.2 Binary Representations of Real Data

The Boolean functions described by Eq. 4.4-4.5 are defined according to Boolean values u_j , which are the bits in the n -bit representation of the input signal. If the elements of $\mathbf{W}_{in}$ are drawn randomly from a single distribution, then the reservoir state is as much affected by the least significant bit of $\mathbf{u}(t)$ as it is the most significant. This leads to the reservoir state being distracted by small differences in the input signal and fails to produce a working reservoir computer.

For a scalar input $u(t)$, I can correct for this shortcoming by choosing the rows of $\mathbf{W}_{in}$ such that

$$\sum_j W_{in}^{ij} u_j \approx \tilde{W}_{in}^i u, \quad (4.8)$$

where $\tilde{\mathbf{W}}_{in}$ is an effective input matrix with non-zero values drawn randomly between 1 and -1 . The relationship is approximate in the sense that u is a real-number and u_j is a binary representation of that number. For the two's complement representation, this is done by choosing

$$W_{in}^{i,j} = \begin{cases} -2^{(n-1)}\tilde{W}_{in}^i & \text{if } j = n, \\ +2^{(j-1)}\tilde{W}_{in}^i & \text{else.} \end{cases} \quad (4.9)$$

A disadvantage of the proposed scheme is that every bit in the representation of u must go to every node in the reservoir. If a node has k recurrent connections, then it must execute a $n + k$ to 1 Boolean function, as can be seen from Eq. 4.4. Boolean functions with more inputs take more FPGA resources to realize in hardware, and it takes more time for a compiler to simplify the function. I find that an 8-bit representation of u is sufficient for the prediction task considered here while maintaining achievable networks.

4.5 Output Layer

Similar to the input layer, the output layer is constructed from synchronous FPGA logic. Its function is to observe the reservoir state and, based on a learned output matrix $\mathbf{W}_{out}$, produce the output $\mathbf{v}(t)$. As I note Sec. 4.2, this operation requires a time τ_{out} that I interpret as a propagation delay through the output layer and requires that $\mathbf{v}(t)$ be calculated from $\mathbf{X}(t - \tau_{out})$.

For the time-series prediction task, the desired reservoir output $\mathbf{v}_d(t)$ is just $\mathbf{u}(t)$. As discussed in the previous section, the input signal is discretized both in time and in precision so that the true state of the input signal is similar to the signal in Fig. 4.3b. Thus, $\mathbf{v}(t)$ must be discretized in the same fashion. Note that, because the reservoir state $\mathbf{X}(t)$ is Boolean valued, a linear transformation $\mathbf{W}_{out}$ of the reservoir state is equivalent to a partial sum of the weights $\mathbf{W}_{out}$, where W_{out}^i is included in the sum only if $X_i(t) = 1$.

I find that the inclusion of a direct connection from input to output greatly improve prediction performance. Though this involves a multiplication of 8-bit numbers, it only slightly increases τ_{out} because this multiplication can be done in parallel with the calculation of the addition of the Boolean reservoir state.

With the above considerations in mind, the output layer is constructed as follows: on the rising edge of a global clock with period t_{global} , the reservoir state is passed to a register in the output layer. The output layer calculates $\mathbf{W}_{out}\mathbf{X}$ with synchronous logic and in one clock cycle, where the weights $\mathbf{W}_{out}$ are stored in on-board memory blocks. The calculated output $\mathbf{v}(t)$ is passed to a register on the edge of the global clock. If $t > 0$, *i.e.* if the training period has ended, the input layer passes $\mathbf{v}(t)$ to the reservoir rather than the next stored value of $\mathbf{u}(t)$.

For $\mathbf{v}(t)$ to have the same discretized form as $\mathbf{u}(t)$, I must have the global clock period t_{global} be equal to the input period t_{sample} , which means the fastest my reservoir computer can produce predictions is once every $\max\{\tau_{out}, t_{sample}\}$. While t_{sample} is independent of the size of the reservoir and precision of the input, τ_{out} in general depends on both. I find that $\tau_{out} = 6.25$ ns is the limiting period for a reservoir of 100 nodes, an 8-bit input precision, and the Arria 10 FPGA considered here. The reservoir computer is therefore able to make predictions at a rate of 160 MHz, which is currently the fastest prediction rate of any real-time RC to the best of my knowledge.

4.6 Results Analysis

I apply the complete reservoir computer—the autonomous reservoir and synchronous input and output layers—to the task of predicting a chaotic time-series. To quantify the performance of my prediction algorithm, I compute the normalized root-mean-square error (NRMSE) over one Lyapunov time $T_{Lyapunov}$, where $T_{Lyapunov}$ is the inverse of the largest Lyapunov exponent. The $NRMSE_T$ is therefore defined as

$$NRMSE_T = \sqrt{\frac{\sum_{t=0}^{T_{Lyapunov}} (u(t) - v(t))^2}{T_{Lyapunov} \sigma^2}}, \quad (4.10)$$

where σ^2 is the variance of $u(t)$.

To train the reservoir computer, the reservoir is initially driven with the stored values of $u(t)$ as described in Sec. 4.4 and the reservoir response is recorded. This reservoir response is then transferred to a host PC. The output weights $\mathbf{W}_{out}$ are chosen to minimize

$$\sum_{t=-T_{train}}^0 (u(t) - v(t))^2 + r |\mathbf{W}_{out}|^2, \quad (4.11)$$

where r is the ridge regression parameter and is included in Eq. 4.11 to discourage over-fitting to the training set. The value of r is chosen by leave-one-out cross validation on the training set. I choose a value of T_{train} so that 1,500 values of $u(t)$ are used for training.

4.6.1 Generation of the Mackey-Glass System

The Mackey-Glass system is described by the time-delay differential equation

$$\dot{u}(t) = \beta \frac{u(t - \tau)}{1 + u^n(t - \tau)} - \gamma u(t), \quad (4.12)$$

where β, γ, τ , and n are positive, real constants. The Mackey-Glass system exhibits a range of ordered and chaotic behavior. A commonly chosen set of parameters is $\beta = 0.2, \gamma = 0.1, \tau = 17, n = 10$ for which Eq. 4.12 exhibits chaotic behavior with an estimated largest Lyapunov exponent of 0.0086 ($T = 116$).

Equation 4.12 is integrated using a 4th-order Runge-Kutta method, and the resulting series is normalized by shifting by -1 and passing $u(t)$ through a hyperbolic tangent function as in Jaeger, 2001, resulting in a variance $\sigma^2 = 0.046$. As noted in Sec. 4.5, $u(t)$ must be discretized according to Fig. 4.3b. I find an optimal temporal sampling of $\Delta t = 5$ as in Fig. 4.3a.

FIGURE 4.4: An example of the output of a trained reservoir computer. Autonomous generation starts at $t = 0$. The target signal is the state of the Mackey-Glass system described by Eq. 4.12. The particular hyperparameters are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11 \text{ ns}, 0.5)$.

The reservoirs considered here are constructed from random connection matrices $\mathbf{W}$ and $\mathbf{W}_{in}$. However, I seek to understand the reservoir properties as functions of the hyperparameters that control the distributions of these random matrices. Recall from Sec. 4.3 that these hyperparameters are:

- the largest absolute eigenvalue of $\mathbf{W}$, denoted by ρ ;
- the fixed in-degree of each node, denoted by k ;
- the mean delay between nodes, denoted by $\bar{\tau}$;
- and the number of nodes which receive the input signal, denoted by σ .

Because t_{sample} and, consequently, the global temporal properties of the predicting reservoir are coupled to the network size N , I fix $N = 100$ and consider the effects of varying the four hyperparameters given above.

Obviously, many instances of $\mathbf{W}_{in}$ and $\mathbf{W}$ have the same hyperparameters. I therefore consider the dynamical properties considered in this section as well as prediction performance to be random variables whose mean and variance I wish to investigate. For each set of reservoir parameters, 5 different reservoirs are created and each tested 5 times at the prediction task.

For optimal choice of reservoir parameters $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11 \text{ ns}, 0.5)$, I measure $NRMSE = 0.028 \pm 0.010$ over one Lyapunov time. The predicted and actual signal trajectories for this reservoir are in Fig. 4.4. For comparison to other works, I prepared an ESN as in Jaeger, 2001 with the same network size (100 nodes) and training length (1500 samples) and find a $NRMSE_T = 0.057 \pm 0.007$.

4.6.2 Spectral Radius

The spectral radius ρ controls the scale of the weights $\mathbf{W}$. Though there are many ways to control this scale (such as tuning the bounds of the uniform distribution (Büsing, Schrauwen, and Legenstein, 2010)), ρ is often seen as a useful way to characterize a classical ESN (Caluwaerts *et al.*, 2013; Lukoševičius, 2012). Optimizing this parameter has been critical in many applications of RC, with a spectral radius near 1 being a common starting point. More abstractly, the memory capacity has been demonstrated to be maximized at $\rho = 1.0$ from numerical experiments (Verstraeten *et al.*, 2007) and it has been shown that ESNs do not have the fading memory property for all inputs for $\rho > 1.0$ (Jaeger, 2001).

It is not immediately clear that ρ will be a similarly useful characterization of these Boolean networks, since the activation function (see Eq. 4.2) is discontinuous and includes time-delays—both factors which are typically not assumed to be true in the current literature. Nonetheless, I proceed with this scaling scheme and investigate the decay times and prediction performance properties of the reservoirs as I vary this parameter.

I see from Fig. 4.5 that the performance on the Mackey-Glass prediction task is indeed optimized at $\rho = 1.0$. However, performance is remarkably flat, quite unlike more traditional ESNs. The performance will obviously fail as $\rho \rightarrow 0$ (corresponding to no recurrent connections) and

as $\rho \rightarrow \infty$ (corresponding to no input connections), and it appears that a range of ρ in between yield similar performance.

This flatness in prediction performance is reflected in measures of the dynamics of the reservoir as seen in Fig. 4.5a and 4.5b. Note that the decay time of the reservoir decreases for smaller ρ . This behavior is expected, because, as the network becomes more loosely self-coupled, it is effectively more strongly coupled to the input signal, and thus will more quickly forget previous inputs. More surprising is the flatness beyond $\rho = 1.0$, which mirrors flatness in the performance error in this region of spectral radii.

I propose that this insensitivity to ρ is due to the nature of the activation function in Eq. 4.4. Note that, because of the flat regions of the Heaviside step function and the fact that the Boolean state variables take discrete values, there exists a range of weights that correspond to precisely the same Λ_i for a given node. Thus, the network dynamics are less sensitive to the exact tuning of the recurrent weights than in an ESN.

4.6.3 Connectivity

The second component to characterizing $\mathbf{W}$ is the in-degree k of the nodes, which is the density of non-zero entries in the row vectors of $\mathbf{W}$. Because the Λ_i 's are populated by explicit calculation of the functions in Eq. 4.4 and because larger Λ_i 's require more resources to realize in hardware, it is advantageous to limit k . I therefore ensure that each node has fixed k rather than simply some mean degree that is allowed to vary.

From the study of purely Boolean networks with discrete-time dynamics (*i.e.*, dynamics defined by a map rather than a differential equation), a transition from order to chaos is seen in a number of network motifs at $k = 2$ (Derrida and Pomeau, 1986; Rohlf and Bornholdt, 2002). In fact, Hopfield type nodes are seen to have this critical connectivity in the explicit context of RC (Büsing, Schrauwen, and Legenstein, 2010). The connectivity is a commonly optimized hyperparameter in the context of ESNs as well (Jaeger, 2001; Jaeger, 2002) with the common heuristic that low-connectivity ($1 - 5\%$ of N) promotes a richer reservoir response.

From the above considerations, I study the reservoir dynamics and prediction performance as I vary $k = 1 - 4$. From Fig. 4.6, I see stark contrasts from the picture of RC with a Boolean network in discrete time. First, the reservoirs remain in the ordered phase for $k = 2 - 4$, which clearly demonstrates that the real-valued nature of the underlying dynamical variables in Eq. 4.4 are critically important to the network dynamics.

I see further in Fig. 4.6b that the mean decay time increases with increasing k , *i.e.*, that the network takes longer to forget past inputs when the nodes are more densely connected. This phenomenon is perhaps understood by the increased number of paths in networks with higher k . These paths provide more avenues for information about previous network states to propagate, thus prolonging the decay of the difference in Eq. 4.7. The variance in decay time also significantly increases for increasing k . This may be an indicator of eventual criticality for large enough k .

Given the strong differences in reservoir dynamics between $k = 1, 4$, it is surprising that no significant difference at the prediction task is detected. However, it is useful for the design of efficient reservoirs to observe that very sparsely connected reservoirs suffice for complicated tasks. As noted in Sec. 4.4, nodes with more inputs require more resources to realize in hardware and more processing time to compute the corresponding Λ_i in Eq. 4.4.

4.6.4 Mean Delay

As argued in Sec. 4.4, adding time-delays along the network links increases the characteristic time scale of the network. I distribute delays by randomly choosing, for each network link, a delay time from a uniform distribution from $\bar{\tau}/2 - 3\bar{\tau}/2$. The shape of this distribution is chosen to fix the mean delay time while keeping the minimum delay time above the characteristic time of the nodes themselves.

In Fig. 4.7, I compare the prediction performance vs. $\bar{\tau}$. Note that this parameter is most critical in achieving good prediction performance in the sense that $\bar{\tau}$ being comparable to τ_{node} yields poor performance. However, the performance is flat past a certain minimum $\bar{\tau}$ near 8.5

ns. This point is important to identify, as adding more delay elements than necessary increases the number of FPGA resources needed to realize the network.

4.6.5 Input Density

I finally consider the effect of tuning the proportion of reservoir nodes that are connected to the input signal. This proportion is often assumed to be 1 (Jaeger, 2002), although recent studies have shown a smaller fraction to be useful in certain situations, such as predicting the Lorenz system (Pathak *et al.*, 2018a).

I observe from Fig. 4.8a that an input density of 0.5 performs better than input densities of 0.25, 0.75, and 1.0. I note from Fig. 4.8b that this corresponds to the point of longest decay time. The decreasing decay time with higher input densities 0.75 and 1.0 are consistent with the expectation that reservoirs that are more highly coupled to the input signal will forget previous inputs more quickly.

It is apparent from Fig. 4.8b that the input density is a useful characterization of the RC scheme, impacting the fading memory properties of the reservoir-input system and ultimately improving performance by a factor of 3 when compared to a fully dense input matrix. This result suggests the input density to be a hyperparameter deserving of more attention in general contexts.

4.6.6 Attractor Reconstruction

Prediction algorithms are commonly evaluated on their short-term prediction abilities as I have done so far in this section. The predicted and actual signal trajectories will always diverge in the presence of chaos due to the positivity of at least one Lyapunov exponent. However, it has been seen recently that reservoir computers (Pathak *et al.*, 2017) and other neural network prediction schemes (Qiao *et al.*, 2018) can have similar long-term behavior as the target system. In particular for ESNs, it has been noted that different reservoirs can have similar short-term

prediction capabilities, but very different long-term behavior, with some reservoirs capturing the climate of the Lorenz system and others eventually collapsing onto a non-chaotic attractor (Pathak *et al.*, 2017).

To observe a similar phenomenon in the RC scheme considered here, I allow a trained reservoir to evolve for 100 Lyapunov times (about $15 \mu\text{s}$) beyond the training period. The last half of this period is visualized in time-delay phase-space to see if the climate of the true Mackey-Glass system is replicated.

The results show phenomena consistent with previous observations in ESNs. Figure 4.9a shows the true attractor of Eq. 4.12, which has fractal dimension and is non-periodic. Figure 4.9b shows the attractor of a well-chosen autonomous, Boolean reservoir. Although the attractor is “fuzzy,” the trajectory remains on a Mackey-Glass-like shape well beyond the training period. On the other hand, a reservoir with similar short-term prediction error is shown in Fig. 4.9c. Although this network is able to replicate the short-term dynamics of Eq. 4.12, its attractor is very unlike the true attractor in Fig. 4.9a. This results shows that, even in the presence of noise inherent in physical systems, the autonomous Boolean reservoir can learn the long-term behaviors of a complicated, chaotic system.

4.7 Conclusion and Future Directions

I conclude that an autonomous, time-delay, Boolean network serves as a suitable reservoir for RC. I have demonstrated that such a network can perform the complicated task of predicting the evolution of a chaotic dynamical system with comparable accuracy to software-based RC. I have demonstrated the state-of-the-art speed with which my reservoir computer can perform this calculation, exceeding previous hardware-based solutions to the prediction problem. I have demonstrated that, even after the trained reservoir computer deviates from the target trajectory, the attractor stays close to the true attractor of the target system.

This work demonstrates that fast, real-time computation with autonomous dynamical systems is possible with readily-available electronic devices. This technique may find applications in design of systems that require estimation of the future state of a system that evolves on a nanosecond to microsecond time scale, such as the evolution of cracks through crystalline structures, the motion of molecular proteins, or the transmission of symbols through a noisy optical line.

Further, this work motivates increased attention to the development of ABN reservoir computers and suggests a number of future research directions. One aspect not explored in this work is placement and routing constraints, where the designer specifies the physical position of physical logic elements and the connections between them. In this work, these choices were left up to the Quartus compiler, which may not be optimal. Another aspect is the potentially excessive use of delay elements necessary to achieve good performance. These elements take up the majority of FPGA resources, so reducing their need is desirable. One way to reduce their use would be to speed up the rate of input data, possibly by taking advantage of the dedicated transceiver/receiver logic that is common on FPGA boards. Finally, a numerical model that captures the essential reservoir features is desired.

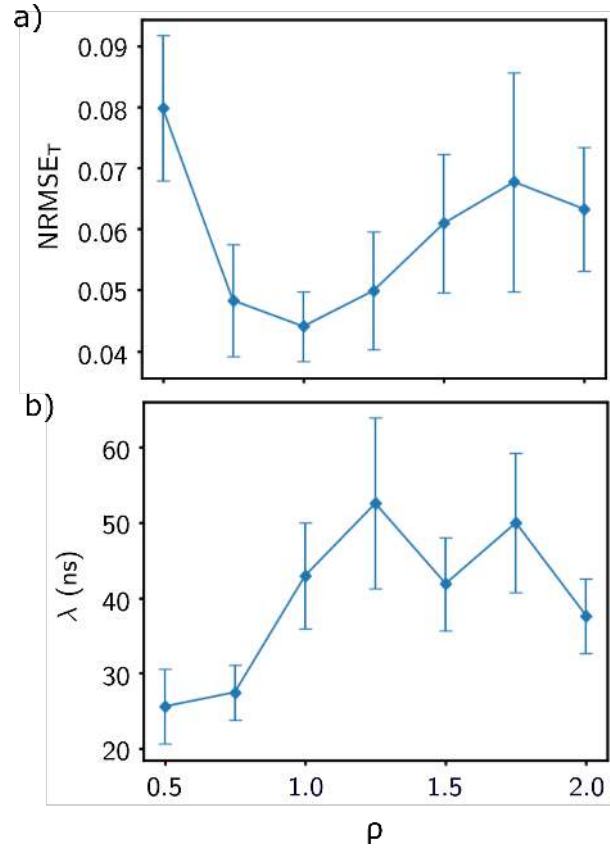


FIGURE 4.5: Prediction performance and fading memory of reservoirs with $(k, \bar{\tau}, \sigma) = (2, 11 \text{ ns}, 0.75)$ and varying ρ . (a) Somewhat consistent with observations in echo-state networks, ρ near 1.0 appears to be a good choice. However, a much wider range of ρ suffice as well. (b) As ρ becomes small and the reservoir becomes more strongly coupled to the input, the reservoir more quickly forgets previous inputs. The decay time levels out above $\rho = 1.0$. Note that λ is everywhere the same order of magnitude as $\bar{\tau}$.

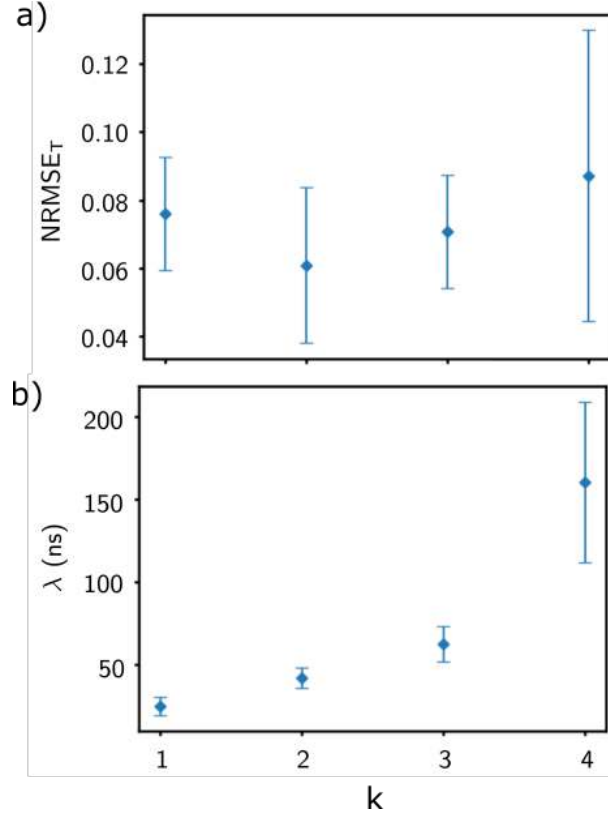


FIGURE 4.6: Prediction performance and fading memory of reservoirs with $(\rho, \bar{\tau}, \sigma) = (1.5, 11 \text{ ns}, 0.75)$ and varying k . (a) I see effectively no difference over this range, contrary to intuitions from studies of Boolean networks in discrete time. (b) For $k = 1$, λ is approximately equal to $\bar{\tau}$. However, as I increase k to 4, both the mean and variance of λ approaches almost an order of magnitude larger than $\bar{\tau}$.

FIGURE 4.7: Prediction performance of reservoirs with $(\rho, k, \sigma) = (1.5, 2, 0.75)$ and varying $\bar{\tau}$. The NRMSE decreases until approximately $\bar{\tau} = 9.5$, after which point it remains approximately constant.

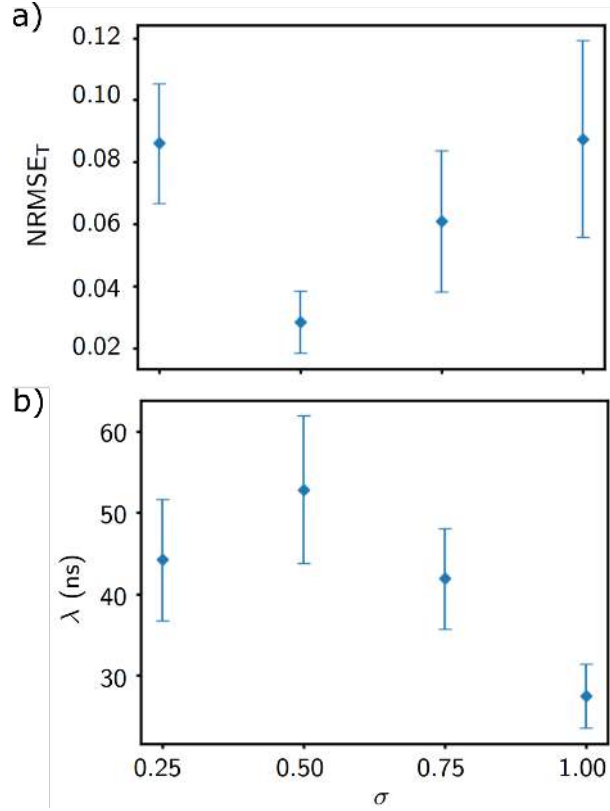


FIGURE 4.8: Prediction performance and fading memory of reservoirs with $(\rho, k, \bar{\tau}) = (1.5, 2, 11 \text{ ns}, 0.75)$ and varying ρ . (a) Choosing $\sigma = 0.5$ improves prediction performance by a factor of 3 over the usual choice of $\sigma = 1.0$ (b) With larger σ , the reservoir is more strongly coupled to the input signal. Consequently, λ decreases, signifying that the reservoir is more quickly forgetting previous inputs.

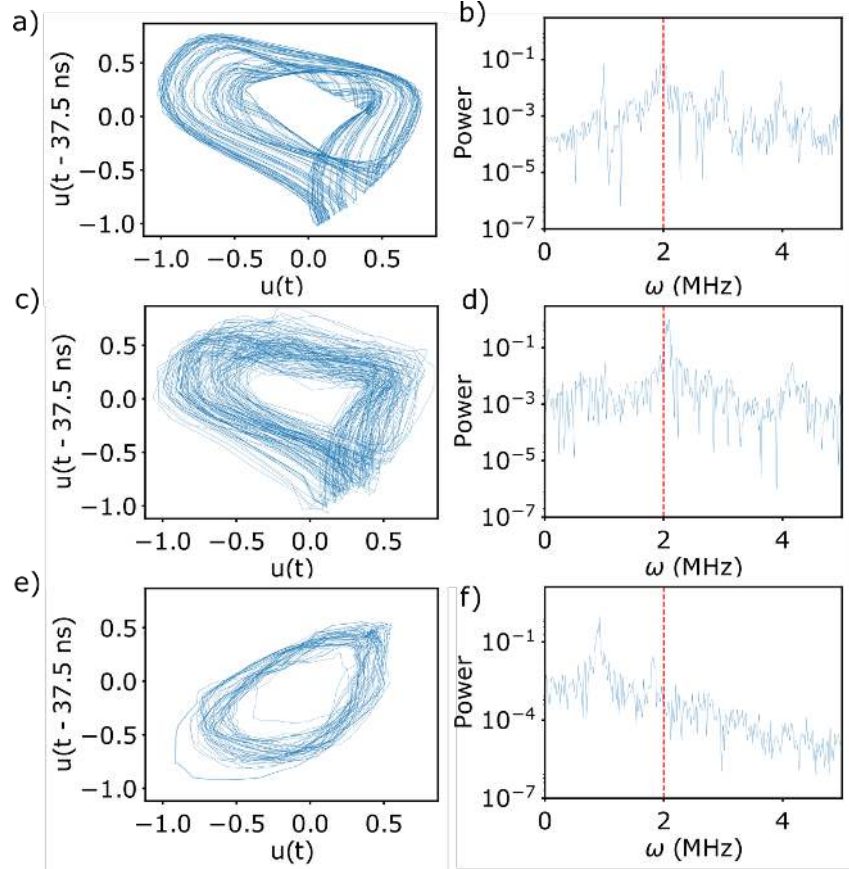


FIGURE 4.9: Phase-space representations and power spectra of the attractors of Eq. 4.12 and trained reservoirs. (a) The true attractor and (b) normalized power spectrum of the Mackey-Glass system, as presented to the reservoir. (c) The attractor and (d) normalized power spectrum for a reservoir whose long-term behavior is similar to the true Mackey-Glass system. Although “fuzzy,” the attractor remains near the true attractor. The power spectrum shows a peak 0.10 MHz away from the true peak. The hyperparameters for this reservoir are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 2, 11 \text{ ns}, 0.75)$. (e) The attractor and (f) normalized power spectrum of a reservoir whose long-term behavior is different than the true Mackey-Glass system. The dominate frequency of the true system is highly suppressed, while a lower-frequency mode is amplified. The hyperparameters for this reservoir are $(\rho, k, \bar{\tau}, \sigma) = (1.5, 4, 11 \text{ ns}, 0.75)$. The dashed, red line in the power spectrum plots indicates the peak of the spectrum in the true Mackey-Glass system.

Chapter 5

Dimensionality Reduction in Reservoir Computers

Reservoir computing (RC) is a machine learning framework for processing time-dependent data that is founded on random, recurrent neural networks. Due to this random nature, trained networks are definitionally sub-optimum for any specific task. This observation has led to a search for pre- and post-training algorithms to optimize the reservoir with the goal of identifying minimum complexity reservoirs for a given task and error tolerance. In this chapter, I develop such an algorithm that can be applied to a variety of RC algorithms, including the popular echo state network (ESN). I demonstrate its efficacy by studying benchmark chaotic time-series prediction tasks.

The rest of this chapter is outlined as follows: First, I overview previous attempts to maximize these properties with a wide range of pre-training algorithms. I then demonstrate with a series of numerical examples that random ESNs have poor separation and approximation due to a high degree of collinearity in the network response. Next, I exploit this collinearity to derive a dimension-reduction algorithm based on a singular value decomposition (SVD), resulting in what I call a compressed ESN (CESN). Then, I show that the SVD-derived CESNs generalize, in the sense that they can be re-used for similar tasks. Finally, I examine the linear stability of these CESNs to derive high-performance ESNs capable of predicting chaotic

time-series with the accuracy of standard ESNs more than 20 times their size.

5.1 Previous Pre-Training Algorithms

Several methods exist for optimizing reservoir computers by improving, either explicitly or heuristically, the separation and approximation properties of reservoirs, sometimes referred to as *pre-training algorithms*. These methods may be unsupervised (without regard to desired output), supervised (with regard to desired output), local, or global.

Early approaches for reservoir optimization relied on biological motivation. Several of the first attempts were surprisingly unsuccessful (Jaeger, 2005). Some are successful when applied to real-world inputs but not random inputs (Norton and Ventura, 2006). One related approach that has shown greater success is tuning the distribution of spiking reservoirs towards an exponential distribution, as seen in biological neurons, through intrinsic plasticity learning rules (Triesch, 2005). More general learning rules for generating exponential activation distributions were later derived for ESNs and related reservoirs (Schrauwen *et al.*, 2008b).

Another general approach, which has been previously attempted in some forms (Dutoit, Van Brussel, and Nuttin, 2007) is to take an initial, large reservoir and devise a supervised algorithm for *pruning* or reducing the dimension of the reservoir while maintaining all of the important dynamics for a given task. This approach is motivated by the observation, seen frequently in a variety of contexts, that an increased reservoir size generally leads to greater separation and approximation measures and performance generally. The goal of pruning is then to remove nodes that don't adequately contribute to these measures, resulting in a more efficient reservoir.

In this chapter, I propose a compression algorithm related to reservoir pruning. It is motivated by the observation that randomly created reservoirs exhibit a surprisingly high degree of linear redundancy, or *collinearity*. This redundancy can be quantified by standard statistical techniques. A particular method for defining a new variable with minimum redundancy,

known as *singular value decomposition* (SVD) is particularly effective in this case. The SVD can be exploited to find ESN-like equations for the reduced networks as I show in the proceeding sections.

5.2 Collinearity in Echo State Networks

In this section, I motivate my dimension reduction algorithm by illustrating the degree to which a standard ESN exhibits collinearity when coupled to a complex system. As an example, I consider the input system to be described by

$$\dot{y}(t) = -\gamma y(t) + \beta \frac{y(t-\tau)}{1 - y^n(t-\tau)} \quad (5.1)$$

$$u = \tanh(y). \quad (5.2)$$

Equation 5.1 defines the Mackey-Glass time-delay equation, while the observation function is a commonly used "squashing" function that facilitates comparison to prior works. I consider the parameter set $\gamma = 0.1$, $\beta = 0.2$, $n = 10$, and $\tau = 17$, for which Eq. 5.1 exhibits chaotic dynamics (Mackey and Glass, 1977). The attractor of the system for positive initial values is depicted in Fig. 5.1.

To perform tasks such as forecasting, calculating Lyapunov exponents, or detecting anomalies, an ESN can be used by coupling to the system of interest and training an output layer $\mathbf{W}_{out}$ accordingly. Recall that an ESN is defined by the differential equations

$$\begin{aligned} c\dot{\mathbf{x}} &= -\mathbf{x} + \tanh(\mathbf{W}\mathbf{x} + \mathbf{W}_{in}\mathbf{u} + \mathbf{b}) \\ \mathbf{y} &= \mathbf{W}_{out}\mathbf{x} \end{aligned} \quad (5.3)$$

where $\mathbf{W}$, $\mathbf{W}_{in}$, and $\mathbf{b}$ are random matrices and c is a time constant.

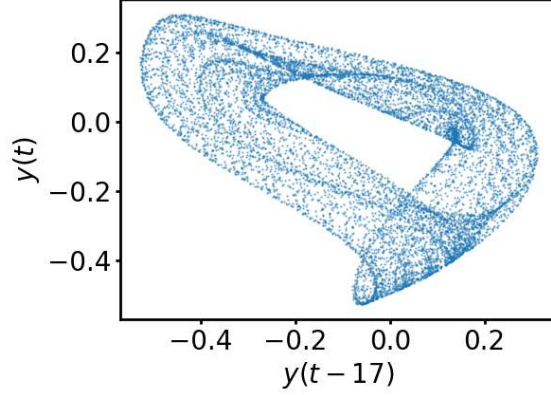


FIGURE 5.1: The attractor of the Mackey-Glass system in the chaotic regime. It is a benchmark system for prediction of chaotic time series.

Consider an ESN driven by the observed variable in Eq. 5.2 and a particular node x_i , prior to any training algorithm. I define $\mathbf{x}_{-i} = \{x_j : j \neq i\}$ to be the set of all nodes in the reservoir that aren't x_i . If there exists some time-independent vector $\mathbf{v}$ such that $x_i = \mathbf{v}^T \mathbf{x}_{-i}$, then the inclusion of x_i in the output layer does not contribute to the approximation property (see Sec. 2.5.3). This is because any $\mathbf{W}_{out}$ can exclude x_i and produce the exact same output. Further, it is also clear that x_i does not contribute meaningfully to the separation property of the reservoir (see Sec. 2.5.2), because any similarity in the response of x_i to similar inputs is already captured by $\mathbf{x}_{-i}$, and conversely for dissimilar inputs. The inclusion of x_i may even be a misleading indicator of separation, since it does not contain any additional information about the reservoir response, but would be included in the determination of the output layer.

Despite the nonlinear dynamics in the ESN and Mackey-Glass equations, a typical node from a typical ESN realization *does* depend linearly on the rest of the nodes to a high degree, in the sense described above. To see this, I solve Eq. 5.1-5.2 with a 100 node reservoir generated in the usual way (see Ch. 2), with the choice of hyperparameters given in Table 5.1. The reservoir state is collected in a matrix $\mathbf{X}$ of size $14,000 \times 100$, where columns of $\mathbf{X}$ correspond to the sampled reservoir state at $h = 0.1$ intervals. Without loss of generality, the 0^{th} node is selected, and a $\mathbf{v}$ is chosen from $\mathbf{X}$ by means of a pseudoinverse calculation.

c	k	ρ	σ	b_{max}	b_{mean}
3	1	0.95	1	1	0

TABLE 5.1: The hyperparameters used for the compression experiments, unless otherwise noted. See Ch. 2 for an explanation of these parameters and the reservoir computing algorithm.

To examine the degree to which x_0 linearly depends on $\mathbf{x}_{-0}$, I plot $x_0(t)$, $\mathbf{v}^T \mathbf{x}_{-0}(t)$ and their difference in Fig. 5.2. To be sure $\mathbf{v}$ has truly revealed a functional dependence, I continue the plot from $t = 1400$ to $t = 1900$ to see if the relationship generalizes. In Fig. 5.2a, I see no visual difference in these signals, even after $t = 1400$. The calculated difference doesn't exceed 10^{-6} , even after $\mathbf{v}^T$ was chosen, showing that the linear redundancy exists to a high degree and generalizes well.

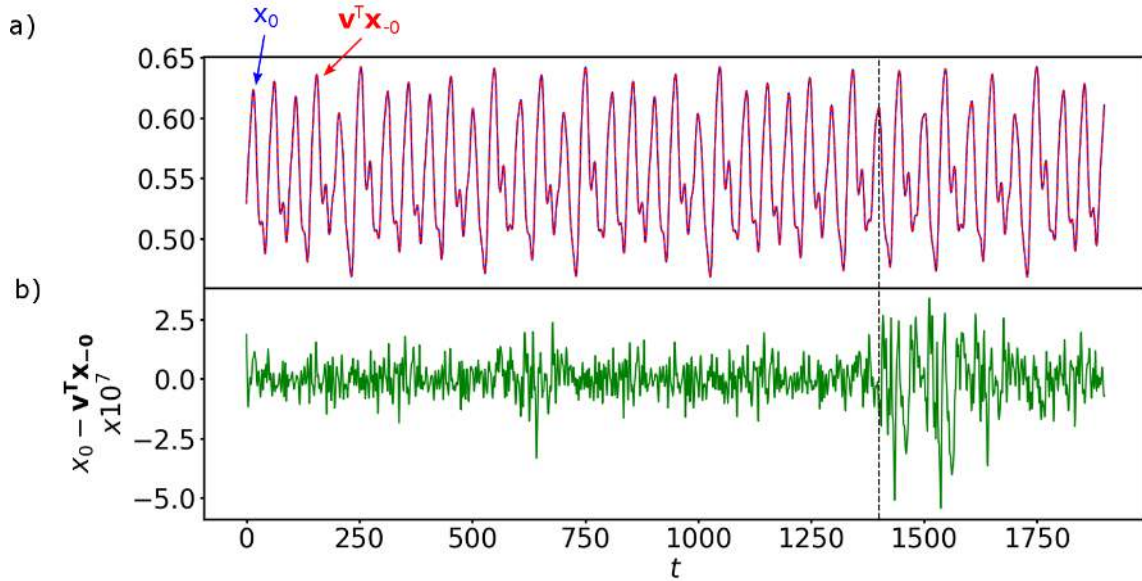


FIGURE 5.2: The redundancy of a node x_0 in a typical ESN driven by the Mackey-Glass system. a) Based on observations of x_0 and $\mathbf{x}_{-0}$ from $t = 0$ to $t = 1400$, a linear transformation $\mathbf{v}$ is chosen based on the pseudoinverse of the collected data. The curves of x_0 and $\mathbf{v}^T \mathbf{x}_{-0}$ appear identical, even after $t = 1400$. b) The difference between the two curves in Fig. 5.2a differ by only approximately 10^{-7} , even at times not used to identify $\mathbf{v}$.

5.2.1 Dynamical Equivalence

This situation of redundancy maybe even worse than x_0 being unnecessary for the output. Note that the dynamics of $\mathbf{x}_{-0}$ from the Eq. 5.3 also depend on a linear combination of $\mathbf{x}$. If x_0 is so accurately approximated by $\mathbf{v}^T \mathbf{x}_{-0}$, then the former can be replaced by the latter in the differential equation without substantially changing the reservoir response. Upon making this replacement, the *dynamics* of $\mathbf{x}_{-0}$ don't depend on x_0 either, and the network can be replaced by an equivalent network with 99 rather than 100 nodes. Making this replacement reduces the dimension of the ESN while not reducing the approximation or separation properties of the reservoir.

To investigate the effect of collinearity on network dynamics, I make the replacement discussed above in Eq. 5.3. This leads to the reduced network equations

$$c\dot{\tilde{\mathbf{x}}} = -\tilde{\mathbf{x}} + \tanh(\tilde{\mathbf{W}}\tilde{\mathbf{x}} + \tilde{\mathbf{W}}_{in}\mathbf{u} + \tilde{\mathbf{b}}), \quad (5.4)$$

where $\tilde{\mathbf{W}}_{in}$ and $\tilde{\mathbf{b}}$ are truncated to not include the 0th row and $\tilde{\mathbf{W}} = \mathbf{W}(\mathbf{I}_N + (\mathbf{v}, \mathbf{0}, \dots, \mathbf{0}))$, also truncated to not include the 0th row or column.

I propagate Eq. 5.4 to collect an observation matrix $\tilde{\mathbf{X}}$ of the reduced 99-node network. I compare this matrix to that of the full network, starting from the same initial conditions. The difference $\tilde{\mathbf{x}}(t) - \mathbf{x}(t)$ is displayed as a function of time in Fig. 5.3 and shows that this replacement has little effect on the reservoir response to the Mackey-Glass system.

The fact that the difference in Fig. 5.3 does not grow in time is dependent on the generalized synchronization property of the network. The error term $x_0 - \mathbf{v}^T \mathbf{x}_{-0}$ can be viewed as a small noise term that is suppressed by the fading memory in the network.

5.2.2 Autonomous Reduced Network

In the previous subsection, I demonstrated for a typical ESN driven by a Mackey-Glass system that a node can be replaced by a linear combination of the other nodes in the network, thereby

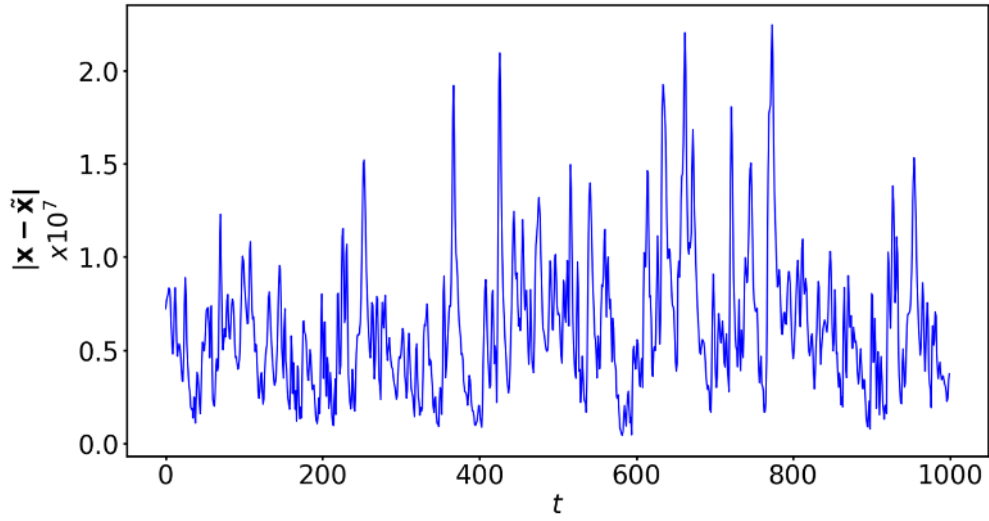


FIGURE 5.3: The difference in the dynamics of $\mathbf{x}_{-0}$ when x_0 is replaced by a linear approximation from the other nodes. The median difference is around 10^{-7} , and the difference does not exceed 2.5×10^{-7} . Note that this is the total vector difference $\mathbf{x} - \tilde{\mathbf{x}}$, so the difference of a typical node is on the order of 10^{-9} .

reducing the network dimension without impacting the separation or approximation properties. However, RC is often applied to autonomous use-cases where the output is fed into the input after training, and these cases must be considered as well.

To confirm that x_0 is truly not necessary in the example discussed in Sec. 5.2, I consider the relative performances of a full and reduced reservoir on a typical time-series prediction task. I compare a 100 node initial reservoir and a 99 node reduced reservoir, where a randomly chosen node is completely replaced by a linear combination of the other nodes. The compressed reservoir is lower dimensional and should perform strictly worse according to the conventional RC wisdom that a high-dimension reservoir is necessary for computational purposes.

I train both reservoirs for the prediction task of the Mackey-Glass system and allow them to evolve autonomously beyond $t = 0$. Some typical trajectories $\tilde{y}$ and y are shown in Fig. 5.4 along with the true reference signal y_d . I observe from Fig. 5.4a that all three signals stay close within several Lyapunov times, and in fact the two predictive models stay close to each other *after* they begin to diverge from the reference signal, as can be seen from Fig. 5.4b. If we

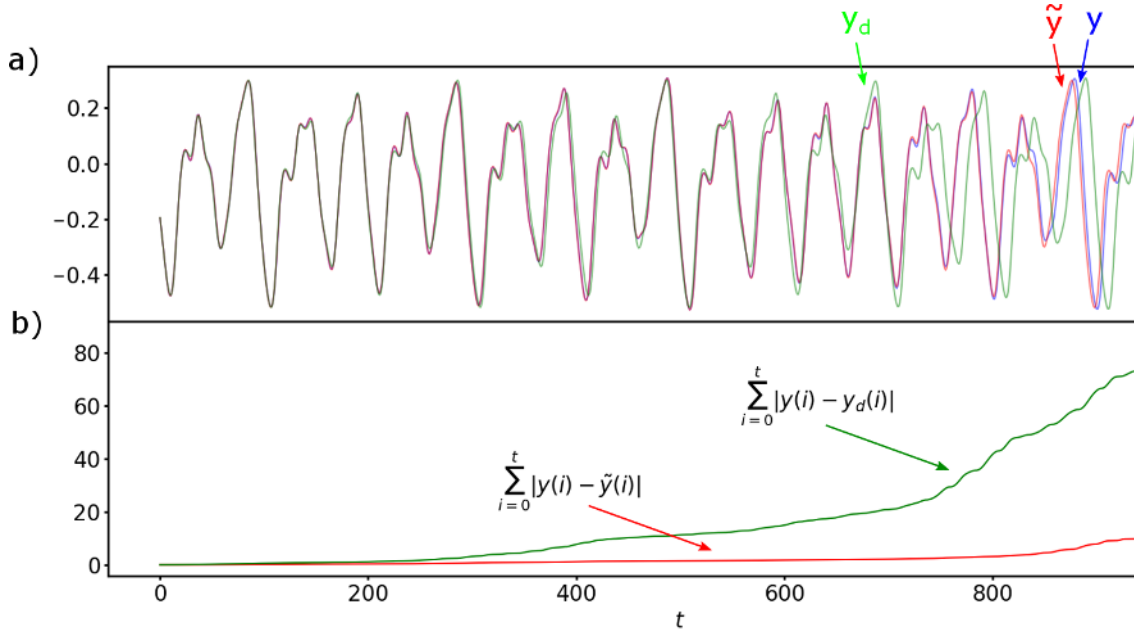


FIGURE 5.4: Comparing the autonomous evolution of a 100 node trained reservoir, a 99 node reservoir with one linear replacement, and the true Mackey-Glass system. a) Traces of the autonomous systems. Calculating the error after 1 Lyapunov time, the errors for the full and reduced system agree within 0.1%. b) Difference between the full reservoir and the reduced reservoir vs the full reservoir and the true system. The full reservoir eventually diverges from the true system, as must happen in the presence of chaos. Similarly, the full reservoir diverges with the reduced reservoir, but only after both systems have already lost track of the true system.

measure the prediction performance as the NRMSE over one Lyapunov time, both reservoirs perform equally well within 0.1%. Additionally, the long-term behavior of the reduced reservoir appears unaffected, as can be seen by examining the attractors of the trained reservoirs in Fig. 5.5.

5.3 SVD Compression Algorithm

The results of the previous section reveal that a randomly chosen node in a randomly instantiated ESN can be removed with a simple supervised algorithm. As long as the original network is in generalized synchronization with the input system, the reduced network has the same

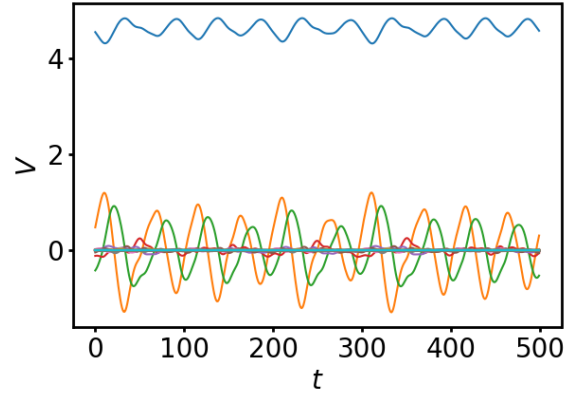


FIGURE 5.5: The SVD of a trace of observations of a 100 node network driven by the Mackey-Glass system. The node magnitudes indicate how much they contribute to a linear reconstruction of the full network. Despite the apparently rich dynamics in the 100 node network, only the first hand-full of reduced nodes are visible.

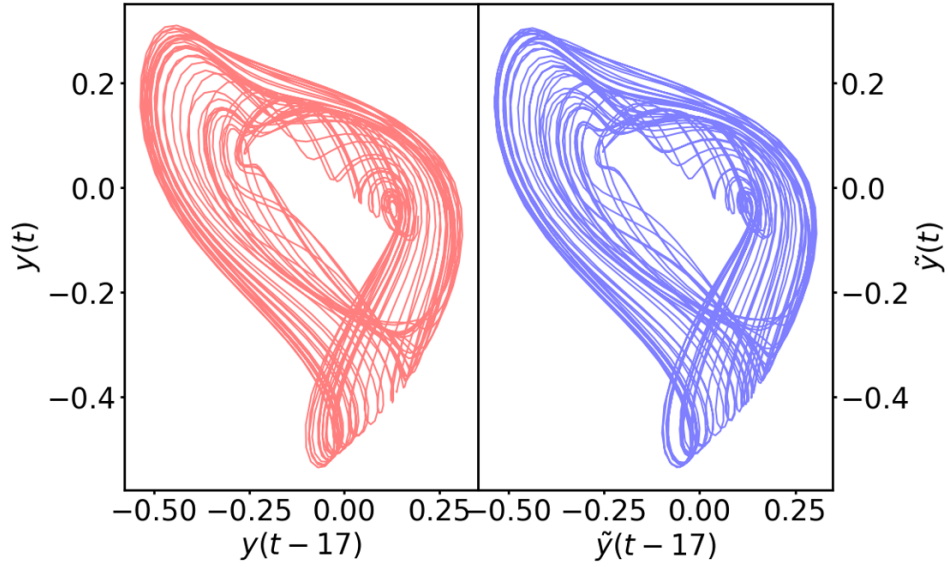


FIGURE 5.6: A comparison of the attractors of the full and reduced reservoirs.

dynamical response to the input and be trained to form an equally performing model for the input.

Of course, one can iterate this simple replacement algorithm to remove more nodes and further reduce the network dimension. This approach, however, runs into at least two issues:

- replacing nodes one-by-one is computationally intensive, particularly if selecting which nodes to replace in any non-arbitrary way, and
- after making too many replacements, the pruned ESN response suddenly deviates significantly from the original ESN response.

A method for systematically selecting a low-dimensional network with equivalent dynamics is desired, particularly one that requires only a single observation matrix $\mathbf{X}$. Fortunately, there exists a wide breadth of techniques for low-dimensional representations of high-dimensional data (see, *e.g.*, Engel, Hüttenberger, and Hamann, 2012). The observations in this section suggest that a linear representation will suffice, *i.e.*,

$$\mathbf{x} = \mathbf{A}\tilde{\mathbf{x}}, \tag{5.5}$$

where $\tilde{\mathbf{x}}$ is the state of a d -dimensional, reduced network, and $\mathbf{A}$ is a $d \times N$ matrix that is determined in some way from $\mathbf{X}$. In the proceeding sections, I describe one effective method for choosing $\mathbf{A}$ and find the resulting dynamics of $\tilde{\mathbf{x}}$.

5.3.1 SVD

As noted above, a variety of statistical techniques exist for understanding low-dimensional representations of temporal data. One such technique is singular value decomposition (SVD). Generally, SVD is a representation of an $m \times n$ matrix $\mathbf{M}$ as

$$\mathbf{M} = \mathbf{U}\mathbf{S}\mathbf{V} \tag{5.6}$$

where $\mathbf{U}$ is a $m \times m$ unitary matrix, $\mathbf{S}$ is a diagonal $m \times n$ matrix whose entries are the singular values of $\mathbf{M}$, and $\mathbf{V}$ is a $n \times n$ unitary matrix. See, *e.g.*, Golub and Reinsch, 1971 for a more complete discussion of SVD, singular values, and how the matrices in Eq. 5.6 can be computed.

The usefulness of the SVD representation is that, when basis vectors are chosen such that the singular values in $\mathbf{S}$ are listed in descending order, the first d rows of $\mathbf{SV}$ are the best d -dimensional representation of $\mathbf{M}$ with respect to the L_2 norm. In Fig. 5.5, I plot $\mathbf{SV}$, where $\mathbf{M}$ is a matrix of $m = 5,000$ samples of each of the $n = 100$ nodes after being driven by the Mackey-Glass system. Because $\mathbf{U}$ is unitary, the magnitude of each trace represents its importance to the reconstruction of the full $\mathbf{M}$. Observe that only a small number of curves (blue, orange, green, red) appear deviate from 0 on this scale. Thus, from Fig. 5.5, it appears that as few as 4 dimensions are necessary to capture all of the dynamics in the 100 node reservoir.

Given the SVD, there exists a straight-forward procedure to produce a low-dimensional reservoir whose dynamics approximate the full reservoir. Note that, because $\mathbf{U}$ is unitary, its inverse is guaranteed to exist. I define a new variable $\tilde{\mathbf{x}}(t)$ by

$$\tilde{\mathbf{x}}(t) = \mathbf{U}^{-1}\mathbf{x}(t). \quad (5.7)$$

The t/h -th column of $\mathbf{SV}$ is exactly $\tilde{\mathbf{x}}(t)$ for $0 \leq t \leq 5,000$, $\tilde{\mathbf{x}}(t)$, but if the SVD generalizes well as suggested by the previous sections, then relationship holds for all t .

To calculate the evolution of $\tilde{\mathbf{x}}$ without Eq. 5.3, I solve for its dynamics by substituting Eq. 5.5 into Eq. 5.3, yielding

$$c\dot{\tilde{\mathbf{x}}} = -\tilde{\mathbf{x}} + \mathbf{U}^{-1}\tanh(\tilde{\mathbf{W}}\tilde{\mathbf{x}} + \mathbf{W}_{in}\mathbf{u} + \mathbf{b}), \quad (5.8)$$

where $\tilde{\mathbf{W}} = \mathbf{W}\mathbf{U}$. Equation 5.8 is an ESN-like equation that involves an additional linear transformation after the non-linear $\tanh$ has been applied.

Recall that the rows of $\mathbf{SV}$ (and therefore also the nodes of $\tilde{\mathbf{x}}$) are ordered such that the first are more important to the reconstruction of $\mathbf{x}$ than the last. In fact, the typical fluctuations of

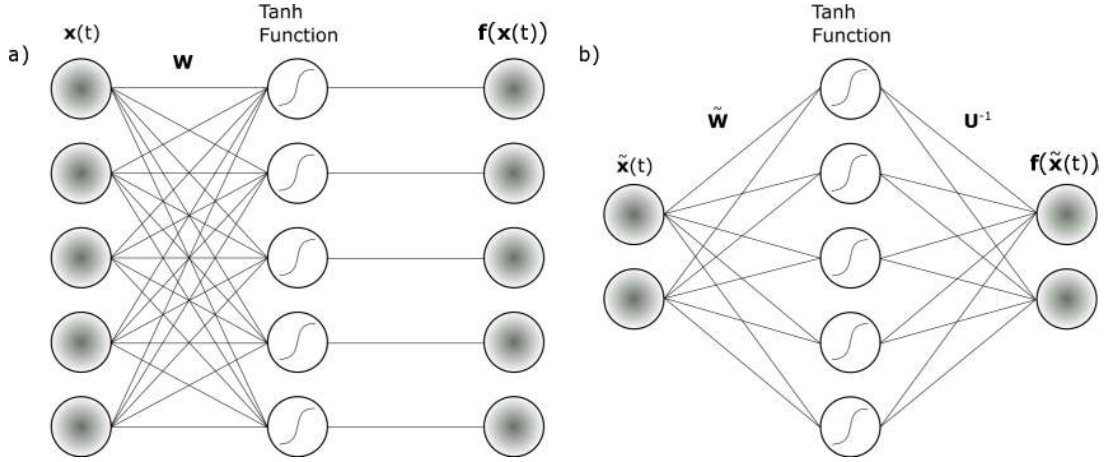


FIGURE 5.7: A schematic comparison of an ESN to an CESN. a) The connections and nonlinear operations required to compute $\mathbf{x}(t+1)$ for a 5-dimensional ESN. The majority of the operations come from a 5×5 matrix multiplication and 5 applications of the tanh function. b) The connections and nonlinear operations required to compute $\tilde{\mathbf{x}}(t+1)$ for a 2-dimensional CESN that was derived from a 5-dimensional ESN. The function $\mathbf{f}$ defines the derivative of $\mathbf{x}$ required to calculate a successive value of $\mathbf{x}$.

$\tilde{\mathbf{x}}_d$ become quite small for $d > 4$, as can be seen from Fig. 5.5. I therefore consider a low-dimensional approximation of Eq. 5.8 by truncating $\tilde{\mathbf{x}}$ to the first d nodes, as well as truncating the first d rows in the appropriate matrices. Defining this truncated network state as $\tilde{\mathbf{x}}_d$ and the truncated matrices similarly, the reduced network has dynamics defined by

$$c\dot{\tilde{\mathbf{x}}}_d = -\tilde{\mathbf{x}}_d + \mathbf{U}_d^{-1} \tanh(\tilde{\mathbf{W}}_d \tilde{\mathbf{x}}_d + \mathbf{W}_{in,d} \mathbf{u} + \mathbf{b}_d), \quad (5.9)$$

which defines our final, d -dimensional reduced network equations.

5.3.2 Compressed Echo State Networks

I refer to a network with dynamics defined by Eq. 5.9 as a *compressed echo state network*, or CESN. The "compressed" quantifier is due to the presence of the $\mathbf{U}_d^{-1}$ matrix applied after the tanh function. A schematic comparison of an CESN to an ESN is in Fig. 5.7.

Before I proceed to study Eq. 5.9 in the context of reservoir computing, I note a few important observations. First, unlike a traditional ESN, the adjacency matrix $\tilde{\mathbf{W}}_d$ is not randomly instantiated, but determined from the observed state matrix $\mathbf{X}$ and is therefore a stochastic function of both $\mathbf{W}$ and $\mathbf{u}$. I note that the derivation of the CESN is a *supervised* pre-training algorithm, because its determination depends on the input signal (although, see Sec. 5.4).

Second, the rectangular nature of the truncated matrices as well as the expansion matrix $\mathbf{U}_d^{-1}$ means that a d -dimensional CESN requires more computations per time-step to simulate than an ESN of the same size. This, at first, appears to negate the utility of working with an SVD-derived CESN, since the number of operations required per update cycle is of principle importance in dedicated-hardware and other RC use-cases. However, as long as $2d < N$, the CESN requires fewer operations, despite having apparently equal (or greater) computational power than the N -dimensional ESN, as I show in Sec. 5.4.3.

To see the $2d < N$ requirement, consider a full ESN described by Eq. 5.3 with input dimension m and reservoir dimension N . Simultaneously computing $\dot{\mathbf{x}}(t)$ and $\mathbf{y}(t)$ requires a number of additions, multiplications, and applications of the tanh function. More specifically, simple counting of these operations reveals that $N(N + m + 1)$ multiplications, $N(\log_2[N + m + 1] + 1)$ additions, and N tanh operations are required. On the other hand, consider an CESN described by Eq. 5.9 with input dimension m , starting dimension N , and reduced dimension d . Then, again by simple counting from the operations in Eq. 5.9, there are now $N(2d + m + 1)$ multiplications, $N(\log_2[2d + m + 1] + 1)$ additions, and N tanh operations.

Of the operations involved, tanh is typically the most computationally intensive (especially so if floating-point resolution is required). Fortunately, computing the evolution of the reduced reservoir requires no additional tanh operations. It does, however, require $N(2d - N)$ more multiplications. The reduced reservoir therefore requires fewer operations than the full reservoir only if $d < N/2$, which is seen to be the case for reservoirs of a typical use size.

Finally, I note that, although the CESN was derived by considering the SVD of the response of an ESN, the analysis in this section can be easily extended to any reservoir computer of the

form

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}) \\ \mathbf{y} &= \mathbf{g}(\mathbf{x}),\end{aligned}\tag{5.10}$$

as long as $\mathbf{f}$ and $\mathbf{g}$ are known explicitly. Although I focus on the ESN in this and the proceeding sections, similar results can be seen for other reservoir computers, such as a polynomial reservoir (Carroll, 2018) with linear readout, and a linear reservoir with cubic readout.

5.3.3 Performance Analysis

I now demonstrate the extent to which Eq. 5.9 is an approximation of the full ESN equations, subject to the Mackey-Glass drive. I do this by evolving the reduced equations for some period of time and comparing $\mathbf{U}\mathbf{x}_d(t)$ to $\mathbf{x}(t)$, where $\mathbf{x}$ is obtained by evolving the full ESN equations. For a typical $N = 200$ node reservoir, this is done for each $0 \leq d \leq N$, and the resulting difference is plotted in red in Fig. 5.8. Remarkably, one can maintain less than 1% difference in the reduced equations down to $d = 25$, again demonstrating the high degree of collinearity that was present in the full ESN.

The comparison described above is useful for understanding the differences in the non-autonomous dynamics of the reduced and full reservoirs, but I am often interested in using reservoirs for feedback for prediction tasks. I perform a similar comparison for each d by training CESN and autonomously predicting the Mackey-Glass system. I compare the performances of the reduced reservoirs in the usual way by computing the NRMSE over one Lyapunov time. The result for the same reservoir used to compare non-autonomous differences is plotted in blue in Fig. 5.8.

As seen in Fig. 5.8, the difference in the non-autonomous states of the reduced and full

reservoir is smooth and monotonic as a function of reduced dimension. The prediction performance, however, has a minimum at $d = 52$. Despite measurable variations in the response of the reduced network, the performance curve is essentially flat ($< 2\%$ variation) until $d = 90$. This suggests that the information lost by reducing the reservoir state never plays any independent role in the prediction task.

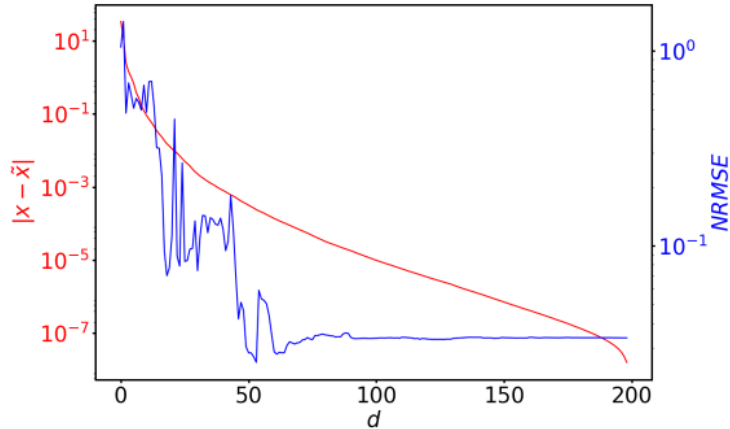


FIGURE 5.8: Comparing the full 200 node reservoir and various d node reduced networks. During the listening phase, the mean difference between the full network trace and the reconstruction from the reduced trace are calculated and plotted in red, showing a smooth increase as the size is decreased, closely following the ordered singular values of $\mathbf{U}$. I also compare the predictions of the Mackey-Glass system of the autonomous systems, plotted in blue, as measured by the NRMSE after one Lyapunov time. Remarkably, performance is flat down to approximately $d = 100$, even though there are measurable differences in the reservoir traces.

The blue curve in Fig. 5.8 is very different from what is typically observed when varying ESN size. Although scaling laws vary by task, normalized performance measures typically follow a power law, often with exponent $-1/2$ for prediction of a chaotic time series. This suggests that CESNs formed with a large N and small d perform significantly better than classically trained reservoirs of the same size, since the small pruned reservoir performs as well as the large reservoir from which it was pruned. To emphasize this performance difference, I consider three ensembles of reservoirs consisting of $N = 200$ node ESNs, $N = 5,000$ node ESNs, and $N = 5,000, d = 200$ node CESNs. The mean performance of each ensemble is computed

by randomly generating 20 each and determining the NRMSE of autonomously forecasting the Mackey-Glass signal.

$N = 200$ ESN	$N = 5,000$ ESN	$N = 5,000, d = 200$ CESN
0.0287 ± 0.0053	0.0090 ± 0.0014	0.0055 ± 0.0005

TABLE 5.2: Prediction errors for the ESN versus CESN at the Mackey-Glass prediction task. Optimal results are obtained by the CESN.

As the results in Table 5.2 show, the CESN compressed by the algorithm discussed in this section performs much better than an ESN of the same size. In fact, it even outperforms the large ESN, despite requiring far fewer operations to computer and being of much smaller dimension.

5.4 Re-using Reduced Reservoirs

In the previous section, I showed that by examining the SVD of the reservoir response to the Mackey-Glass input, a reservoir with similar dynamics but much lower dimension can be derived that performs equally well at the Mackey-Glass prediction task. As I demonstrate in this section, reservoirs produced in this way also perform better on related tasks, such as predicting a different time-series. This suggests that the SVD compression algorithm reveals redundancy that is inherent in the network itself and not just the coupled input-reservoir system. In particular, I consider the Lorenz system defined by

$$\tau \dot{x}_1 = \sigma(x_2 - x_1) \quad (5.11)$$

$$\tau \dot{x}_2 = x_1(\rho - x_3) - x_2 \quad (5.12)$$

$$\tau \dot{x}_3 = x_1 x_2 - \beta x_3 \quad (5.13)$$

$$y = ax_1 - b. \quad (5.14)$$

Equations 5.11-5.13 define the Lorenz system. I choose parameters $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$ to place the system in a chaotic regime. The factors of $\tau = 300$ are to scale time such that the characteristic time-scale of the system is similar to the Mackey-Glass system. Similarly, the observation function defined by Eq. 5.14 and $a = 0.027$, $b = 0.008$ shifts the mean and standard deviation of the one-dimensional output signal to be equal to that of Mackey-Glass. These scaling choices facilitate the use of Mackey-Glass-appropriate reservoirs for the Lorenz system without changing any of the physical properties of the system.

I first examine the prediction performance of the CESN versus the reduced dimension d by constructing a curve similar to that in Fig. 5.7. I again emphasize that the compressed reservoir (specifically, the reduced matrices) were determined by examining the ESN's response to the *Mackey-Glass* system, but the NRMSE is calculated using the CESNs to predict the *Lorenz* system.

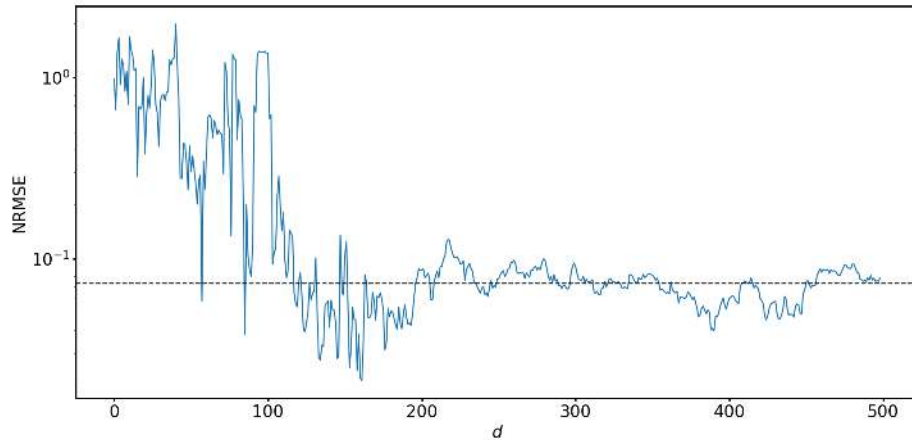


FIGURE 5.9: Comparing the full 1,000-node reservoir and various d -node reduced networks, where the reduction is performed based on the Mackey-Glass system. The CESNs are then tested by predicting the scaled Lorenz system. The performance of the 1,000 node ESN is given by the horizontal dashed line. Similar to testing with the Mackey-Glass system, the performance is relatively flat until some minimum d . When testing with Lorenz, however, the dependence on d is much noisier.

Similar to the example in Sec. 5.3, an CESN with significantly reduced dimension can predict the chaotic system approximately as accurately as the full ESN down to a certain size d . The difference from Fig. 5.9 appears to be that the performance as d is reduced is significantly noisier.

To fully characterize the ability of the Mackey-Glass-reduced reservoir to generalize to the Lorenz system, I consider ensembles of reservoirs constructed in the three following ways:

- $N = 100$ node ESNs,
- $N = 1,000, d = 100$ node CESNs that have been reduced based on the response to the Mackey-Glass system, and
- $N = 1,000, d = 100$ node CESNs that have been reduced based on the response to the Lorenz system.

If the SVD compression algorithm generalizes to different input signals, then the performance of the second ensemble of reservoirs should be somewhere in between the first and third.

To test the generalization, I create 20 reservoirs for each ensemble described above. Their performance is measured by autonomously forecasting Eq. 5.11-5.14 after a fixed training period and computing the NRMSE over one Lyapunov time. The hyperparameters for each initial reservoir is the same for the Mackey-Glass experiments in this section. The mean of these metrics are reported in Table 5.3.

As I can see from the table, the second ensemble of reservoirs performs significantly better than the first. That is, the pre-training algorithm using the Mackey-Glass system produces a reservoir that is significantly better than an ordinary ESN. I also see that using the Mackey-Glass system for the SVD algorithm is not as efficient as using the Lorenz system, as expected.

Similarly, it is worth considering how much of the benefit of pre-training with Mackey-Glass is due to the SVD algorithm revealing properties of the input signal versus properties of the reservoir. In other words, does the second ensemble perform better than the first because

the attractors are similar, or because the ESN differential equations themselves have redundancies that can be pruned with the SVD algorithm? To investigate this, I consider a fourth ensemble of reservoirs that are 100 node ESNs reduced based on the response to a *random* input signal. This is repeated for 100 reservoirs and is also presented in Table. 5.3.

$N = 100$ ESN	$N = 1,000, d = 100$ CESN Reduced with Lorenz	$N = 1,000, d = 100$ CESN Reduced with MG	$N = 1,000, d = 100$ CESN Reduced with noise
0.229 ± 0.036	0.164 ± 0.031	0.153 ± 0.028	0.143 ± 0.028

TABLE 5.3: Prediction error for the ESN versus several CESNs at the Lorenz prediction task. Each CESN has been derived from an ESN based on its response to a different input signal. All CESNs outperform the standard ESN, and all perform within a standard error of each other.

Surprisingly, the SVD-derived CESNs all performed equally well (within one standard error) of each other, and all performed better than the ESN of the same size. This suggests that the CESNs have some common dynamical features, despite being derived from the response to different input signals. This phenomenon is described in more detail in the next section.

5.5 Deriving High-Performance ESNs

As I noted in the section 5.3, the equations for the reduced network have a different form than the unpruned ESN. In particular, the recurrent connection matrix is rectangular, and there is an additional matrix transformation after the application of the tanh function. Although the reduced equations have greatly improved performance for a fixed number of nodes, the form of Eq. 5.9 is potentially undesirable in at least two ways:

- Comparison to classical ESNs is not straightforward, making it unclear how to infer from CESNs how to better design reservoirs in the first place.
- Even though the computational complexity of the reduced ESN is significantly reduced, it is greater than a classical ESN of the same size.

In this section, I make progress towards remedying these problems by examining the linearized reduced ESN equations.

5.5.1 Linear Analysis

In the typical ESN scheme, the recurrent weights are left completely random. In the compression algorithm, however, the initially random weights are modified by the data-derived matrix $\mathbf{U}_d$, and may therefore have some structure. In particular, the linear stability of the network is derived from the square matrix $\mathbf{U}_d^{-1}\mathbf{W}\mathbf{U}_d$. Particularly in light of the observation that reduced reservoirs using $\mathbf{U}_d$ derived from completely different tasks work well for other tasks, the emergent structure in these matrices may reveal some intuition behind how to better design reservoirs in the first place.

To understand the emergent nonlinear coupling in the CESN, I linearize the tanh functions in Eq. 5.8 to yield

$$c\dot{\tilde{\mathbf{x}}} = -\tilde{\mathbf{x}} + \mathbf{W}^{eff}\tilde{\mathbf{x}} + \mathbf{W}_{in}^{eff}\mathbf{u} + \mathbf{b}^{eff}, \quad (5.15)$$

where the effective matrices are given by

$$\mathbf{W}^{eff} = \mathbf{U}_d^{-1}\mathbf{x}\mathbf{U}_d, \mathbf{W} \quad (5.16)$$

$$\mathbf{W}_{in}^{eff} = \mathbf{U}_d^{-1}\mathbf{W}_{in} \quad (5.17)$$

$$\mathbf{b}^{eff} = \mathbf{U}_d^{-1}\mathbf{b}. \quad (5.18)$$

Note that the effective adjacency matrix is a *square* matrix of size $d \times d$.

Since $\mathbf{W}$ is taken from a symmetric, uniform distribution with constrained spectral radius, the distribution of the elements of $\mathbf{W}$ are fully known. The effective adjacency matrix of the CESN defined above, however, is a stochastic function of specific input system. In Fig. 5.10, I

visualize a typical matrix $\mathbf{W}$ compared to several $\mathbf{W}^{eff}$ that have been determined by several different input systems. Several characteristics stand out:

- The CESNs all have strong, positive self-coupling through their nonlinear interaction, as indicated by the diagonal lines in Fig. 5.10b-d. This has the effect of partially negating the $-\tilde{\mathbf{x}}_d$ term in Eq. 5.9 and may indicate that individual CESN nodes have a preference to be less stable than their ESN counterparts.
- The coupling is no longer symmetric. In the ESN in Fig. 5.10a, as expected, there is no preference for $\mathbf{W}_{i,j}$ over $\mathbf{W}_{j,i}$. However, this is clearly not the case in Fig. 5.10b-d. Recall that the nodes in the reduced network are ordered in a particular way—higher indexed nodes are less important for recreating the full network dynamics than lower indexed nodes. In Fig. 5.10b-d, I see that, not surprisingly, less-important nodes don't drive the dynamics of more-important nodes very strongly, whereas more-important nodes strongly drive less-important nodes. This has the effect of producing an approximately upper-triangular matrix for the CESNs.
- The i_{th} node prefers to drive the $m i_{th}$ node, where m is some constant that is characteristic of the input system. This is particularly evident in the effective matrix of the CESN subject to the random input, as indicated by the off-diagonal line of large coefficients.

Overall, Fig. 5.10 shows a strong preference for certain network topologies in the CESN. While the exact topology depends on the input system, they all appear to drastically diverge from the typical symmetric, uniform adjacency matrices used in traditional reservoir computing.

5.5.2 Linear-Equivalent ESNs

The emergence of preferred topology of the CESNs, which perform very well at time-series prediction tasks, suggests an alternative way of choosing ESN topologies. It is possible to

choose an ESN that has the same linear response as an CESN by simply choosing

$$\mathbf{W} = \mathbf{W}^{eff} \quad (5.19)$$

$$\mathbf{W}_{in} = \mathbf{W}_{in}^{eff} \quad (5.20)$$

$$\mathbf{b} = \mathbf{b}^{eff}. \quad (5.21)$$

Equations 5.19-5.21 defines a d -dimensional ESN that has the same linear response as a d -dimensional CESN, but without expanding after the tanh function. This reduces the computational complexity and recasts the network into a familiar form at the expense of a difference in the dynamical response.

Despite not fully having the same input response as the CESN, ESNs with matrices defined by Eq. 5.19-5.21 perform significantly better than randomly chosen ESNs. To characterize this phenomenon, I consider an additional ensemble of reservoirs to compare to the results in Table 5.2. This ensemble consists of ESNs of size $N = 200$, with matrices defined by Eq. 5.13, where the effective matrices were determined from a CESN of size $N = 5,000, d = 200$ and the Mackey-Glass input signal. From 20 reservoirs, the resulting NRMSE at predicting Mackey-Glass is 0.018 ± 0.003 , representing a 60 % improvement over the random ESNs.

5.6 Conclusion and Future Directions

In this chapter, I develop a supervised pre-training algorithm for reducing the dimensionality of ESNs and related reservoir computers. I investigate the properties of the resulting ESN-like networks, which I call compressed echo state networks, and find that they perform significantly better on benchmark time-series prediction tasks than ESNs of the same size.

By examining a linearization of the CESN, I show that preferred topologies emerge, and that traditional ESNs with these topologies outperform their random counterparts. These results show that data-driven network topologies within the reservoir computing paradigm are

possible and come with significant performance increases.

The results and examples in this chapter suggest several avenues of future research. A potential roadblock to efficient hardware realizations of the networks described in this chapter is the fact that the coupling matrices derived from the SVD algorithm are all dense, containing no strictly non-zero elements. The effective matrices in Fig. 5.10 suggest that this is only approximately the case, and that many connections may be negligible. Investigating connection pruning strategies, such as that proposed in Scardapane *et al.*, 2014, is desired. Second, the derivation of the ESN matrices is based on the linearization of the tanh function about the origin, but reservoir nodes spend much of their time in the nonlinear regions away from the origin. A more data-driven approach to the linearization of CESN dynamics may further reveal important structures.

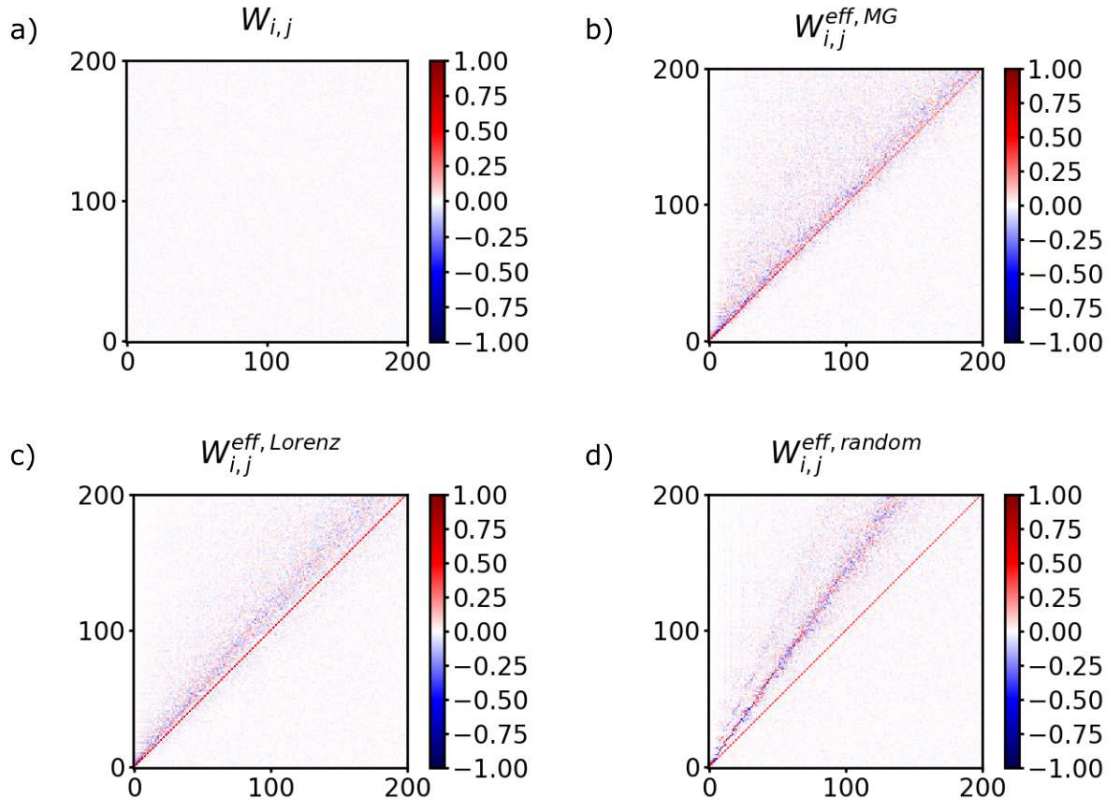


FIGURE 5.10: A visualization of typical adjacency matrices. a) The random adjacency matrix in a typical ESN. Note that the typical weight is very small, and weights are randomly distributed. b) The effective adjacency matrix derived from the Mackey-Glass system. c) The effective matrix derived from the Lorenz system. d) The effective matrix derived from a random input. Note that all effective matrices are approximately upper-triangular, with strong self-coupling, and a preference to couple to particular nodes.

Chapter 6

Conclusions and Future Research

In this thesis, I discuss several novel ideas advancing the state of understanding of RC, particularly as applied to the probing of dynamical systems. I do this through a discussion of numerical and experimental results derived a number of experiments, including a novel dimension-reduction algorithm, a deep control algorithm, and a technique for realizing RC on readily-available hardware.

In this chapter, I discuss the major discoveries in this thesis and their context within the field of RC as well as physics more generally. I also provide a discussion of future research directions that are natural extensions of the projects I have discussed in this thesis.

6.1 Discussion

In Ch. 3, I discuss an algorithm for deriving a controller for unknown dynamical systems. To RC experts, the most surprising result in this chapter is likely the fact that increasing the size of a single-layer controller does not generally increase control performance. This study emphasizes the difficulty in applying a ML algorithm when the final performance measure (control error) is different from the measure minimized during training (plant inversion error), which my results show are only loosely related to each other. It is in this sense that the algorithm in Ch. 3. is really an unsupervised training algorithm, despite the fact that labeled outputs are

used to identify $\mathbf{W}_{out}$. My strategy of iteratively adding layers to improve the final performance measure might be generalized to other problems where an unsupervised algorithm is required.

Within the broader context of physics and chaos control more specifically, the results in Ch. 3 demonstrate that chaotic systems can be robustly controlled to orbits beyond USSs and UPOs. This observation is, strictly speaking, not new—several modern techniques for control engineering have been applied in such a way. However, these techniques often require at least a learned model of the chaotic system as a first step, as I emphasize in Ch. 3. To my knowledge, this is the first technique that is capable of *directly* learning general control of a chaotic system, which is quite a leap from techniques of small perturbations that are commonly discussed in chaos control.

In Ch. 4, I analyze an unconventional RC approach using an ABN as the reservoir. From a practical perspective, the demonstrable utility of this technique is the minimization of the decision latency, which is simply the time between data inputs and derived outputs. I explore this technique with respect to the problem of real-time signal prediction, because this is a task where decision latency is often the limiting time scale. There are other applications where decision latency near the GHz regime is required, however, such as decoding bits transmitted with an optical cable. The ABN reservoir computer is a suitable candidate for next-generation ML solutions to problems such as this.

Another interesting perspective of the ABN reservoir computer is through generalized synchronization of chaos. As I emphasize throughout this thesis, the ability of a reservoir to synchronize to a driving system is critical to the RC paradigm. I explicitly demonstrate in Ch. 4 that the ABN does synchronize to chaotic and even random inputs. This is perhaps surprising given the exotic nature of the ABN. In particular, it is a time-delay system and is approximately Boolean, which are two features not shared with the state-space of the drive system. A general ABN fabricated on an FPGA does not synchronize, however. It is important that the nodes take the form of the threshold function, where other node forms do not result in synchronization. This suggests a criticality phenomenon that is not explored in this system, but may provide

guidance for better design of ABN reservoir computers.

In Ch. 5, I investigate a dimensionality-reduction algorithm for a general class of RC techniques and report numerical results on ESNs in particular. These results demonstrate a new avenue for reservoir optimization, which is a long-term goal of RC study that has not yet yielded many practical techniques.

The studies in Ch. 5 further suggest interesting dynamical phenomena in random, recurrent neural networks. It is a reasonable *a priori* guess that the recurrent weights as identical, independently distributed variables yields an optimal reservoir, but this is clearly not the case, even for a completely random input signal. This means that relative node indices impact the ideal weight distributions, suggesting that nonlocality be built into the network. This also suggests an influence of the node dynamics on the optimal network structure that deserves exploration.

6.1.1 Future Directions

The results in this thesis suggest several avenues of future research. Some are specific extensions of the projects I discuss in this thesis, and some branch out into more general RC questions. Below, I describe some of these possible future projects.

The control algorithm discussed in Ch. 3 invites a number of extensions. First, a stability analysis is desired. While this is notoriously difficult for RNNs, it is likely necessary before such a control algorithm could be employed when human safety is involved. The analysis is complicated primarily by two factors, namely the delay between $\mathbf{y}$ and $\mathbf{r}$ and the unknown form of the plant. Assumptions about $\mathbf{f}$ and $\mathbf{g}$ will surely have to be made.

A first step towards a robust stability analysis is likely found by restricting attention to constant $\mathbf{r}(t)$ to eliminate the delay in the dynamics of the controlled plant. One suggestion is to linearize the controller dynamics around the requested fixed point. One can then determine sufficient conditions on the plant for stability of the coupled system.

Second, a more robust process for identifying the hyperparameters, particularly those of the deeper layers, is desired. While I have provided some analysis in Ch. 3, I have not fully escaped

the RC paradigm of employing heuristics to design the controller. This algorithm surely invites investigation with gradient descent or Bayesian optimization techniques (Griffith, Pomerance, and Gauthier, 2019). It would be very interesting to see how optimal parameters vary from layer to layer.

Third, a possible implementation of the control scheme is to use a different, potentially imperfect controller as the first layer. The deeper layers of the controller then modify this first control signal in the same way that they modify the first reservoir's control signal in the experiments I describe in Ch. 3. I have done this for the simple example of a quadcopter controlled by a PID controller, but more sophisticated controllers are used in practice. It is worth investigating whether using a modern controller as the first layer and then adding reservoir layers is an efficient approach for more realistic use-cases.

The success of the ABN reservoir computer is, in some sense, surprising, given the spatial simplicity of the reservoir state. It is precisely this simplicity that enables a rapid output layer and therefore output feedback. It appears that all of the complexity is in the temporal domain, very similar to spiking neural networks such as LSMs. However, LSMs perform poorly at time-series prediction, often requiring a time-dependent output layer such as a finite impulse response filter. The ABN reservoir computer suggests that it is the spiking nature of these networks that makes prediction difficult, not the Boolean state at any given time. A better understanding of this discrepancy is desired.

Further, and possibly important to understanding the aforementioned discrepancy, a more accurate modeling of the ABN reservoir computer is desired. Although the Glass model discussed in Ch. 4 accurately describes some properties of the ABN reservoir computer, it fails to fully characterize the dynamics of the reservoir in response to a time-dependent input. Having an accessible model will aid in design of ABN reservoir computer, particularly since the compilation process is currently quite long.

Further still, there is presently a problem with large resource consumption by the delay lines in the ABN reservoir computer. If the use of shorter delays was possible, the footprint of

the ABN reservoir computer could be significantly reduced, as the majority of the resources are currently consumed by the necessary inverter gates. Using fewer delays necessitates a faster-switching input signal. This could possibly be accomplished with some FPGA's dedicated-logic transceivers. It may also be the case that an input approaching the characteristic rise time of the nodes themselves is beneficial, allowing the input signal to more fully access the nonlinearities of the node response.

The compression algorithm discussed in Ch. 5 exposes very clearly the disadvantages of the present methods for instantiating the reservoir connection matrices. While the topologies and connection strengths revealed in Sec. 5.3 are data-driven, they remain stochastic in the sense that they are derived from random initial matrices. This suggests that random, but not uniformly random, is a more appropriate way to instantiate networks.

Expanding on this study, a more network-theoretic analysis of the efficient reservoirs is likely to be illuminating. As I have pointed out, an upper-triangular and strongly self-coupling structure emerges, but there is surely much more to understand. A study of the spectral radius, the distribution of eigenvalues, and the effective connectivity of the compressed reservoirs may reveal interesting patterns.

Finally, combining the compression strategy with a pruning strategy such as Scardapane *et al.*, 2014 is likely to produce reservoirs highly that are highly efficient for hardware realization, such as the controller used in Sec. 3.5. These strategies should be investigated and demonstrated.

Bibliography

- Abarbanel, Henry DI, Nikolai F Rulkov, and Mikhail M Sushchik (1996). "Generalized synchronization of chaos: The auxiliary system approach". In: *Physical Review E* 53.5, p. 4528.
- Abdelsalam, Ahmed M, JM Pierre Langlois, and Farida Cheriet (2017). "A configurable FPGA implementation of the tanh function using DCT interpolation". In: *2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, pp. 168–171.
- Alomar, Miquel L *et al.* (2016). "FPGA-based stochastic echo state networks for time-series forecasting". In: *Comput. Intel. Neurosc.* 2016, p. 15.
- Antonik, Piotr *et al.* (2016). "Towards pattern generation and chaotic series prediction with photonic reservoir computers". In: *SPIE LASE*. International Society for Optics and Photonics, 97320B–97320B.
- Apostel, Stephen (2017). "Dynamics of driven complex autonomous Boolean networks with application to reservoir computing". PhD thesis. Technische Universitat Berlin.
- Appeltant, Lennert *et al.* (2011). "Information processing using a single dynamical node as complex system". In: *Nat. Commun.* 2, p. 468.
- Bojarski, Mariusz *et al.* (2016). "End to end learning for self-driving cars". In: *arXiv preprint arXiv:1604.07316*.
- Bueno, Julián *et al.* (2017). "Conditions for reservoir computing performance using semiconductor lasers with delayed optical feedback". In: *Opt. Express* 25.3, pp. 2401–2412.

- Büsing, Lars, Benjamin Schrauwen, and Robert Legenstein (2010). "Connectivity, dynamics, and memory in reservoir computing with binary and analog neurons". In: *Neural Comput.* 22.5, pp. 1272–1311.
- Caluwaerts, Ken *et al.* (2013). "The spectral radius remains a valid indicator of the echo state property for large reservoirs". In: *Neural Networks (IJCNN), The 2013 International Joint Conference on.* IEEE, pp. 1–6.
- Caluwaerts, Ken *et al.* (2014). "Design and control of compliant tensegrity robots through simulation and hardware validation". In: *Journal of the royal society interface* 11.98, p. 20140520.
- Canaday, Daniel, Aaron Griffith, and Daniel J Gauthier (2018). "Rapid time series prediction with a hardware-based reservoir computer". In: *Chaos: An Interdisciplinary Journal of Nonlinear Science* 28.12, p. 123119.
- Carroll, Thomas L (2018). "Using reservoir computers to distinguish chaotic signals". In: *Physical Review E* 98.5, p. 052209.
- Chang, Austin *et al.* (1998). "Stabilizing unstable steady states using extended time-delay autosynchronization". In: *Chaos* 8.4, pp. 782–790.
- Chiou, CB *et al.* (2009). "The application of fuzzy control on energy saving for multi-unit room air-conditioners". In: *Applied thermal engineering* 29.2-3, pp. 310–316.
- Chowdhary, Girish *et al.* (2013). "Guidance and control of airplanes under actuator failures and severe structural damage". In: *Journal of Guidance, Control, and Dynamics* 36.4, pp. 1093–1104.
- Derrida, Bernard and Yves Pomeau (1986). "Random networks of automata: a simple annealed approximation". In: *Europhys. Lett.* 1.2, p. 45.
- Dion, Guillaume, Salim Mejaouri, and Julien Sylvestre (2018). "Reservoir computing with a single delay-coupled non-linear mechanical oscillator". In: *Journal of Applied Physics* 124.15, p. 152132.
- Du, Chao *et al.* (2017). "Reservoir computing using dynamic memristors for temporal information processing". In: *Nature communications* 8.1, p. 2204.

- Dutoit, Xavier, Hendrik Van Brussel, Marnix Nuttin, *et al.* (2007). "A first attempt of reservoir pruning for classification problems." In: *ESANN*. Citeseer, pp. 507–512.
- Engel, Daniel, Lars Hüttenberger, and Bernd Hamann (2012). "A survey of dimension reduction methods for high-dimensional data analysis and visualization". In: *Visualization of Large and Unstructured Data Sets: Applications in Geospatial Planning, Modeling and Engineering-Proceedings of IRTG 1131 Workshop 2011*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- Fernando, Chrisantha and Sampsa Sojakka (2003). "Pattern recognition in a bucket". In: *European Conference on Artificial Life*. Springer. Berlin, Heidelberg, pp. 588–597.
- Funahashi, Ken-ichi and Yuichi Nakamura (1993). "Approximation of dynamical systems by continuous time recurrent neural networks". In: *Neural Networks* 6.6, pp. 801–806.
- Glass, Leon and Stuart A Kauffman (1973). "The logical analysis of continuous, non-linear biochemical control networks". In: *J. Theor. Biol.* 39.1, pp. 103–129.
- Golub, Gene H and Christian Reinsch (1971). "Singular value decomposition and least squares solutions". In: *Linear Algebra*. Springer, pp. 134–151.
- Griffith, Aaron, Andrew Pomerance, and Daniel J Gauthier (2019). "Forecasting Chaotic Systems with Very Low Connectivity Reservoir Computers". In: *arXiv preprint arXiv:1910.00659*.
- Haynes, Nicholas D. *et al.* (2015). "Reservoir computing with a single time-delay autonomous Boolean node". In: *Phys. Rev. E* 91 (2), p. 020801.
- Hinton, Geoffrey E (2007). "Learning multiple layers of representation". In: *Trends in cognitive sciences* 11.10, pp. 428–434.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White (1989). "Multilayer feedforward networks are universal approximators". In: *Neural Networks* 2.5, pp. 359–366.
- Islam, Raza ul, Jamshed Iqbal, and Qudrat Khan (2014). "Design and comparison of two control strategies for multi-DOF articulated robotic arm manipulator". In: *Journal of Control Engineering and Applied Informatics* 16.2, pp. 28–39.

- Jaeger, Herbert (2001). “The “echo state” approach to analysing and training recurrent neural networks-with an erratum note”. In: *Bonn, Germany: German National Research Center for Information Technology GMD Technical Report 148.34*, p. 13.
- (2002). *Tutorial on training recurrent neural networks, covering BPPT, RTRL, EKF and the “echo state network” approach*. Vol. 5. GMD-Forschungszentrum Informationstechnik Bonn.
- (2005). “Reservoir riddles: Suggestions for echo state network research”. In: *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005*. Vol. 3. IEEE, pp. 1460–1462.
- Jiang, Fei, Hugues Berry, and Marc Schoenauer (2008). “Supervised and evolutionary learning of echo state networks”. In: *International Conference on Parallel Problem Solving From Nature*. Springer, pp. 215–224.
- Khodabandehlou, Hamid and Mohammad Sami Fadali (2017). “Echo State versus Wavelet Neural Networks: Comparison and Application to Nonlinear System Identification”. In: *IFAC-PapersOnLine* 50.1, pp. 2800–2805.
- Kim, Edward J and Robert J Brunner (2016). “Star-galaxy classification using deep convolutional neural networks”. In: *Monthly Notices of the Royal Astronomical Society*, stw2672.
- Kocarev, Ljupco and Ulrich Parlitz (1996). “Generalized synchronization, predictability, and equivalence of unidirectionally coupled dynamical systems”. In: *Physical review letters* 76.11, p. 1816.
- Larger, Laurent *et al.* (2012). “Photonic information processing beyond Turing: an optoelectronic implementation of reservoir computing”. In: *Opt. Express* 20.3, pp. 3241–3249.
- Larger, Laurent *et al.* (2017). “High-speed photonic reservoir computing using a time-delay-based architecture: Million words per second classification”. In: *Phys. Rev. X* 7.1, p. 011015.
- Letellier, Christophe (2013). *Chaos in nature*. Vol. 81. World Scientific.
- Li, Decai, Min Han, and Jun Wang (2012). “Chaotic time series prediction based on a novel robust echo state network”. In: *IEEE T. Neur. Net. Lear.* 23.5, pp. 787–799.
- Lu, Zhixin *et al.* (2017). “Reservoir observers: Model-free inference of unmeasured variables in chaotic systems”. In: *Chaos* 27.4, p. 041102.

- Lukoševičius, Mantas (2012). "A practical guide to applying echo state networks". In: *Neural networks: Tricks of the trade*. Berlin, Heidelberg: Springer, pp. 659–686.
- Lukoševičius, Mantas and Herbert Jaeger (2009). "Reservoir computing approaches to recurrent neural network training". In: *Comput. Sci. Rev.* 3.3, pp. 127–149.
- Maass, Wolfgang, Thomas Natschläger, and Henry Markram (2002). "Real-time computing without stable states: A new framework for neural computation based on perturbations". In: *Neural Comput.* 14.11, pp. 2531–2560.
- Mackey, Michael C and Leon Glass (1977). "Oscillation and chaos in physiological control systems". In: *Science* 197.4300, pp. 287–289.
- Manjunath, G, P Tino, and H Jaeger. "Theory of Input Driven Dynamical Systems". In:
- Mazzoni, Pietro, Richard A Andersen, and Michael I Jordan (1991). "A more biologically plausible learning rule for neural networks." In: *Proceedings of the National Academy of Sciences* 88.10, pp. 4433–4437.
- Mehta, Pankaj and David J Schwab (2014). "An exact mapping between the variational renormalization group and deep learning". In: *arXiv preprint arXiv:1410.3831*.
- Melin, Patricia *et al.* (2006). "Voice Recognition with Neural Networks, Type-2 Fuzzy Logic and Genetic Algorithms." In: *Engineering Letters* 13.3.
- Nagy, Zoltan K *et al.* (2007). "Evaluation study of an efficient output feedback nonlinear model predictive control for temperature tracking in an industrial batch reactor". In: *Control Engineering Practice* 15.7, pp. 839–850.
- Ng, Hong-Wei *et al.* (2015). "Deep learning for emotion recognition on small datasets using transfer learning". In: *Proceedings of the 2015 ACM on international conference on multimodal interaction*. ACM, pp. 443–449.
- Nørgård, Peter Magnus *et al.* (2000). "Neural Networks for Modelling and Control of Dynamic Systems-A Practitioner's Handbook". In:

- Norton, David and Dan Ventura (2006). "Preparing more effective liquid state machines using hebbian learning". In: *The 2006 IEEE International Joint Conference on Neural Network Proceedings*. IEEE, pp. 4243–4248.
- Ott, Edward, Celso Grebogi, and James A Yorke (1990). "Controlling chaos". In: *Phys. Rev. Lett.* 64.11, p. 1196.
- Paraskevopoulos, Paraskevas N (2017). *Modern control engineering*. CRC Press.
- Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio (2013). "On the difficulty of training recurrent neural networks". In: *International conference on machine learning*, pp. 1310–1318.
- Pathak, Jaideep *et al.* (2017). "Using machine learning to replicate chaotic attractors and calculate Lyapunov exponents from data". In: *Chaos* 27.12, p. 121102.
- Pathak, Jaideep *et al.* (2018a). "Hybrid forecasting of chaotic processes: Using machine learning in conjunction with a knowledge-based model". In: *Chaos* 28.4, p. 041101.
- Pathak, Jaideep *et al.* (2018b). "Model-free prediction of large spatiotemporally chaotic systems from data: a reservoir computing approach". In: *Phys. Rev. Lett.* 120.2, p. 024102.
- Pyragas, Kestutis (1992). "Continuous control of chaos by self-controlling feedback". In: *Phys. Lett. A* 170.6, pp. 421–428.
- Qiao, Junfei *et al.* (2018). "A deep belief network with PLSR for nonlinear system modeling". In: *Neural Networks* 104, pp. 68–79.
- Rahimi, Ali and Benjamin Recht (2008). "Random features for large-scale kernel machines". In: *Advances in neural information processing systems*, pp. 1177–1184.
- Rivera, Daniel E *et al.* (2003). "'Plant-Friendly' system identification: a challenge for the process industries". In: *IFAC Proceedings Volumes* 36.16, pp. 891–896.
- Robinson, Jennifer L *et al.* (2010). "Metaanalytic connectivity modeling: delineating the functional connectivity of the human amygdala". In: *Human brain mapping* 31.2, pp. 173–184.
- Rohlf, Thimo and Stefan Bornholdt (2002). "Criticality in random threshold networks: annealed approximation and beyond". In: *Physica A* 310.1-2, pp. 245–259.

- Rulkov, Nikolai F *et al.* (1995). "Generalized synchronization of chaos in directionally coupled chaotic systems". In: *Physical Review E* 51.2, p. 980.
- Sande, Guy Van der, Daniel Brunner, and Miguel C Soriano (2017). "Advances in photonic reservoir computing". In: *Nanophotonics* 6.3, pp. 561–576.
- Scardapane, Simone *et al.* (2014). "An effective criterion for pruning reservoir's connections in echo state networks". In: *2014 International joint conference on neural networks (IJCNN)*. IEEE, pp. 1205–1212.
- Schaetti, Nils, Michel Salomon, and Raphaël Couturier (2016). "Echo state networks-based reservoir computing for mnist handwritten digits recognition". In: *2016 IEEE Intl Conference on Computational Science and Engineering (CSE) and IEEE Intl Conference on Embedded and Ubiquitous Computing (EUC) and 15th Intl Symposium on Distributed Computing and Applications for Business Engineering (DCABES)*. IEEE, pp. 484–491.
- Schrauwen, Benjamin *et al.* (2008a). "Compact hardware liquid state machines on FPGA for real-time speech recognition". In: *Neural Networks* 21.2-3, pp. 511–523.
- Schrauwen, Benjamin *et al.* (2008b). "Improving reservoirs using intrinsic plasticity". In: *Neurocomputing* 71.7-9, pp. 1159–1171.
- Silver, David *et al.* (2017). "Mastering chess and shogi by self-play with a general reinforcement learning algorithm". In: *arXiv preprint arXiv:1712.01815*.
- Takens, Floris (1981). "Detecting strange attractors in turbulence". In: *Dynamical systems and turbulence, Warwick 1980*. Springer, pp. 366–381.
- Tanaka, Gouhei *et al.* (2019). "Recent advances in physical reservoir computing: a review". In: *Neural Networks*.
- Torrejon, Jacob *et al.* (2017). "Neuromorphic computing with nanoscale spintronic oscillators". In: *Nature* 547.7664, p. 428.
- Triesch, Jochen (2005). "A gradient rule for the plasticity of a neuron's intrinsic excitability". In: *International Conference on Artificial Neural Networks*. Springer, pp. 65–70.

- Van Nieuwenburg, Evert PL, Ye-Hua Liu, and Sebastian D Huber (2017). “Learning phase transitions by confusion”. In: *Nature Physics* 13.5, p. 435.
- Verstraeten, David (2009). “Reservoir Computing: computation with dynamical systems”. PhD thesis. Ghent University.
- Verstraeten, David *et al.* (2007). “An experimental unification of reservoir computing methods”. In: *Neural Networks* 20.3, pp. 391–403.
- Waegeman, Tim, Benjamin Schrauwen, *et al.* (2012). “Feedback control by online learning an inverse model”. In: *IEEE T. Neur. Net. Lear.* 23.10, pp. 1637–1648.
- Wyffels, Francis and Benjamin Schrauwen (2010). “A comparative study of reservoir computing strategies for monthly time series prediction”. In: *Neurocomputing* 73.10, pp. 1958–1964.
- Yperman, Jan and Thijs Becker (2016). “Bayesian optimization of hyper-parameters in reservoir computing”. In: *arXiv preprint arXiv:1611.05193*.
- Zhang, Rui *et al.* (2009). “Boolean chaos”. In: *Phys. Rev. E* 80.4, p. 045202.

Appendix A

Hardware Descriptions for ABN-RC

In this appendix, I present the hardware description code for the reservoir nodes, delay lines, and a small reservoir. The code is written in *Verilog* and compiled using Altera's *Quartus Prime* software. Some parts of the code depend on the number of reservoir nodes N , the node in-degree k , and the number of bits n used to represent the input signal $u(t)$. I give explicitly the code only for $N = 3$, $k = 2$, and $n = 1$, but generalizations are straightforward.

A.1 LUT Nodes

As discussed in Sec. 4.3, reservoir nodes implement a Boolean function $\Lambda_i : \mathbf{Z}_2^{k+n} \rightarrow \mathbf{Z}_2$ of the form given in Eq. 4.4. Each Boolean function can be defined by a Boolean string of length 2^{k+n} that specifies the look-up-table (LUT) corresponding of the Boolean function. For example, the AND function maps $\mathbf{Z}_2^2 \rightarrow \mathbf{Z}_2$ and has the LUT defined in Fig. A.1. The Boolean string that defines the AND function is 0001 as can be seen from the the right-most column of the LUT.

FIGURE A.1: The LUT for the AND function. It can be specified by the Boolean string that makes up the right-most column.

```

1 module node(node_in, node_out);
2 parameter lut = 8'b00000001;
3 input [2:0] node_in;
4 output reg node_out;
5
6 always @(*) begin
7     case (node_in)
8         3'b000 : node_out = lut[7];
9         3'b001 : node_out = lut[6];
10        3'b010 : node_out = lut[5];
11        3'b011 : node_out = lut[4];
12        3'b100 : node_out = lut[3];
13        3'b101 : node_out = lut[2];
14        3'b110 : node_out = lut[1];
15        3'b111 : node_out = lut[0];
16    endcase
17 end
18
19 endmodule

```

FIGURE A.2: Verilog code for a generic node that can implement any 3-input Boolean function, specified by a Boolean string of length 8.

The code given in Fig. A.2 generates a node with Boolean function based on any LUT of length $2^3 = 8$. The module **node** is declared in line 1 with inputs **node_in** and output **node_out**. The width of **node_in** is 3 bits as specified in line 3. The parameter **lut** is declared in line 2. Note that it is initialized to some value as required by *Quartus*, but this value is changed whenever a node is declared within the larger code that defines the complete reservoir.

The main part of the code is within an **always @(*)** block, which creates an inferred sensitivity list and is used to create arbitrary combinational logic. Line 7 specifies that values before the colon in the proceeding lines correspond to **node_in**. The statement following the colon determines which value is assigned to **node_out**. In effect, line 8 simply specifies that, whenever the value of **node_in** is a 3-bit string equal to 000, the value of **node_out** is whatever the value of **lut[7]** is. For example, if I create an instance of the module **node** with parameter **lut=8'b00000001**, then the node will execute the 3 input AND function.

```

1 module delay_line(delay_in, delay_out);
2   parameter m = 15;
3   input delay_in;
4   output delay_out;
5
6   wire [2*m-1:0] delay /*synthesis keep*/;
7
8   assign delay[0] = ~delay_in;
9   assign delay_out = delay[2*m-1];
10
11   genvar i;
12   generate
13     for (i=0; i<2*m-1; i=i+1) begin : generate_delay
14       assign delay[i+1] = ~delay[i];
15     end
16   endgenerate
17
18 endmodule

```

FIGURE A.3: Verilog code for a delay line with $2m$ inverter gates.

A.2 Autonomous Reservoir

As discussed in Sec. 4.3, delay lines are created as chains of pairs of inverter gates. Such a chain of length $2m$ is created with the code in Fig. A.3. Similarly to the **node** module, the **delay_line** module is declared in line 1 with the input **delay_in** and output **delay_out**. It has a parameter m which specifies the number of pairs in the chain and can be changed when calling a specific instance of **delay_line**. A number of wires are declared in line 5 and will be used as the inverter gates. Note the important directive */*synthesis keep*/*, which instructs the compiler to not simplify the module by eliminating the inverter gates. This is necessary, because otherwise the compiler would realize that **delay_line**'s function is trivial and remove all of the inverter gates.

Lines 7-8 specify the beginning and end of the delay chain as the **delay_in** and **delay_out**, respectively. Lines 10-16 use a **generate** block to create a loop that places inverter gates in between **delay_in** and **delay_out**, resulting in a delay chain of length $2m$.

The **reservoir** module is the code that creates N instances of **node** and connects them Nk

instances of **delay_line**. As an illustrative example, consider a 3-node reservoir with the following parameters

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.3 & 0 \\ -0.2 & 0 & 0.1 \\ -0.3 & 0.2 & 0 \end{bmatrix} \quad (\text{A.1})$$

$$\mathbf{W}_{in} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.2 \end{bmatrix} \quad (\text{A.2})$$

$$\boldsymbol{\tau} = \begin{bmatrix} 10 & 15 & 0 \\ 6 & 0 & 7 \\ 12 & 10 & 0 \end{bmatrix} \quad (\text{A.3})$$

and only a 1-bit representation of $u(t)$. When I pass $u(t)$ and $x(t)$ into the **node** module, I index such that $u(t)$ comes first, as seen from the **reservoir** module below.

With Eq. 4.4 and Eq. A.1-A.3, the LUTs for each node can be explicitly calculated as 01111111, 0100000000, and 01001101 for nodes 1-3, respectively. The matrix $\boldsymbol{\tau}$ specifies the delays in integer multiples of $2\tau_{inv}$. A network with this specification is realized by the module **reservoir** in Fig. A.4 and the **node** and **delay_in** modules described in this section.

Like the other modules, **reservoir** requires a module declaration, parameter declarations, and input/output declarations. Here, I also declare a wire **x_tau** that is the delayed reservoir state. In lines 9-11, the nodes are declared with the appropriate parameters and connections and are named **node_0**, **node_1**, and **node_2** respectively. The 6 delay lines are declared and named in lines 13-18.


```

1 module reservoir(u, x);
2 parameter N = 3;
3 parameter k = 2;
4 parameter m = 1;
5 input [m-1:0] u;
6 output [N-1:0] x;
7
8 wire [N*k-1:0] x_tau;
9
10 node #(8'b01111111) node_0 {(u, x_tau[0], x_tau[1], x[0]);
11 node #(8'b01000000) node_1 {(u, x_tau[2], x_tau[3], x[1]);
12 node #(8'b01001101) node_2 {(u, x_tau[4], x_tau[5], x[2]);
13
14 delay_line #(10) delay_0 (x[0], x_tau[0]);
15 delay_line #(15) delay_1 (x[1], x_tau[1]);
16 delay_line #(6) delay_2 (x[0], x_tau[2]);
17 delay_line #(7) delay_3 (x[2], x_tau[3]);
18 delay_line #(10) delay_4 (x[0], x_tau[4]);
19 delay_line #(12) delay_5 (x[1], x_tau[5]);
20
21 endmodule

```

FIGURE A.4: Verilog code describing a simple reservoir. The connections and LUTs are determined from Eq. 5.2 and Eq. A.1-A.3. Lines 9-11 declare 3 nodes. Lines 13-18 declare delay lines that connect them.

A.3 Synchronous Components

In this section, I discuss the details of the synchronous components that interact with the autonomous reservoir. These components regulate the reservoir input signal, the operation mode (training or autonomous), the calculation of the output signal, and record the reservoir state.

Crucial to successful operation is access to a **sampler** module that reads data from the reservoir and a **player** module that writes data into the reservoir. The details of these modules are not discussed here as they depend on the device and the application of the reservoir computer. I assume that these modules are synchronized by a global clock **clk** such that **sampler** (**player**) reads (writes) data on the rising edge of **clk**,

In Fig. A.5 I present a sample Verilog code for a high-level module **reservoir_computer** containing the reservoir and synchronous components. An instance of a **sampler** module is coupled to a global clock **clk** and outputs an m -bit wide signal **u**, a 1 bit signal **mode** that

```

1 module reservoir_computer(clk);
2 input clk;
3
4 wire mode;
5 wire [m-1:0] u;
6 wire [2*m*(N+1)-1:0] W_out;
7 wire [N-1:0] x;
8 wire [m-1:0] v;
9
10 reg [N-1:0] x_reg;
11 reg [m-1:0] v_reg;
12 wire [m-1:0] u_v;
13
14 sampler sampler_1 (clk, mode, u, W_out);
15 player player_1 (clk, x, v);
16
17 assign u_v = mode ? u : v;
18
19 reservoir reservoir_1 (u_v, x);
20
21 output_layer output_layer_1 (W_out, u_v, x_reg, v);
22
23 always @(posedge clk) begin
24     x_reg <= x;
25     v_reg <= v;
26 end
27
28 endmodule

```

FIGURE A.5: Verilog code describing the reservoir computer. It contains the **reservoir** module discussed in App. A and various synchronous components.

determines the mode of operation for the reservoir, and a $2m(N+1)$ -bit wide signal **W_out** that determines the output weight matrix. An instance of a **player** module is also coupled to a global clock **clk** and inputs an N -bit wide signal **x** and a m -bit wide signal **v**. Depending on how these modules are implemented, they may also be coupled to other components, such as on-board memory or other FPGA clocks.

As seen in line 17, the state of **mode** determines whether **u** or **v** drives the reservoir. This bit is set to 1 during training and 0 after training to allow the reservoir to evolve autonomously.

The wire **clk** registers **x** and **v** so that **output_layer** sees a value of **x** that is constant throughout one period t_{sample} and outputs a value **v** that is constant over that same interval (see Fig. 4.3). The module **output_layer** performs the operation $\mathbf{W}_{out}(\mathbf{x}, \mathbf{u})$, as described in Sec. 4.4.2.

W_{out} is a flattened array of the $N + 1$ output weights represented by $2m$ bits, with the extra bits being necessary to avoid errors in the intermediate addition calculations.

Appendix B

Hardware Description for dESN Controller

In this appendix, I detail the hardware descriptions necessary to construct the dESN controller. This includes constructing the tanh activation function, the synchronous delay line, the hard-coding of fixed reservoir weights, and a regulator for the various signals.

B.1 Tanh LUT

I choose to implement the tanh function with a 10-bit LUT. The advantage of this straightforward approach is that it is fast and accurate, while the disadvantage is that it requires more LEs than some alternative approaches (Abdelsalam, Langlois, and Cheriet, 2017). It also requires minimal memory, since one LUT for each tanh function is stored, making it attractive to my implementation where I wish to conserve memory for storing of circuit and controller signals.

The i th row of the 10-bit LUT is determined by dividing the real interval $[-8, 8]$ into 2^{10} segments and calculating the floor of tanh for the floor of each of these segments. This LUT requires $10 \times 2^{10} = 10240$ bits of memory to store. It can be stored in a single module **TanhLUT** with the Verilog in Fig. B.1. This module has no inputs and simply outputs an array of wires **tanh_lut** that connect to on-board RAM.

```

1 module TanhLUT(tanh_lut);
2 output reg [9:0] tanh_lut [1024];
3
4 initial begin
5     tanh_lut[0] = 10'b0000000000;
6     tanh_lut[1] = 10'b0000000001;
7     tanh_lut[2] = 10'b0000000010;
8     ...
9     tanh_lut[1023] = 10'b1111111111;
10 end
11 endmodule

```

FIGURE B.1: Verilog code for the TanhLUT module. It only outputs a single wire, which defines the LUT for the tanh function. The assignments in the initial block are the rows of the LUT as determined by the procedure outlined in this appendix.

The memory in **TanhLUT** can be accessed by any node simultaneously, but each node needs its own multiplexer, which takes up 225 LEs per node on Cyclone V devices. Each activation is realized with a separate **Tanh** module described in Fig. B2. This module takes the **tanh_lut** wires and **activation** as inputs and produces **tanh** as output. The final resource cost of the tanh activation functions is then $N225$ LEs and 10240 bits of RAM.

```

1 module Tanh(in, tanh_lut, out);
2 input [9:0] in;
3 input [9:0] tanh_lut [1024];
4 output reg [9:0] out;
5
6 always @(*) begin
7     case (in)
8         10'b0000000000: out <= tanh_lut[0];
9         10'b0000000001: out <= tanh_lut[1];
10        ...
11        10'b1111111111: out <= tanh_lut[1023];
12    endcase
13 end
14 endmodule

```

FIGURE B.2: Verilog code for the Tanh module. It takes in a 10-bit input and the **tanh_lut** wire outputted by an instance of the TanhLUT module. The **always** block defines combinational logic that is effectively a 10-to-10 multiplexer.

```

1 module SyncDelayLine(in, clk, delay, out);
2 parameter MAX_DELAY_WIDTH = 8;
3 input [11:0] in;
4 input clk;
5 input [MAX_DELAY_WIDTH-1:0] delay;
6 output [11:0] out;
7
8 localparam MAX_DELAY = 2**MAX_DELAY_WIDTH;
9
10 reg [11:0] in_delayed [MAX_DELAY];
11
12 always @(posedge clk) begin
13     in_delayed[0] <= in;
14 end
15
16 genvar i;
17 generate
18 for (i=0; i<MAX_DELAY-1; i=i+1) begin : generate_delays
19     always @(posedge clk) begin
20         in_delayed[i+1] <= in_delayed[i];
21     end
22 end
23 endgenerate
24
25 assign out = in_delayed[delay];
26 endmodule

```

FIGURE B.3: Verilog code for the SyncDelayLine module. It has a single parameter determining the maximum number of delaying registers. It operates by generating a series of registers, passing along the **in** wire on the rising edge of **clk**. The selector wire **delay** determines which of these registers is connected to the output.

B.2 Synchronous Delay Line

In order to realize a δ delay as described in Sec. 3.5, I use a series of registers and a multiplexer to select the value of δ . A more general delay line can be created with inverter gates as in A.2, but I require the delay be a multiple of the sampling period so that it can be read into memory without error. The module for the synchronous delay line is described in Fig. B.3.

The experiment described in Sec. 3.5 requires a single instance of **SyncDelayLine**, which takes as input a wire **delay** that selects the value of δ , a global clock **clk**, and a wire **in** which is either the measured signal from the ADC or the stored value of the reference signal from RAM, depending on the mode of operation. It has a single parameter **max_delay** that determines the

```

1 ...
2 parameter N=2
3 wire signed [11:0] u[4]
4 wire signed [31:0] W_in_u [N][4];
5 wire signed [31:0] W_x [N][N];
6 reg signed [31:0] x [N];
7
8 assign W_in_u[0][0] = u[0] * 4'sd1;
9 assign W_in_u[0][1] = u[1] * 4'sd6;
10 assign W_in_u[0][2] = u[2] * -4'sd6;
11 assign W_in_u[0][3] = u[3] * 4'sd4;
12 assign W_in_u[1][0] = u[0] * 4'sd1;
13 assign W_in_u[1][1] = u[1] * 4'sd4;
14 assign W_in_u[1][2] = u[2] * 4'sd5;
15 assign W_in_u[1][3] = u[3] * 4'sd3;
16
17 assign W_x[0][0] = x[0] * 4'sd4;
18 assign W_x[0][1] = x[1] * 4'sd4;
19 assign W_x[1][0] = x[1] * 4'sd6;
20 assign W_x[1][1] = x[0] * -4'sd3;
21 ...

```

FIGURE B.4: Verilog code for the multiplying by hard-coded weights. Note that this is just a snippet of code that might go inside a reservoir module or a top-level module, depending on design. The parameter N specifies the number of nodes. The weights are hard-coded as 4-bit signed, decimal numbers (4'sdxx). The multiplied matrices $\mathbf{W}_{in}\mathbf{u}$ and $\mathbf{W}_x$ correspond to the sign register arrays $\mathbf{W}_{in_u}$ and $\mathbf{W}_x$ in hardware, respectively.

largest value of the delay that can be selected.

B.3 Weights

To dramatically reduce resource count, the bits describing $\mathbf{W}_{in}$, $\mathbf{W}$, $\mathbf{b}$, and c are written directly into the hardware description prior to compiling the design. To further reduce complexity, $\mathbf{W}_{in}$ and $\mathbf{W}$ are left to a 4-bit resolution. The reservoir dynamics are then described by Verilog code such as the snippet in Fig. B.4.

The output weights, on the other hand, are determined from real-time data and therefore must be left to change mid-operation. This is done by altering RAM labeled $\mathbf{W}_{out}$ and determining the reservoir, such as with an Altera Megafunction to implement the adder tree.

B.4 Regulator

Recall from Sec. 3.5 that the configuration of the controller with respect to the plant changes from training to control phases. It is also the case that, since reservoirs are added to the controller one at a time, the control signal depends on which layer is currently being trained. This requires a global regulator that determines which signals get sent to reservoirs and which signal gets sent to the DAC depending on the operating mode. The Verilog description for such a regulator is in Fig. B.5.

The code directs the wires and registers **v_train**, **adc**, **adc_delay**, **v1**, **v2**, and **r**, which are the RAM-stored training signal, the measured ADC value, the delayed ADC value, the first reservoir output, the second reservoir output, and the RAM-stored reference signal. It produces the wires **dac**, **u1**, and **u2**, which are the DAC output, the input to the first reservoir, and the input to the second reservoir, respectively. How these signals are directed depends on the register **mode** according to

- 0 : training the first layer,
- 1 : controlling with the first layer,
- 2 : training the second layer,
- 3 : controlling with the second layer.


```

1 ...
2 wire [11:0] u1 [4];
3 wire [11:0] u2 [4];
4 wire [11:0] adc [2];
5 wire [11:0] adc_delay [2];
6 wire [15:0] dac;
7 wire [15:0] v1;
8 wire [15:0] v2;
9 reg [11:0] r [2];
10 reg [15:0] v_train;
11 reg [1:0] mode;
12
13 always @(*) begin
14     case (mode)
15         0 : begin
16             dac = v_train;
17             u1[0:1] = adc;
18             u1[2:3] = adc_delay;
19             u2[0:1] = adc;
20             u2[2:3] = adc_delay;
21         end
22         1 : begin
23             dac = v1;
24             u1[0:1] = r;
25             u1[2:3] = adc;
26             u2[0:1] = adc;
27             u2[2:3] = adc_delay;
28         end
29         2 : begin
30             dac = v1 + v_train;
31             u1[0:1] = r;
32             u1[2:3] = adc;
33             u2[0:1] = adc;
34             u2[2:3] = adc_delay;
35         end
36         3 : begin
37             dac = v1 + v2;
38             u1[0:1] = r;
39             u1[2:3] = adc;
40             u2[0:1] = r;
41             u2[2:3] = adc;
42         end
43     endcase
44 end
45 ...

```

FIGURE B.5: Verilog code for the multiplying by hard-coded weights. Note that this is just a snippet of code that might go inside a top-level module, depending on design. It takes various signals and directs them appropriately, depending on the operating mode.